

BỘ CÔNG THƯƠNG
TRƯỜNG ĐẠI HỌC KINH TẾ - KỸ THUẬT CÔNG NGHIỆP

KHÓA LUẬN TỐT NGHIỆP

TÊN ĐỀ TÀI KHÓA LUẬN:
XÂY DỰNG HẠ TẦNG TRUYỀN THÔNG VÀ QUẢN LÝ DỮ LIỆU
CHO MẠNG LƯỚI CẢM BIẾN CỦA HỆ THỐNG CẢNH BÁO CHÁY

Ngành đào tạo: Công nghệ Kỹ thuật Điện tử - Viễn thông
Mã số ngành: 7510302

Họ và tên sinh viên: Nguyễn Thị Hằng
Hoàng Minh Phương

Người hướng dẫn khóa luận tốt nghiệp
TS. Vũ Ngọc Dân

Hà Nội - 2026

BỘ CÔNG THƯƠNG
TRƯỜNG ĐẠI HỌC KINH TẾ - KỸ THUẬT CÔNG NGHIỆP

KHÓA LUẬN TỐT NGHIỆP

TÊN ĐỀ TÀI KHÓA LUẬN:
XÂY DỰNG HẠ TẦNG TRUYỀN THÔNG VÀ QUẢN LÝ DỮ LIỆU
CHO MẠNG LƯỚI CẢM BIẾN CỦA HỆ THỐNG CẢNH BÁO CHÁY

Ngành đào tạo: Công nghệ Kỹ thuật Điện tử - Viễn thông
Mã số ngành: 7510302

Họ và tên sinh viên: Nguyễn Thị Hằng
Hoàng Minh Phương

Người hướng dẫn khóa luận tốt nghiệp
TS. Vũ Ngọc Dân

Hà Nội - 2026

LỜI CAM ĐOAN

Chúng em xin cam đoan nội dung được trình bày trong đề tài “*Xây dựng hạ tầng truyền thông và quản lý dữ liệu cho mạng lưới cảm biến của hệ thống cảnh báo cháy*” là quá trình nghiên cứu do chính nhóm em thực hiện dưới sự hướng dẫn của giảng viên – TS. Vũ Ngọc Dân.

Nội dung, số liệu và kết quả trong báo cáo của chúng em là hoàn toàn trung thực, được xử lý và thu thập qua quá trình nghiên cứu thực tế. Có tham khảo và sử dụng các tài liệu, thông tin đã được đăng tải trên các tạp chí, website theo đúng danh mục tài liệu tham khảo.

Chúng em chịu toàn bộ trách nhiệm về nội dung được trình bày trong quyển báo cáo này.

Hà Nội, ngày 05 tháng 05 năm 2026

Nhóm sinh viên thực hiện!

LỜI CẢM ƠN

Trong suốt quá trình tìm hiểu và nghiên cứu đề tài “*Xây dựng hạ tầng truyền thông và quản lý dữ liệu cho mạng lưới cảm biến của hệ thống cảnh báo cháy*”, chúng em đã nhận được rất nhiều sự quan tâm, hướng dẫn và hỗ trợ từ các thầy cô, cùng với sự quan tâm, giúp đỡ từ gia đình và bạn bè.

Qua đây, chúng em xin bày tỏ lòng biết ơn sâu sắc đến Nhà trường vì đã tạo điều kiện cho chúng em được học tập phát triển, cùng các Thầy Cô giáo trong khoa Điện Tử và Kỹ Thuật Máy Tính, là những người truyền thụ tri thức, kỹ năng, kinh nghiệm cho chúng em trong suốt những năm học vừa qua. Đó chính là nền tảng vững chắc cho chúng em trong quá trình thực hiện đề tài này.

Đặc biệt, em xin được gửi lời cảm ơn chân thành đến người đã trực tiếp hướng dẫn và đồng hành với chúng em trong quá trình nghiên cứu đề tài này là giảng viên – TS. Vũ Ngọc Dân. Thầy không những tận tình chỉ bảo, truyền đạt cho chúng em về những kiến thức chuyên môn và những kinh nghiệm nghiên cứu, mà còn định hướng, sửa chữa cho chúng em về hướng đi, những lỗi sai còn mắc phải trong quá trình thực hiện đề tài. Sự nghiêm túc trong chuyên môn cùng sự tận tâm của thầy đã giúp em từng bước hoàn thiện nội dung khóa luận một cách tốt hơn.

Cuối cùng, tôi xin chân thành cảm ơn các thành viên trong nhóm cùng tất cả mọi người đã giúp đỡ và đóng góp ý kiến để tôi hoàn thành đồ án này. Mặc dù đã có nhiều cố gắng trong quá trình thực hiện, song do kiến thức và kinh nghiệm thực tiễn của bản thân còn hạn chế nên khóa luận khó tránh khỏi những thiếu sót. Nhưng nhờ có sự đồng hành của các bạn cùng nhau sửa chữa và cải thiện nên đề tài đã hoàn thành đúng thời gian quy định.

Lời kết, chúng em xin kính chúc quý thầy cô luôn dồi dào sức khỏe, hạnh phúc và ngày càng thành công trong công việc. Em xin chân thành cảm ơn!

Hà Nội, ngày 05 tháng 05 năm 2026

Nhóm sinh viên thực hiện!

MỤC LỤC

LỜI CAM ĐOAN	ii
LỜI CẢM ƠN	iii
MỤC LỤC	iv
CÁC TỪ VIẾT TẮT	vii
DANH MỤC CÁC BẢNG.....	ix
DANH MỤC CÁC HÌNH.....	x
PHẦN MỞ ĐẦU	1
CHƯƠNG 1: TỔNG QUAN VỀ HẠ TẦNG MẠNG IoT	4
1.1. KIẾN TRÚC TỔNG THỂ CỦA MỘT HỆ THỐNG IOT.....	4
1.1.1. Device Layer (Lớp thiết bị).....	5
1.1.2. Network Layer (Lớp mạng)	5
1.1.3. Server Layer (Lớp ứng dụng)	5
1.2. VAI TRÒ CỦA BROKER VÀ CƠ SỞ DỮ LIỆU TRONG HỆ THỐNG CẢNH BÁO THỜI GIAN THỰC.	6
1.2.1. Vai trò của Broker trong hệ thống cảnh báo thời gian thực	6
1.2.2. Vai trò của Cơ sở dữ liệu trong hệ thống cảnh báo	6
1.2.3. Quy trình hoạt động trong hệ thống cảnh báo thời gian thực	7
CHƯƠNG 2: CÁC GIAO THỨC TRUYỀN THÔNG VÀ CÔNG NGHỆ LƯU TRỮ	8
2.1. CÁC GIAO THỨC TRUYỀN THÔNG	8
2.1.1. Giao thức MQTT.....	8
2.1.2. Giao thức HTTP	12
2.1.3. Giao thức CoAP	13
2.1.4. Kết Luận.....	15
2.2. CƠ SỞ DỮ LIỆU CHO IOT:	15
2.2.1. SQL và NoSQL	15
2.2.2. So sánh cơ sở dữ liệu quan hệ (SQL) và NoSQL	20
2.2.3. Lựa chọn giải pháp lưu trữ dữ liệu lịch sử cảm biến	22

CHƯƠNG 3: THIẾT KẾ VÀ TRIỂN KHAI MÁY CHỦ TRUNG GIAN (Broker) VÀ CƠ SỞ DỮ LIỆU	24
3.1. LỰA CHỌN PHẦN CỨNG MÁY CHỦ RASPBERRY PI.	24
3.1.1. Giới thiệu Raspberry Pi 3B+.....	24
3.1.2. Lý do lựa chọn Raspberry Pi 3B+.....	25
3.2. CÀI ĐẶT VÀ CẤU HÌNH MQTT BROKER.....	26
3.2.1. Vai trò của MQTT Broker trong kiến trúc hệ thống	26
3.2.2. Cài đặt Mosquitto Broker trên Raspberry Pi.....	27
3.2.3. Cấu hình broker cục bộ và thiết lập bảo mật	30
3.2.4. Thiết kế cấu trúc topic và cấu hình bridge lên HiveMQ Cloud....	31
3.2.5. Kiểm tra hoạt động của broker sau cấu hình.....	32
3.3. THIẾT KẾ CƠ SỞ DỮ LIỆU:	33
3.3.1. Mục tiêu và nguyên tắc thiết kế	34
3.3.2. Cấu trúc các bảng dữ liệu.....	35
3.3.3. Quan hệ dữ liệu và vai trò của các bảng trong hệ thống.....	42
3.3.4. Kiểm tra và đánh giá thiết kế cơ sở dữ liệu	43
3.4. XÂY DỰNG PHẦN TRUNG GIAN ĐỂ TRUY XUẤT DỮ LIỆU. ..	44
3.4.1. Vai trò và môi trường triển khai của chương trình trung gian	45
3.4.2. Thiết kế luồng xử lý bản tin MQTT.....	47
3.4.3. Cơ chế ghi dữ liệu vào MariaDB	49
3.4.4. Triển khai chạy nền và kiểm tra hoạt động.....	53
CHƯƠNG 4: THỬ NGHIỆM VÀ ĐÁNH GIÁ HIỆU NĂNG MẠNG.....	56
4.1. KỊCH BẢN KIỂM THỬ.....	56
4.1.1. Mục đích và phạm vi kiểm thử	56
4.1.2. Môi trường, điều kiện và nguyên tắc kiểm thử.....	57
4.1.3. Các kịch bản kiểm thử.....	58
4.1.4. Tổng hợp kịch bản và tiêu chí đánh giá.....	61
4.2. ĐO ĐẠC ĐỘ TRỄ TỪ LÚC CẢM BIẾN PHÁT HIỆN ĐẾN KHI SERVER NHẬN ĐƯỢC.....	62

4.2.1. Mục đích và phương pháp đo đạc	62
4.2.2. Mô hình tuyến truyền và các chặng đo độ trễ.....	63
4.2.3. Kết quả đo độ trễ theo từng node	66
4.2.4. Nhận xét chung	68
4.3. ĐÁNH GIÁ ĐỘ ỔN ĐỊNH VÀ KHẢ NĂNG CHỊU TẢI CỦA BROKER.....	68
4.3.1. Mục đích và kịch bản kiểm thử tải.....	68
4.3.2. Phương pháp đánh giá mất mát gói tin và tài nguyên hệ thống....	70
4.3.3. Kết quả kiểm thử khả năng chịu tải của broker	71
4.3.4. Nhận xét độ ổn định của hệ thống khi chịu tải	73
KẾT LUẬN VÀ ĐỀ NGHỊ.....	75
TÀI LIỆU THAM KHẢO.....	76
PHỤ LỤC	79

CÁC TỪ VIẾT TẮT

Từ viết tắt	Cụm từ đầy đủ	Nghĩa tiếng Việt / Giải thích
ACID	Atomicity, Consistency, Isolation, Durability	Các thuộc tính đảm bảo giao dịch trong cơ sở dữ liệu
API	Application Programming Interface	Giao diện lập trình ứng dụng
BASE	Basically Available, Soft state, Eventual consistency	Mô hình nhất quán thường dùng trong NoSQL
CoAP	Constrained Application Protocol	Giao thức ứng dụng cho thiết bị tài nguyên hạn chế
DBMS	Database Management System	Hệ quản trị cơ sở dữ liệu
DTLS	Datagram Transport Layer Security	Giao thức bảo mật cho dữ liệu truyền qua UDP
ESP32	Espressif ESP32	Dòng vi điều khiển tích hợp Wi-Fi/Bluetooth
GPIO	General Purpose Input/Output	Chân vào/ra đa dụng
HTTP	HyperText Transfer Protocol	Giao thức truyền tải siêu văn bản
IP	Internet Protocol	Giao thức Internet / địa chỉ mạng
IoT	Internet of Things	Internet vạn vật
LAN	Local Area Network	Mạng cục bộ
M2M	Machine to Machine	Truyền thông giữa máy với máy
MQTT	Message Queuing Telemetry Transport	Giao thức truyền thông điệp nhẹ trong IoT
NAT	Network Address Translation	Cơ chế biên dịch địa chỉ mạng
NoSQL	Not Only SQL	Cơ sở dữ liệu phi quan hệ / không chỉ SQL
OS	Operating System	Hệ điều hành
PoE	Power over Ethernet	Cấp nguồn qua Ethernet
QoS	Quality of Service	Chất lượng dịch vụ trong MQTT
RAM	Random Access Memory	Bộ nhớ truy cập ngẫu nhiên
RDBMS	Relational Database Management System	Hệ quản trị cơ sở dữ liệu quan hệ
SQL	Structured Query Language	Ngôn ngữ truy vấn có cấu trúc
SSL	Secure Sockets Layer	Giao thức bảo mật kết nối mạng
TCP/IP	Transmission Control Protocol / Internet Protocol	Bộ giao thức truyền thông Internet

TLS	Transport Layer Security	Giao thức bảo mật tầng truyền tải
UDP	User Datagram Protocol	Giao thức truyền dữ liệu không kết nối
URI	Uniform Resource Identifier	Định danh tài nguyên thống nhất

DANH MỤC CÁC BẢNG

Bảng 2.1: Tổng hợp những khác biệt cốt lõi giữa SQL (Relational Database Management System – RDBMS) và NoSQL.	21
Bảng 3.1: Một số lệnh kiểm tra hoạt động của hệ thống MQTT Broker.....	33
Bảng 3.2: Mô tả các trường trong bảng lưu trữ thông tin thiết bị.....	36
Bảng 3.3: Mô tả các trường trong bảng mqtt_topic_history.....	38
Bảng 3.4: Cấu trúc bảng lưu trạng thái hiện tại theo phòng	40
Bảng 3.5: Tổng hợp chức năng các bảng trong cơ sở dữ liệu.....	43
Bảng 4.1: Tổng hợp các kịch bản kiểm thử hệ thống	61
Bảng 4.3: Tổng hợp độ trễ trung bình theo từng chặng	66
Bảng 4.4: Cấu trúc topic trong kịch bản kiểm thử tải.....	69
Bảng 4.5: Thông số kịch bản kiểm thử tải broker.....	69
Bảng 4.6: Kết quả đối chiếu số lượng bản tin trong MariaDB	71
Bảng 4.7: Số lượng bản tin lưu theo từng node	72
Bảng 4.8: Kết quả tổng hợp độ trễ truyền dữ liệu của chặng 1	79
Bảng 4.9: Kết quả tổng hợp độ trễ truyền dữ liệu của chặng 2	79
Bảng 4.10: Kết quả tổng hợp độ trễ truyền dữ liệu của chặng 3	79

DANH MỤC CÁC HÌNH

Hình 1.1: Mô hình kiến trúc IoT	4
Hình 2.1: Kiến trúc giao thức MQTT	8
Hình 2.2: Sơ đồ kết nối hoạt động của MQTT	10
Hình 2.3: Kiến trúc giao thức HTTP.....	12
Hình 2.4: Các lớp của CoAP.....	14
Hình 3.1: Module Raspberry Pi	24
Hình 3.2: Sơ đồ vị trí của MQTT Broker trong hệ thống cảnh báo cháy	27
Hình 3.3: Phần mềm Raspberry Pi.....	27
Hình 3.4: Sử dụng VNC	28
Hình 3.5: VNC điều khiển Raspberry	28
Hình 3.6: Kết thúc quá trình đăng nhập	29
Hình 3.7: Tập cấu hình broker cục bộ của hệ thống.	31
Hình 3.8: Cấu trúc topic MQTT của hệ thống báo cháy theo từng khu vực và từng thông số	32
Hình 3.9: Sơ đồ khởi hoạt động của chương trình trung gian mqtt_to_sql.py	45
Hình 4.1: Mô hình truyền dữ liệu từ node cảm biến đến server cục bộ và nền tảng cloud	64
Hình 4.2: Tổng độ trễ trung bình từ Node đến HiveMQ Cloud	66
Hình 4.3: Phân bố độ trễ trung bình qua các chặng xử lý.....	67
Hình 4.4: Biểu đồ so sánh số bản tin kỳ vọng và số bản tin lưu thực tế trong mqtt_topic_history	72

PHẦN MỞ ĐẦU

1. Đặt vấn đề

Quá trình đô thị hóa và công nghiệp hóa diễn ra với tốc độ nhanh chóng đã làm gia tăng đáng kể mật độ dân cư và số lượng công trình xây dựng, kéo theo nguy cơ cháy nổ luôn hiện hữu và ngày càng trở nên phức tạp tại nhiều loại hình không gian như nhà ở, nhà xưởng, kho bãi hay trung tâm thương mại. Hậu quả từ các vụ hỏa hoạn không chỉ dừng lại ở thiệt hại vật chất mà còn đe dọa trực tiếp đến tính mạng con người, gây ra những tác động tiêu cực đến an ninh trật tự và sự phát triển bền vững của xã hội. Trước thực trạng đó, việc nghiên cứu và phát triển các hệ thống có khả năng phát hiện sớm sự cố, truyền tải thông tin cảnh báo nhanh chóng và quản lý dữ liệu giám sát một cách hệ thống đã trở thành một đòi hỏi cấp thiết.

Sự phát triển mạnh mẽ của công nghệ Internet vạn vật (IoT) đã tạo tiền đề cho việc ứng dụng rộng rãi các mạng cảm biến không dây vào lĩnh vực giám sát môi trường và phòng cháy chữa cháy. Trong các hệ thống này, những nút cảm biến đầu cuối liên tục thu thập các thông số trọng yếu như nhiệt độ, nồng độ khói, khí gas hay sự hiện diện của ngọn lửa, sau đó truyền tải dữ liệu thô về trung tâm xử lý. Tuy nhiên, hiệu quả vận hành thực tế của toàn bộ giải pháp không chỉ phụ thuộc vào độ nhạy của cảm biến mà còn được quyết định bởi độ tin cậy của **hạ tầng truyền thông** và **cơ chế quản trị dữ liệu** ở lớp máy chủ trung gian. Đây chính là thành phần cốt lõi đảm bảo dòng dữ liệu cảm biến được luân chuyển liên tục, toàn vẹn, có độ trễ thấp và sẵn sàng cho các tác vụ phân tích, truy vấn lịch sử về sau.

2. Mục tiêu nghiên cứu

Xuất phát từ những yêu cầu kỹ thuật đặc thù đó, đề tài “**Xây dựng hạ tầng truyền thông và quản lý dữ liệu cho mạng lưới cảm biến của hệ thống cảnh báo cháy**” được lựa chọn làm trọng tâm nghiên cứu. Nội dung nghiên cứu tập trung vào việc thiết kế mô hình kết nối tối ưu giữa các thiết bị đầu cuối và máy chủ trung tâm dựa trên giao thức MQTT, xây dựng cơ chế truyền nhận dữ liệu đảm bảo tính liên tục và chính xác, đồng thời tổ chức lưu trữ và quản lý khối lượng dữ liệu chuỗi thời gian phát sinh từ mạng cảm biến. Mục tiêu hướng đến là tạo lập một hạ tầng kỹ thuật đủ mạnh để hỗ trợ hiệu quả cho công tác giám sát an toàn và cảnh báo sớm nguy cơ hỏa hoạn. Kết quả đạt được không chỉ có ý nghĩa minh chứng về mặt học thuật cho các giải pháp truyền thông trong

mạng cảm biến không dây mà còn mang giá trị ứng dụng cao, góp phần nâng cao năng lực phòng ngừa và ứng phó sự cố trong điều kiện triển khai công nghệ hiện đại.

3. Đối tượng và phạm vi nghiên cứu

Đối tượng nghiên cứu: Hạ tầng truyền thông và hệ thống quản lý dữ liệu phía máy chủ trong mạng cảm biến không dây dành cho cảnh báo cháy. Trọng tâm là giao thức MQTT, MQTT Broker, mô hình truyền nhận dữ liệu và cơ sở dữ liệu chuỗi thời gian phục vụ lưu trữ, truy vấn thông tin đo đạc.

Phạm vi nghiên cứu: Đề tài tập trung vào việc thiết kế và triển khai lớp truyền thông cùng lớp quản lý dữ liệu ở máy chủ trung tâm, không đi sâu vào chế tạo phần cứng nút cảm biến hay phát triển thuật toán phát hiện cháy. Các thực nghiệm đánh giá được tiến hành ở quy mô phòng thí nghiệm nhằm kiểm chứng các chỉ tiêu hiệu năng mạng.

4. Phương pháp nghiên cứu

Để đạt được các mục tiêu đặt ra, nghiên cứu được thực hiện theo quy trình kết hợp giữa nghiên cứu lý thuyết và thực nghiệm:

Nghiên cứu lý thuyết: Tổng quan kiến trúc hạ tầng mạng IoT, làm rõ vị trí và vai trò của lớp truyền thông; phân tích sâu các giao thức truyền thông tiêu biểu (đặc biệt là MQTT) cùng các giải pháp lưu trữ dữ liệu phù hợp với ứng dụng IoT.

Thiết kế và triển khai: Cấu hình MQTT Broker và xây dựng máy chủ trung tâm; thiết kế cơ sở dữ liệu phục vụ quản lý thiết bị và lưu trữ dữ liệu đo đạc, bảo đảm khả năng tiếp nhận dữ liệu thời gian thực và truy vấn lịch sử.

Thực nghiệm và đánh giá: Tiến hành thử nghiệm hệ thống, đo lường các chỉ số hiệu năng như độ trễ truyền thông, tính ổn định và khả năng đáp ứng của hạ tầng; từ đó rút ra kết luận về độ tin cậy và tính khả thi của giải pháp.

Tổng kết: Trên cơ sở kết quả đạt được, đánh giá ý nghĩa học thuật và giá trị ứng dụng của nghiên cứu, đồng thời đề xuất hướng phát triển tiếp theo.

Luận văn được tổ chức thành bốn chương chính. **Chương 1** trình bày tổng quan về kiến trúc hạ tầng mạng IoT, làm rõ vị trí và vai trò của lớp truyền thông trong một hệ thống giám sát hoàn chỉnh. **Chương 2** đi sâu phân tích các giao thức truyền thông tiêu biểu và các giải pháp lưu trữ dữ liệu phù hợp với đặc thù ứng dụng IoT. **Chương 3** tập trung mô tả quá trình thiết kế và triển khai máy

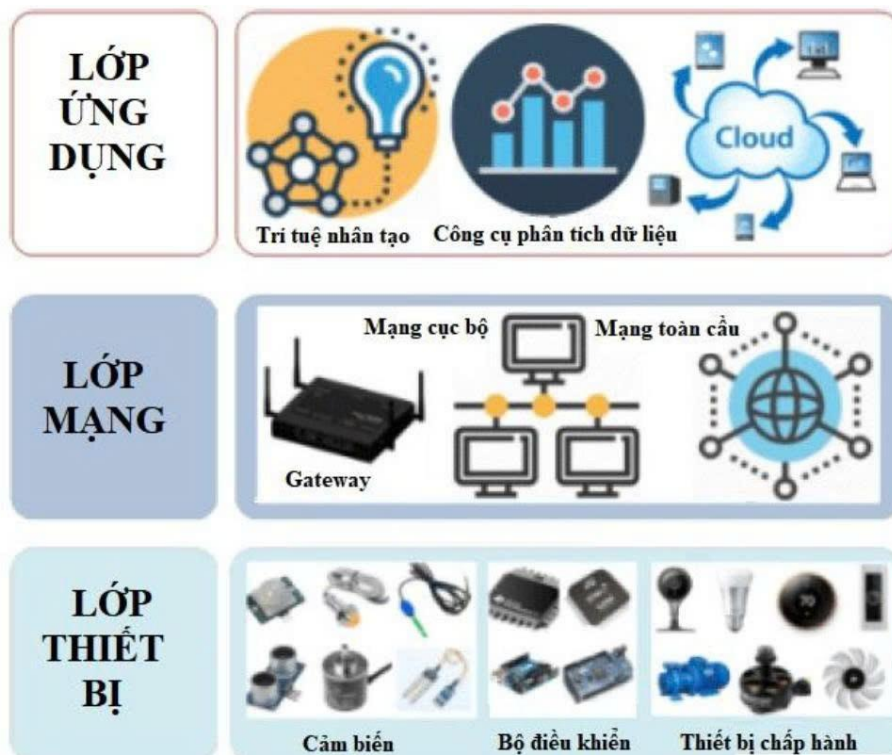
chủ trung tâm, bao gồm việc cấu hình MQTT Broker và xây dựng cơ sở dữ liệu phục vụ quản lý thiết bị cũng như dữ liệu đo đạc. **Chương 4** dành cho công tác thử nghiệm và đánh giá các chỉ số hiệu năng mạng, từ đó rút ra những kết luận về độ ổn định và khả năng đáp ứng thực tế của hạ tầng đã xây dựng.

CHƯƠNG 1: TỔNG QUAN VỀ HẠ TẦNG MẠNG IOT

1.1. KIẾN TRÚC TỔNG THỂ CỦA MỘT HỆ THỐNG IOT.

Internet vạn vật (Internet of Things – IoT) được định nghĩa là một hệ thống mạng lưới bao gồm các thiết bị vật lý có khả năng kết nối Internet, qua đó cho phép tự động hóa quá trình thu thập, trao đổi và xử lý dữ liệu mà không đòi hỏi sự can thiệp trực tiếp và thường xuyên của con người [4], [6]. Trong mô hình này, các thực thể vật lý như máy móc công nghiệp, phương tiện giao thông hay đồ dùng sinh hoạt được tích hợp các vi mạch cảm biến hoặc cơ cấu chấp hành, từ đó có được năng lực cảm nhận môi trường xung quanh, truyền tải thông tin đến hệ thống trung tâm và thực thi các mệnh lệnh điều khiển nhận được [4], [7].

Điểm cốt lõi của IoT không chỉ nằm ở khả năng kết nối thiết bị với Internet mà còn ở việc khai thác dữ liệu thu thập được nhằm phục vụ các mục tiêu như giám sát từ xa, điều khiển tự động, quy trình vận hành tối ưu hóa và nâng cao hiệu quả hoạt động của hệ thống [2]. Một hệ thống IoT được điều chỉnh hoàn chỉnh thường bao gồm các thành phần cơ bản như thiết bị (đồ vật), kết nối (kết nối), dữ liệu (dữ liệu) và ứng dụng (ứng dụng), tạo thành một chu trình kín từ việc thu thập đến xử lý và sử dụng dữ liệu [7].



Hình 1.1: Mô hình kiến trúc IoT

Về mặt tổ chức hệ thống, kiến trúc IoT thường được mô hình hóa thông qua cấu trúc phân tầng, trong đó phổ biến nhất là mô hình ba lớp nền tảng bao gồm: Lớp Thiết bị (Device Layer), Lớp Mạng (Network Layer) và Lớp Ứng dụng/Máy chủ (Application/Server Layer). Mô hình này không chỉ giúp phân định rõ ràng vai trò chức năng của từng khối thành phần mà còn tạo điều kiện thuận lợi cho việc thiết kế, phát triển và bảo trì hệ thống một cách có hệ thống [3].

1.1.1. Device Layer (Lớp thiết bị)

Lớp thiết bị là nền tảng trong kiến trúc IoT, nơi các biến thể và cơ chế chấp hành trực tiếp tương thích với môi trường vật lý. Nhiệm vụ chính của lớp này là thu thập dữ liệu từ môi trường như nhiệt độ, độ ẩm, ánh sáng, chuyển động,... hoặc thực hiện các hoạt động điều khiển như đóng/ngắt role, điều khiển động cơ hay van.

Các thành phần cấu hình của lớp thiết bị bao gồm cảm biến, cơ chế chấp nhận hành động và các vi điều khiển hoặc hệ thống nhúng như Arduino, ESP32 hay STM32. Dữ liệu được tạo ra ở lớp này đóng vai trò là đầu vào cho toàn bộ hệ thống IoT, quyết định chất lượng và độ chính xác của các quy trình xử lý phía sau [4].

1.1.2. Network Layer (Lớp mạng)

Vai trò: lớp mạng chịu trách nhiệm vận chuyển dữ liệu từ các thiết bị cảm biến đến các hệ thống xử lý trung tâm hoặc đám mây. Lớp này là “xương sống” của toàn bộ kiến trúc IoT, đảm bảo mọi thiết bị có thể giao tiếp với nhau một cách an toàn và ổn định.

Các công nghệ/kênh truyền như: Wi-Fi, Bluetooth, ZigBee, 3G/4G/5G, NB-IoT, LoRa/LoRaWAN...

1.1.3. Server Layer (Lớp ứng dụng)

Đây là tầng cao nhất trong kiến trúc, nơi dữ liệu được tập hợp, lưu trữ và khai thác giá trị. Lớp này thường được triển khai trên các nền tảng điện toán đám mây (Cloud Computing) hoặc máy chủ trung tâm.

- **Xử lý và lưu trữ:** Dữ liệu lớn (Big Data) từ hàng ngàn thiết bị được phân tích bằng các thuật toán máy học để đưa ra các dự báo hoặc quyết định điều khiển thông minh.

- **Giao diện người dùng (Dashboard):** Cung cấp các công cụ trực quan hóa dữ liệu (Web, App) giúp người vận hành dễ dàng theo dõi và tương tác với

hệ thống. Sự phối hợp chặt chẽ giữa ba lớp này tạo nên một chu trình khép kín, từ việc cảm nhận môi trường đến việc ra quyết định và phản hồi thực tế [6], [7].

1.2. VAI TRÒ CỦA BROKER VÀ CƠ SỞ DỮ LIỆU TRONG HỆ THỐNG CẢNH BÁO THỜI GIAN THỰC.

1.2.1. Vai trò của Broker trong hệ thống cảnh báo thời gian thực

Broker là máy chủ trung gian có nhiệm vụ tiếp nhận và phân phối dữ liệu giữa các thiết bị IoT và các hệ thống xử lý hoặc ứng dụng. Trong các hệ thống IoT hiện nay, Broker thường hoạt động dựa trên giao thức MQTT với mô hình truyền thông **publish/subscribe**, giúp tối ưu hóa việc truyền dữ liệu trong môi trường có nhiều thiết bị kết nối.

Cụ thể, các thiết bị như ESP32, cảm biến hoặc module GPS sẽ gửi dữ liệu lên Broker dưới dạng các thông điệp theo từng chủ đề (topic). Broker sau đó sẽ phân phối dữ liệu này đến các thành phần đã đăng ký nhận (subscriber) như server xử lý hoặc ứng dụng giám sát.

Các chức năng chính của Broker bao gồm:

- **Tiếp nhận dữ liệu từ thiết bị IoT:** Các thiết bị gửi dữ liệu như nhiệt độ, vị trí hoặc trạng thái hoạt động lên Broker để xử lý.
- **Phân phối dữ liệu theo cơ chế publish/subscribe:** Thiết bị thực hiện publish dữ liệu lên topic, trong khi các hệ thống khác subscribe để nhận dữ liệu theo thời gian thực.
- **Đảm bảo truyền dữ liệu thời gian thực:** Khi phát hiện dữ liệu vượt ngưỡng (ví dụ nhiệt độ cao bất thường), Broker có thể chuyển tiếp thông tin ngay lập tức đến hệ thống cảnh báo.
- **Giảm tải cho server:** Broker đóng vai trò trung gian, giúp giảm số lượng kết nối trực tiếp giữa thiết bị và server, từ đó nâng cao khả năng mở rộng của hệ thống.

Một số Broker phổ biến hiện nay bao gồm Mosquitto, HiveMQ và EMQX, được sử dụng rộng rãi trong các hệ thống IoT nhờ hiệu năng cao và khả năng mở rộng tốt [9].

1.2.2. Vai trò của Cơ sở dữ liệu trong hệ thống cảnh báo

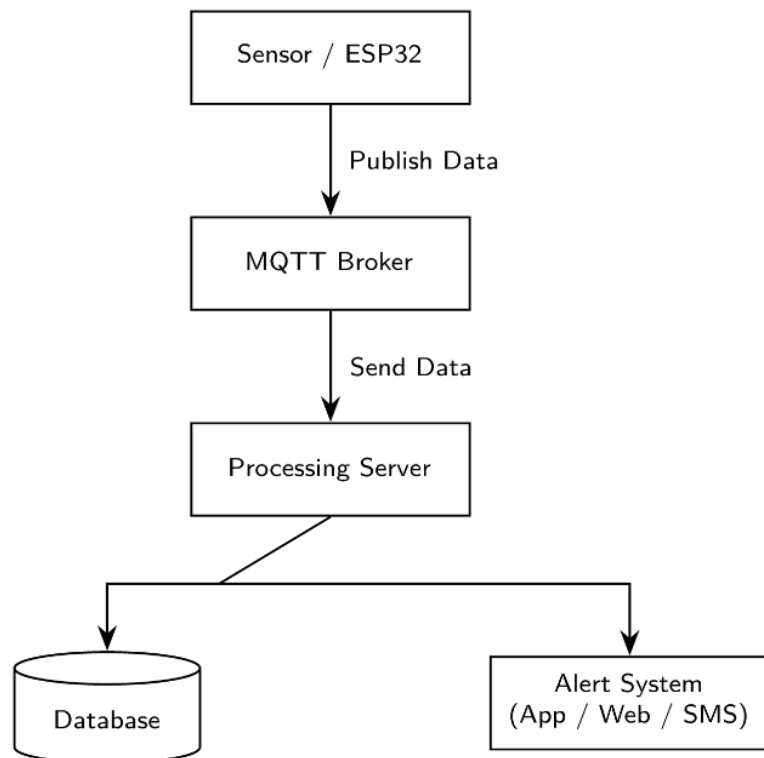
Cơ sở dữ liệu (Database) dùng để lưu trữ và quản lý dữ liệu lâu dài của hệ thống IoT.

Chức năng chính

- Lưu trữ dữ liệu cảm biến: Nhiệt độ; Vị trí GPS; Trạng thái thiết bị
- Lưu lịch sử cảnh báo: Khi nào xảy ra cảnh báo; Thiết bị nào gây ra cảnh báo
- Phân tích dữ liệu: Thống kê số lần cảnh báo; Phân tích xu hướng dữ liệu
- Hiển thị dữ liệu trên dashboard: Web; Ứng dụng điện thoại
- Ví dụ cơ sở dữ liệu IoT: Firebase; MySQL; MongoDB

1.2.3. Quy trình hoạt động trong hệ thống cảnh báo thời gian thực

Quy trình chung:



Dựa trên sơ đồ logic, quy trình hoạt động của hệ thống được chuẩn hóa qua các bước nhằm tối thiểu hóa thời gian phản hồi:

1. **Thu thập & Truyền tải:** ESP32 đọc dữ liệu từ cảm biến và thực hiện lệnh Publish Data lên MQTT Broker.
2. **Chuyển tiếp:** Broker thực hiện Send Data đến Server xử lý (Processing Server) ngay khi nhận được gói tin.
3. **Xử lý Logic:** Server thực hiện kiểm tra điều kiện an toàn:

$$\text{Kết quả} = \begin{cases} \text{kích_hoạt_Alert} , & \text{nếu Giá trị} > \text{Ngưỡng} \\ \text{duy_trì_theo_dõi} , & \text{nếu Giá trị} \leq \text{Ngưỡng} \end{cases}$$

4. **Lưu trữ & Phản hồi:** Dữ liệu đồng thời được đẩy vào **Database** và gửi thông báo đến người dùng qua các kênh App/Web/SMS [2], [7].

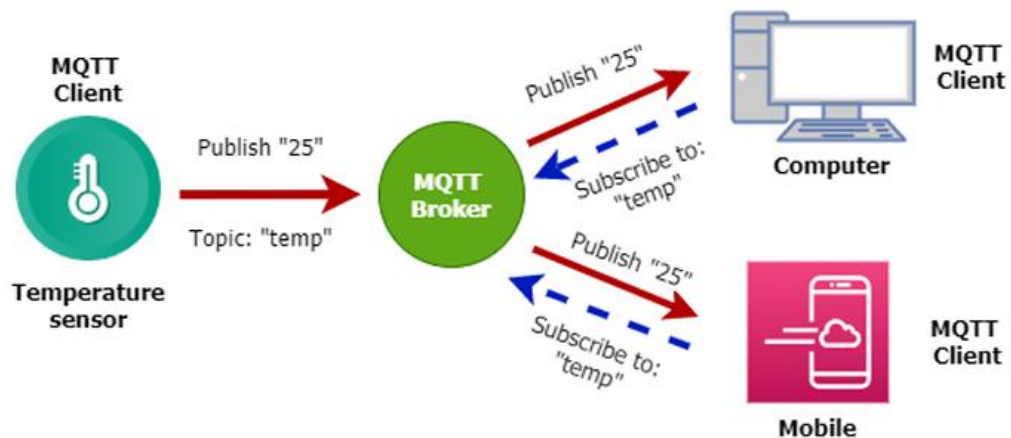
CHƯƠNG 2: CÁC GIAO THỨC TRUYỀN THÔNG VÀ CÔNG NGHỆ LƯU TRỮ

2.1. CÁC GIAO THỨC TRUYỀN THÔNG

2.1.1. Giao thức MQTT

a) Khái niệm cơ bản và các thành phần chính

MQTT (Message Queuing Telemetry Transport) là một giao thức truyền thông điệp hạng nhẹ (lightweight), hoạt động theo mô hình xuất bản/đăng ký (publish/subscribe), được chuẩn hóa theo tiêu chuẩn ISO/IEC 20922 và vận hành trên nền tảng giao thức TCP/IP [1]. Với đặc điểm thiết kế tối giản, MQTT đặc biệt thích hợp cho các thiết bị IoT, các nút cảm biến và các bộ vi điều khiển có tài nguyên hạn chế, hoạt động trong môi trường mạng có băng thông thấp hoặc không ổn định [1].



Hình 2.1: Kiến trúc giao thức MQTT

Kiến trúc của MQTT được xây dựng dựa trên ba thực thể cốt lõi: **Client**, **Broker** và **Topic** [1].

- **Client:** Là các thiết bị hoặc ứng dụng đầu cuối tham gia vào quá trình truyền nhận dữ liệu. Về mặt chức năng, Client được phân thành hai nhóm: **Publisher** (bên xuất bản) thực hiện việc gửi dữ liệu lên Broker, và **Subscriber** (bên đăng ký) thực hiện việc đăng ký để nhận dữ liệu từ Broker. Một Client có thể đảm nhiệm đồng thời cả hai vai trò này trong cùng một phiên kết nối [1].

- **Broker:** Đóng vai trò là máy chủ trung gian, chịu trách nhiệm tiếp nhận toàn bộ dữ liệu được gửi đến từ các Publisher và thực hiện nhiệm vụ chuyển

tiếp dữ liệu đó đến các Subscriber tương ứng dựa trên cơ chế quản lý chủ đề [1].

- **Topic:** Một định danh dạng chuỗi ký tự, đóng vai trò như “chủ đề” để phân loại và định tuyến nội dung dữ liệu. Cấu trúc Topic có tính phân cấp, cho phép nhóm và lọc thông điệp một cách linh hoạt.

Bên cạnh các thành phần cơ bản, hoạt động của MQTT còn được định nghĩa bởi một số khái niệm then chốt sau:

- **Publish:** Hành động mà một Client thực hiện để gửi một gói dữ liệu (message) lên một chủ đề (Topic) xác định trên Broker.

- **Subscribe:** Hành động mà một Client thực hiện để đăng ký nhận dữ liệu từ một hoặc nhiều Topic mà nó quan tâm.

- **Session:** Thể hiện trạng thái của một kết nối giữa một Client và Broker. Tất cả các tương tác giữa Client và Broker đều diễn ra trong phạm vi của một phiên làm việc (session).

- **Subscription:** Thể hiện mối liên kết giữa một Client và một Topic, được thiết lập khi Client gửi yêu cầu Subscribe thành công đến Broker.

b) Chất lượng dịch vụ (Quality of Service – QoS)

Nhằm đáp ứng các yêu cầu khác nhau về độ tin cậy trong truyền tải dữ liệu, MQTT định nghĩa ba mức chất lượng dịch vụ (Quality of Service – QoS). Các mức QoS này cho phép nhà phát triển đánh đổi giữa tốc độ truyền tải và độ bảo đảm của thông điệp:

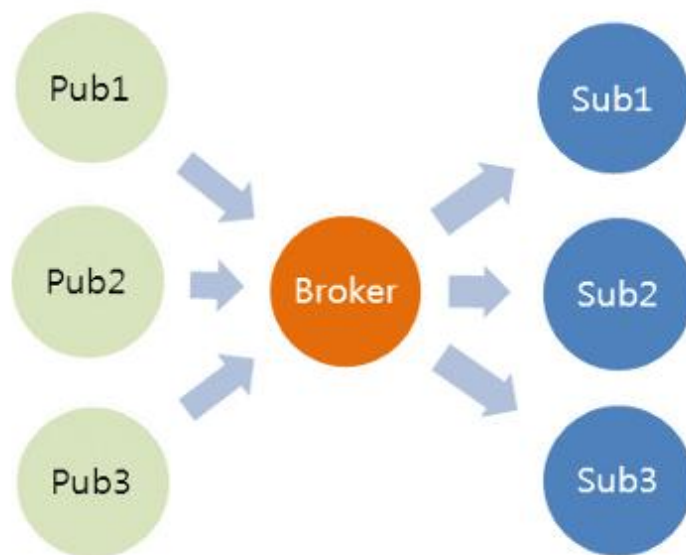
- **QoS 0 (Gửi tối đa một lần – At most once):** Thông điệp được gửi đi duy nhất một lần mà không yêu cầu bất kỳ cơ chế xác nhận nào từ phía nhận. Đây là mức có độ tin cậy thấp nhất, thông điệp có thể bị mất trong quá trình truyền.

- **QoS 1 (Gửi ít nhất một lần – At least once):** Thông điệp được đảm bảo sẽ đến được phía nhận ít nhất một lần thông qua cơ chế yêu cầu xác nhận. Tuy nhiên, trong trường hợp gói xác nhận bị thất lạc, thông điệp có thể được gửi lại và gây ra hiện tượng trùng lặp dữ liệu ở phía nhận.

- **QoS 2 (Gửi đúng một lần – Exactly once):** Đây là mức chất lượng dịch vụ cao nhất, đảm bảo rằng phía nhận chỉ tiếp nhận thông điệp đúng một lần duy nhất, không xảy ra tình trạng thất lạc hay trùng lặp. Cơ chế này đòi hỏi quy trình bắt tay phức tạp hơn và do đó có chi phí về băng thông cũng như độ trễ cao hơn so với hai mức trước [1].

c) Cách thức hoạt động của giao thức MQTT

Hoạt động của MQTT tuân thủ nghiêm ngặt mô hình giao tiếp tập trung qua Broker. Các thiết bị hoặc ứng dụng Client không giao tiếp trực tiếp với nhau mà mọi gói dữ liệu đều được định tuyến thông qua Broker. Cụ thể, khi một Publisher cần gửi dữ liệu, nó sẽ thiết lập kết nối đến Broker và thực hiện hành động Publish dữ liệu đó lên một Topic xác định. Về phía mình, Broker có nhiệm vụ quản lý danh sách các Subscriber đã đăng ký theo dõi Topic đó và tiến hành chuyển tiếp gói dữ liệu đến tất cả các Subscriber trong danh sách một cách tức thời. Một Client có thể đảm nhận vai trò là Publisher, Subscriber, hoặc đồng thời cả hai tùy thuộc vào yêu cầu nghiệp vụ của ứng dụng [1].



Hình 2.2: Sơ đồ kết nối hoạt động của MQTT

d) Tính chất của mô hình xuất bản/dăng ký (Publish/Subscribe)

Mô hình publish/subscribe của MQTT sở hữu ba tính chất tách biệt quan trọng, góp phần tạo nên sự linh hoạt và khả năng mở rộng cao của giao thức này:

- **Không gian tách biệt (Space decoupling):** Publisher và Subscriber không cần biết thông tin định danh hay địa chỉ mạng của nhau. Sự tương tác giữa hai bên hoàn toàn được trung gian hóa qua Broker và định danh Topic.
- **Thời gian tách biệt (Time decoupling):** Publisher và Subscriber không cần phải hoạt động đồng thời. Publisher có thể gửi dữ liệu lên Broker ngay cả khi không có Subscriber nào đang trực tuyến; dữ liệu đó có thể được Broker

lưu tạm (nếu được cấu hình) và chuyển đến Subscriber khi chúng kết nối trở lại.

- **Sự đồng bộ riêng rẽ (Synchronization decoupling):** Hoạt động xuất bản và tiếp nhận thông điệp không bị gián đoạn hay chờ đợi lẫn nhau. Publisher không bị chặn (block) trong quá trình gửi dữ liệu bởi tốc độ xử lý của Subscriber.

Bên cạnh đó, MQTT còn có một số đặc điểm riêng đáng chú ý: giao thức sử dụng cơ chế lọc thông điệp dựa vào chủ đề (subject-based filtering), cho phép Subscriber chỉ nhận được những dữ liệu thực sự liên quan; và việc tích hợp tầng chất lượng dịch vụ (QoS) cung cấp một cơ chế rõ ràng để nhận biết và kiểm soát mức độ thành công của quá trình truyền tải thông điệp [1].

e) Ưu điểm và hạn chế của MQTT

Qua quá trình tìm hiểu và phân tích, có thể rút ra những ưu điểm nổi bật cũng như một số hạn chế cần cân nhắc khi ứng dụng giao thức MQTT trong thực tiễn.

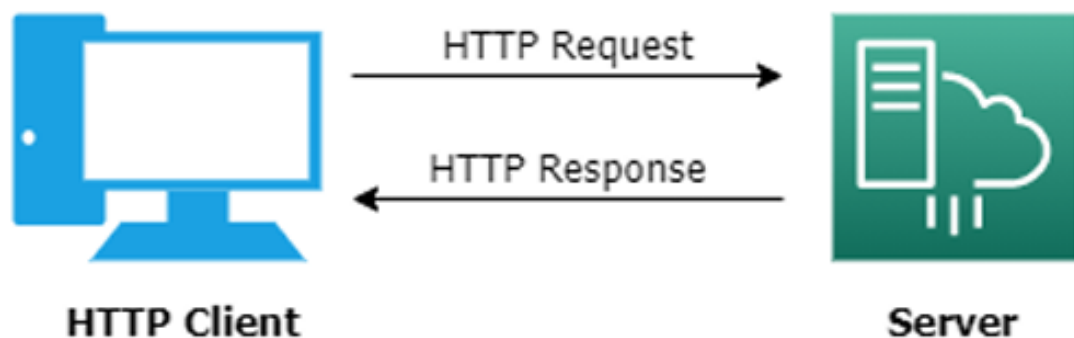
Về ưu điểm: MQTT cho phép truyền tải thông tin một cách nhanh chóng, hiệu quả và tiết kiệm băng thông đáng kể. Với thiết kế tối giản, giao thức này đặc biệt phù hợp cho các ứng dụng giám sát, điều khiển từ xa và các giải pháp IoT nói chung. Chi phí triển khai hệ thống sử dụng MQTT thường thấp, trong khi vẫn có thể đảm bảo tính bảo mật cao nếu được kết hợp với các cơ chế mã hóa như TLS/SSL. Hơn nữa, MQTT có khả năng hỗ trợ thu thập một khối lượng dữ liệu lớn từ nhiều nguồn khác nhau chỉ với yêu cầu tài nguyên phần cứng khiêm tốn, qua đó giúp giảm thiểu thời gian và công sức trong quá trình phát triển và vận hành hệ thống.

Về hạn chế: Mặc dù sở hữu nhiều lợi thế, MQTT cũng tồn tại một số điểm yếu cần được lưu tâm. Tốc độ truyền dữ liệu thô của MQTT có thể thấp hơn so với một số giao thức khác và hoạt động của nó phụ thuộc hoàn toàn vào cơ chế đăng ký động. Một hạn chế đáng kể khác là bản thân MQTT không được trang bị cơ chế mã hóa tích hợp; để bảo đảm an toàn cho dữ liệu trên đường truyền, giao thức này buộc phải dựa vào các giải pháp bảo mật ở tầng vận chuyển như TLS/SSL. Cuối cùng, khả năng mở rộng của MQTT trên quy mô cực lớn (ví dụ, hàng trăm nghìn đến hàng triệu thiết bị kết nối đồng thời đến một Broker) có thể gặp phải những thách thức nhất định và đòi hỏi các giải pháp kiến trúc phân tán phức tạp hơn.

2.1.2. Giao thức HTTP

a) Kiến trúc giao thức HTTP

HyperText Transfer Protocol (HTTP) là một giao thức truyền thông tầng ứng dụng phổ biến, vận hành theo mô hình Client-Server và đặc trưng bởi tính phi trạng thái (Stateless). Trong mô hình này, mọi quá trình giao tiếp đều bắt đầu bằng một yêu cầu (Request) từ phía Client và kết thúc bằng một phản hồi (Response) tương ứng từ phía Server. Việc định danh và truy cập các tài nguyên trên không gian mạng được thực hiện thông qua hệ thống định danh tài nguyên đồng nhất URI (Uniform Resource Identifier).



Hình 2.3: Kiến trúc giao thức HTTP

b) Các thành phần chính trong hệ thống

Một hệ thống trao đổi dữ liệu dựa trên giao thức HTTP bao gồm ba thực thể cơ bản sau:

- **HTTP Client:** Là tác nhân khởi tạo các yêu cầu dữ liệu. Trong hệ thống IoT, Client thường là các thiết bị nhúng (như ESP32) hoặc các ứng dụng trình duyệt web của người dùng.
- **HTTP Server:** Đóng vai trò là máy chủ tiếp nhận, xử lý các yêu cầu từ Client và trả về kết quả tương ứng (ví dụ: Apache, IIS hoặc các Web Server tích hợp trên Cloud).
- **Tài nguyên (Resource):** Đại diện cho các đối tượng dữ liệu được lưu trữ trên Server, có thể tồn tại dưới định dạng trang web (HTML), tệp tin cấu trúc (JSON, XML) hoặc các tệp tin đa phương tiện.

c) Đánh giá kỹ thuật: Ưu điểm và Hạn chế

Dưới góc độ triển khai hệ thống, giao thức HTTP bộc lộ các đặc tính kỹ thuật sau:

- **Ưu điểm:** Cấu trúc giao thức đơn giản, dễ dàng cài đặt trên nhiều nền tảng phần cứng khác nhau. Với việc truyền tải dữ liệu dưới dạng văn bản rõ ràng và sở hữu hệ thống thư viện hỗ trợ phong phú, HTTP cho phép phát triển ứng dụng nhanh chóng và linh hoạt.

- **Hạn chế:** Do thiếu cơ chế mã hóa mặc định và không có các giao thức xác thực danh tính máy chủ nghiêm ngặt, dữ liệu truyền qua HTTP dễ bị đánh chặn, giả mạo hoặc thay đổi trên đường truyền. Điều này đặt ra thách thức lớn về bảo mật và toàn vẹn dữ liệu cho các hệ thống giám sát quan trọng.

d) Cấu trúc bản tin HTTP

Bản tin HTTP được chuẩn hóa để đảm bảo tính nhất quán trong quá trình truyền dẫn thông tin, bao gồm hai phần chính: phần Header chứa các thông tin điều khiển và phần Body chứa nội dung dữ liệu thực tế.

- **HTTP Request (Yêu cầu từ Client):**

- **Request Line:** Xác định phương thức thực thi (GET, POST, PUT, DELETE,...), URI của tài nguyên và phiên bản HTTP đang sử dụng.

- **Request Headers:** Chứa siêu dữ liệu (metadata) bổ sung như định dạng dữ liệu chấp nhận, thông tin tác nhân người dùng.

- **HTTP Response (Phản hồi từ Server):**

- **Status Line:** Thông báo trạng thái xử lý thông qua mã trạng thái (Status Code). Các mã phổ biến bao gồm **200 OK** (Xử lý thành công), **404 Not Found** (Không tìm thấy tài nguyên), hoặc **500 Internal Server Error** (Lỗi hệ thống).

- **Response Headers:** Cung cấp thông tin về loại nội dung dữ liệu, độ dài bản tin và thời điểm phản hồi.

2.1.3. Giao thức CoAP

➤ CoAP là gì?

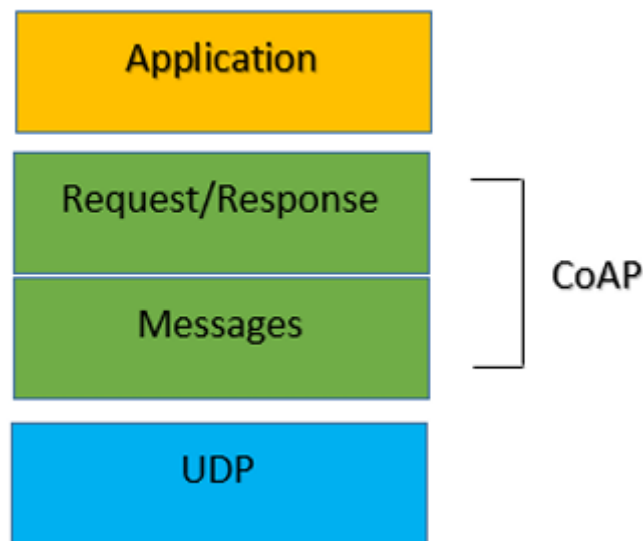
CoAP là một giao thức đơn giản, chi phí thấp được thiết kế riêng cho các thiết bị hiệu năng thấp (chẳng hạn như vi điều khiển) và nơi mạng có băng thông thấp. Giao thức này được sử dụng để trao đổi dữ liệu M2M và rất giống với HTTP.

CoAP có các tính năng chính sau:

- Giao thức web nhỏ gọn được sử dụng trong M2M;
- Bảo mật bằng DTLS;
- Trao đổi thông điệp không đồng bộ;

- Header gói tin nhỏ, dễ tách thông tin;
- Hỗ trợ URI và loại nội dung;
- Khả năng proxy và bộ nhớ đệm;
- Tùy chọn khai thác tài nguyên;
- Liên kết UDP (User Datagram Protocol) với độ tin cậy tùy chọn hỗ trợ các yêu cầu Unicast và Multicast.

➤ **Mô hình cấu trúc của CoAP**



Hình 2.4: Các lớp của CoAP

Mô hình tương tác trong CoAP tương tự như mô hình máy khách – máy chủ (Client/Server) của HTTP. Tuy nhiên, về mặt kiến trúc bên trong, CoAP được tổ chức thành hai lớp chức năng riêng biệt:

- **Lớp bản tin (Message Layer):** Đây là lớp thấp hơn, được thiết kế để hoạt động trực tiếp trên nền giao thức UDP, đảm nhiệm việc xử lý các cơ chế truyền thông không đồng bộ và trao đổi bản tin giữa các điểm cuối.

- **Lớp yêu cầu/phản hồi (Request/Response Layer):** Đây là lớp cao hơn, liên quan trực tiếp đến phương thức giao tiếp và logic xử lý các bản tin yêu cầu và phản hồi. Lớp này mang đậm triết lý RESTful, ánh xạ các phương thức HTTP quen thuộc vào môi trường ràng buộc.

➤ **Ưu điểm và nhược điểm của CoAP**

❖ **Ưu điểm**

- Đây là giao thức đơn giản và header nhỏ gọn hơn do hoạt động qua UDP. Nó cho phép thời gian wake-up ngắn và trạng thái sleep dài. Điều này giúp đạt được tuổi thọ pin dài để sử dụng;

- Nó sử dụng IPSEC (IP Security) hoặc DTLS (Datagram Transport Layer Security) để cung cấp giao tiếp an toàn;
- Giao tiếp đồng bộ không cần thiết trong giao thức CoAP;
- Nó có độ trễ thấp hơn so với HTTP;
- Nó tránh được việc truyền lại không cần thiết, nên nó tiêu thụ năng lượng ít hơn so với HTTP;
- Giao thức CoAP được sử dụng như một lựa chọn giao thức tốt nhất cho các mạng trong các thiết bị thông tin, thiết bị truyền thông và thiết bị điều khiển trong mạng nhà thông minh.

❖ **Nhược điểm**

- CoAP là giao thức không tin cậy lắm do sử dụng UDP. Do đó, các thông điệp CoAP đến không có thứ tự hoặc sẽ bị lạc khi chúng đến đích;
- Nó xác nhận mỗi lần nhận bản tin và do đó tăng thời gian xử lý. Hơn nữa, nó không xác minh xem bản tin nhận đã được giải mã đúng cách hay chưa;
- Đây là giao thức không được mã hóa như MQTT và sử dụng DTLS để cung cấp bảo mật;
- CoAP gặp vấn đề giao tiếp khi các thiết bị nằm sau NAT.

2.1.4. Kết Luận

Nói chung, có rất nhiều giao thức IoT khác nhau hỗ trợ Internet of Things và HTTP, MQTT, CoAP là ba giao thức thống trị chính trên thị trường IoT hiện nay vì những lợi ích tuyệt vời của nó mà chúng ta đã thảo luận ở trên. Tuy nhiên, điều đáng chú ý là trong khi giao thức HTTP phù hợp với ứng dụng duyệt web, hai giao thức còn lại phù hợp hơn với các mục đích IoT như giao tiếp thời gian thực, bảo trì phòng ngừa và giám sát môi trường, bao gồm nhà thông minh và ngành sản xuất.

2.2. CƠ SỞ DỮ LIỆU CHO IOT:

2.2.1. SQL và NoSQL

a) SQL

SQL (Structured Query Language) là ngôn ngữ tiêu chuẩn được sử dụng để tương tác với các hệ quản trị cơ sở dữ liệu quan hệ (RDBMS) hoặc hệ quản trị cơ sở dữ liệu quan hệ – đối tượng (ORDBMS).

SQL cho phép người dùng thực hiện các thao tác như truy vấn, chèn, cập nhật và xóa dữ liệu, đồng thời cung cấp các tính năng quản lý và tối ưu hóa cơ sở dữ liệu để đảm bảo tính nhất quán và toàn vẹn dữ liệu. Với cú pháp đơn giản

và dễ hiểu, SQL được sử dụng rộng rãi trong nhiều lĩnh vực như phân tích dữ liệu, phát triển phần mềm, và quản lý hệ thống [10].

Dữ liệu có cấu trúc là gì?

Dữ liệu có cấu trúc (Structured Data) là dạng dữ liệu được tổ chức theo một định dạng rõ ràng và tuân theo một mô hình nhất định, giúp việc lưu trữ, quản lý và truy vấn dữ liệu trở nên dễ dàng.

Trong các hệ cơ sở dữ liệu quan hệ, dữ liệu có cấu trúc thường được lưu trữ dưới dạng bảng (table) gồm các hàng (rows) và cột (columns). Mỗi bảng có thể được liên kết với các bảng khác thông qua các khóa (key) như:

- PRIMARY KEY: khóa chính dùng để định danh duy nhất một bản ghi.
- FOREIGN KEY: khóa ngoại dùng để tạo liên kết giữa các bảng.

Nhờ cơ chế này, dữ liệu được tổ chức một cách khoa học, giúp đảm bảo tính toàn vẹn dữ liệu và hỗ trợ việc thực hiện các truy vấn phức tạp.

SQL database là gì?

SQL database là hệ quản trị cơ sở dữ liệu (DBMS) sử dụng ngôn ngữ SQL để quản lý và tương tác với dữ liệu được tổ chức dưới dạng các bảng có quan hệ. Các APIs (giao diện lập trình ứng dụng) của SQL cung cấp nhiều nhóm hàm giúp nhà phát triển có thể thực hiện các thao tác với cơ sở dữ liệu một cách hiệu quả mà không cần phải lặp đi lặp lại từng câu lệnh SQL.

Một số hệ quản trị SQL phổ biến bao gồm MySQL, SQL Server và PostgreSQL. Các SQL database hỗ trợ người dùng định nghĩa cấu trúc dữ liệu, thực hiện các truy vấn phức tạp và tối ưu hóa hiệu suất thông qua các tính năng như *index*, *JOIN*, *stored procedures*...

Ưu điểm của SQL

- Giảm dung lượng lưu trữ dữ liệu - Chuẩn hóa dữ liệu đã loại bỏ sự dư thừa và trùng lặp dữ liệu.
- Tuân thủ ACID - Đảm bảo tính toàn vẹn, bảo mật và nhất quán dữ liệu mạnh mẽ.
- Chuẩn hóa - Các công cụ cơ sở dữ liệu tối ưu hóa các truy vấn tốt hơn để phù hợp với dữ liệu được lưu trữ trên đĩa.
- Tuyệt vời cho các truy vấn phức tạp - SQL xử lý truy vấn và kết hợp dữ liệu giữa các bảng một cách hiệu quả.

- Lược đồ được định sẵn - Việc xác thực được thực hiện ở lớp cơ sở dữ liệu chứ không phải lớp ứng dụng.
- Ngôn ngữ được chuẩn hóa trên các hệ quản trị cơ sở dữ liệu quan hệ khác nhau.

Nhược điểm của SQL

- Mở rộng theo chiều dọc - Tốn kém hơn so với mở rộng theo chiều ngang.
- Mô hình dữ liệu cứng nhắc - Yêu cầu thiết kế từ đầu và khó thay đổi hơn (yêu cầu di chuyển dữ liệu và có thể gây gián đoạn hoạt động).
- Điểm lỗi duy nhất - Được giảm thiểu bằng các kỹ thuật sao chép và chuyển đổi dự phòng.

b) NoSQL

Khác với SQL, NoSQL (hay còn gọi là Not Only SQL, Non-SQL) là một loại cơ sở dữ liệu quản lý dữ liệu theo cấu trúc phi quan hệ. Điều này có nghĩa là nó không có lược đồ (không có lược đồ cố định) và tránh các phép nối (join). Nhờ **khả năng mở rộng, hiệu suất cao và tính linh hoạt**, NoSQL đang ngày càng phổ biến trong các ứng dụng dữ liệu lớn và ứng dụng thời gian thực.

Dữ liệu phi cấu trúc là gì?

Dữ liệu phi cấu trúc là dạng dữ liệu không tuân theo một mô hình dữ liệu dạng bảng truyền thống, không có cấu trúc rõ ràng và dễ nhận biết như trong cơ sở dữ liệu quan hệ. Các thông tin như tài liệu văn bản, hình ảnh, video hoặc các bài đăng trên mạng xã hội thường có định dạng khác nhau và không có cấu trúc rõ ràng.

Dữ liệu phi cấu trúc khó xử lý và tổ chức theo dạng bảng truyền thống, nhưng lại phù hợp với cách lưu trữ của NoSQL, nơi mà mỗi tài liệu hoặc đối tượng có thể chứa dữ liệu theo nhiều dạng và kiểu khác nhau.

NoSQL database là gì?

Trong bối cảnh phát triển của các ứng dụng hiện đại, việc lựa chọn mô hình dữ liệu phù hợp đóng vai trò quyết định đến hiệu năng và khả năng mở rộng của hệ thống. Dưới đây là bốn loại cơ sở dữ liệu NoSQL tiêu biểu, mỗi loại sở hữu mô hình dữ liệu và đặc trưng riêng nhằm đáp ứng các bộ bài toán chuyên biệt.

1) Cơ sở dữ liệu dạng tài liệu

Mô hình này lưu trữ dữ liệu dưới dạng các tài liệu phi cấu trúc hoặc bán cấu trúc, thường sử dụng định dạng JSON. Thay vì cố định lược đồ như trong hệ quản trị quan hệ (RDBMS), các tài liệu có thể có cấu trúc độc lập, mang lại tính linh hoạt cao trong quá trình phát triển và tiến hóa của hệ thống.

- Đặc điểm nổi bật: Không yêu cầu lược đồ cố định (schema-flexible), hỗ trợ lưu trữ dữ liệu phân cấp, khả năng tương thích cao với các ngôn ngữ lập trình hướng đối tượng.
- Ứng dụng tiêu biểu: Hệ thống quản lý nội dung, nền tảng thương mại điện tử với cấu hình sản phẩm phức tạp, hồ sơ người dùng.
- Hệ thống tiêu biểu: MongoDB, Couchbase Server, Elasticsearch (cho trường hợp lưu trữ dạng tài liệu).

2) Cơ sở dữ liệu dạng khóa – giá trị

Đây là mô hình NoSQL đơn giản nhất, dựa trên cấu trúc bảng băm phân tán. Dữ liệu được lưu trữ dưới dạng các cặp khóa (key) – giá trị (value), trong đó giá trị là một đối tượng không cần xác định cấu trúc cụ thể. Sự đơn giản này cho phép hệ thống đạt được hiệu suất truy xuất vượt trội và khả năng mở rộng ngang tuyến tính.

- Đặc điểm nổi bật: Độ trễ cực thấp (low latency), thao tác đọc/ghi đơn giản, khả năng mở rộng phân tán tốt.
- Ứng dụng tiêu biểu: Lưu trữ phiên làm việc (session) trong xác thực người dùng, bộ nhớ đệm (cache) giảm tải cho cơ sở dữ liệu chính, lưu giỏ hàng tạm thời trong các ứng dụng bán lẻ.
- Hệ thống tiêu biểu: Redis, Memcached, Amazon DynamoDB.

3) Cơ sở dữ liệu dạng cột mở rộng

Không nên nhầm lẫn với mô hình cột truyền thống trong các hệ quản trị quan hệ, cơ sở dữ liệu dạng cột mở rộng lưu trữ dữ liệu theo từng nhóm cột thay vì theo hàng. Thiết kế này tối ưu cho các truy vấn cần quét một lượng lớn bản ghi nhưng chỉ giới hạn ở một số cột nhất định, thường xuất hiện trong các bài toán phân tích.

- Đặc điểm nổi bật: Hiệu suất cao cho thao tác tổng hợp (aggregation) và truy vấn phân tích trên tập dữ liệu lớn (Big Data), hỗ trợ phân tán mạnh mẽ.

- Ứng dụng tiêu biểu: Kho dữ liệu (Data Warehouse), hệ thống xử lý dữ liệu cho trí tuệ doanh nghiệp (BI), hệ thống IoT với dữ liệu chuỗi thời gian.
- Hệ thống tiêu biểu: Apache Cassandra, Apache HBase, Google Cloud Bigtable.

4) Cơ sở dữ liệu đồ thị

Dựa trên nền tảng lý thuyết đồ thị, mô hình này biểu diễn dữ liệu thông qua các đỉnh (node) và mối quan hệ (edge). Không giống như các mô hình khác phải xử lý quan hệ thông qua phép JOIN phức tạp, cơ sở dữ liệu đồ thị cho phép duyệt các mối quan hệ với độ phức tạp thời gian tuyến tính, rất phù hợp cho dữ liệu có tính kết nối cao.

- Đặc điểm nổi bật: Hiệu năng vượt trội cho các truy vấn liên quan đến mối quan hệ đa cấp, khả năng biểu diễn trực quan, hỗ trợ các thuật toán đồ thị phức tạp.
- Ứng dụng tiêu biểu: Mạng xã hội, hệ thống đề xuất (recommendation system), phát hiện gian lận tài chính, quản lý danh tính và kiểm soát truy cập.
- Hệ thống tiêu biểu: Neo4j, Amazon Neptune, ArangoDB.

Ưu điểm chính của NoSQL:

- Không yêu cầu lược đồ cố định (schema-less): Dữ liệu có thể được ghi vào cơ sở dữ liệu mà không cần định nghĩa trước cấu trúc. Điều này tạo điều kiện thuận lợi cho việc thay đổi yêu cầu nghiệp vụ trong quá trình phát triển, giảm thiểu chi phí bảo trì lược đồ so với mô hình quan hệ.
- Mô hình dữ liệu linh hoạt: Mỗi thực thể có thể có cấu trúc khác nhau, phù hợp với các hệ thống có dữ liệu bán cấu trúc hoặc phi cấu trúc như tài liệu JSON, đồ thị xã hội hay dữ liệu cảm biến.
- Khả năng mở rộng theo chiều ngang (horizontal scaling): NoSQL được thiết kế để hoạt động trên các cụm máy chủ phân tán. Việc bổ sung thêm node để tăng dung lượng và hiệu suất thường có chi phí thấp hơn so với việc nâng cấp cấu hình phần cứng (mở rộng chiều dọc) trong các hệ quản trị quan hệ.

- Độ khả dụng cao (high availability): Các hệ thống NoSQL thường tích hợp cơ chế sao chép dữ liệu tự động, đảm bảo hệ thống không có điểm lỗi đơn (single point of failure) và có thể tiếp tục phục vụ khi một hoặc nhiều node gặp sự cố.
- Hiệu năng cao với các mẫu truy cập đặc thù: Dữ liệu được lưu trữ theo cách tối ưu cho các tác vụ đọc/ghi thường xuyên, ví dụ như lưu dữ liệu theo khóa–giá trị hoặc nhóm cột, giúp giảm thiểu độ trễ.

Nhược điểm và thách thức:

- Bảo mật và tính toàn vẹn dữ liệu hạn chế: Hầu hết các hệ thống NoSQL chỉ đảm bảo tính nhất quán cuối cùng (eventual consistency) thay vì tính nhất quán tức thời như trong mô hình ACID. Điều này gây khó khăn trong việc xác minh tính toàn vẹn của dữ liệu, đặc biệt với các ứng dụng tài chính hoặc yêu cầu giao dịch nghiêm ngặt.
- Thiếu chuẩn hóa và tối ưu truy vấn: Do không có lược đồ cố định, việc đảm bảo kiểu dữ liệu thống nhất trở nên phức tạp. Các công cụ truy vấn NoSQL thường không hỗ trợ mạnh mẽ các phép nối (JOIN) hay truy vấn phức tạp, khiến người phát triển phải tự quản lý tính toàn vẹn ở tầng ứng dụng.
- Dư thừa dữ liệu (data redundancy): Vì thiếu chuẩn hóa, dữ liệu thường được lưu trùng lặp để phục vụ các mẫu truy vấn khác nhau. Mặc dù chi phí lưu trữ hiện nay đã giảm đáng kể, nhưng việc duy trì tính đồng nhất giữa các bản sao vẫn là một bài toán phức tạp.
- Tính không đồng nhất giữa các hệ thống: Mỗi loại NoSQL (document, key-value, wide-column, graph) được tối ưu cho một nhóm bài toán riêng. Việc lựa chọn sai loại có thể dẫn đến hiệu năng kém; chẳng hạn, cơ sở dữ liệu đồ thị không phù hợp để thực hiện các truy vấn phạm vi, ngược lại cơ sở dữ liệu cột mở rộng không tối ưu cho các tác vụ có tính chất giao dịch phức tạp.

2.2.2. So sánh cơ sở dữ liệu quan hệ (SQL) và NoSQL

Để có cái nhìn toàn diện, việc so sánh hai họ cơ sở dữ liệu dựa trên các tiêu chí kỹ thuật là cần thiết.

Bảng 2.1: Tổng hợp những khác biệt cốt lõi giữa SQL (Relational Database Management System – RDBMS) và NoSQL.

Tiêu chí	SQL	NoSQL
Loại cơ sở dữ liệu	Quan hệ, dựa trên bảng (hàng và cột)	Phi quan hệ, gồm các dạng: document, key-value, wide-column, graph
Ngôn ngữ truy vấn	SQL chuẩn hóa, hỗ trợ truy vấn phức tạp và phép JOIN	Không có ngôn ngữ chung, truy vấn phụ thuộc vào từng hệ thống (ví dụ MongoDB Query Language, CQL cho Cassandra)
Lược đồ dữ liệu	Lược đồ cố định, bắt buộc xác định cấu trúc trước khi nhập dữ liệu	Lược đồ động, cho phép thêm trường tự do, dễ dàng thay đổi cấu trúc
Loại dữ liệu	Chủ yếu xử lý dữ liệu có cấu trúc	Xử lý tốt dữ liệu có cấu trúc, bán cấu trúc và phi cấu trúc
Khả năng mở rộng	Mở rộng theo chiều dọc (vertical scaling) - nâng cấp phần cứng máy chủ	Mở rộng theo chiều ngang (horizontal scaling) - thêm các node rẻ tiền, hỗ trợ phân tán tự nhiên
Tính nhất quán giao dịch	Tuân thủ ACID, đảm bảo tính nhất quán tức thời trong các giao dịch	Tuân thủ BASE (Basically Available, Soft state, Eventual consistency), ưu tiên tính sẵn sàng và dung sai phân vùng

Hiệu suất với dữ liệu lớn	Có thể gặp giới hạn do chi phí mở rộng dọc và độ phức tạp của JOIN khi quy mô tăng	Được tối ưu cho dữ liệu lớn, khả năng xử lý song song và phân tán cao
---------------------------	--	---

2.2.3. Lựa chọn giải pháp lưu trữ dữ liệu lịch sử cảm biến

Trong hệ thống cảnh báo cháy và rò rỉ gas mà đề tài xây dựng, mỗi nút cảm biến sử dụng vi điều khiển ESP32 để thu thập các thông số như nồng độ khí gas, nhiệt độ môi trường và tín hiệu phát hiện lửa. Toàn bộ dữ liệu sau đó được truyền không dây về máy chủ trung tâm (Raspberry Pi) thông qua giao thức MQTT. Bên cạnh việc hiển thị trạng thái tức thời, hệ thống còn phải có khả năng lưu trữ lịch sử đo đạc để phục vụ công tác truy vết sự cố, phân tích xu hướng và trực quan hóa dữ liệu dưới dạng biểu đồ theo thời gian. Do đó, việc xác định mô hình cơ sở dữ liệu phù hợp là một khâu quan trọng trong thiết kế tổng thể.

Sau khi cân nhắc giữa hai họ cơ sở dữ liệu chính là SQL (quan hệ) và NoSQL (phi quan hệ), nhóm quyết định chọn **MariaDB** – một hệ quản trị cơ sở dữ liệu quan hệ mã nguồn mở thuộc nhánh của MySQL – làm giải pháp lưu trữ cho hệ thống.

Lý do đầu tiên xuất phát từ chính **đặc điểm cấu trúc của dữ liệu cảm biến**. Dữ liệu từ các nút ESP32 gửi về hầu như luôn có một định dạng nhất quán: mỗi bản tin có thể bao gồm mã thiết bị, khu vực lắp đặt, thời điểm đo, giá trị nhiệt độ, giá trị khí gas, trạng thái ngọn lửa và mức cảnh báo. Đây là dạng dữ liệu có cấu trúc rõ ràng, hoàn toàn phù hợp để tổ chức thành các bảng với các cột được định nghĩa trước. Việc sử dụng SQL giúp nhóm dễ dàng kiểm soát tính toàn vẹn của dữ liệu ngay từ tầng cơ sở dữ liệu, thay vì phải xử lý các ngoại lệ ở tầng ứng dụng.

Lý do thứ hai liên quan đến **nhu cầu truy vấn trong thực tế sử dụng**. Trong quá trình vận hành, người dùng hoặc người quản lý sẽ cần tra cứu dữ liệu theo nhiều tiêu chí khác nhau, chẳng hạn như: “Xem tất cả các lần nhiệt độ vượt ngưỡng tại phòng bếp trong 24 giờ qua” hay “Thống kê số lần cảnh báo khí gas trong tháng”. Ngôn ngữ SQL cung cấp sẵn các mệnh đề lọc (WHERE), nhóm (GROUP BY), sắp xếp (ORDER BY) và các hàm tổng hợp (AVG, MAX, COUNT) cho phép thực hiện các truy vấn này một cách trực quan và hiệu quả. Mặc dù một số hệ NoSQL cũng hỗ trợ các thao tác tương tự, nhưng

việc làm quen và tối ưu truy vấn trên SQL thường đơn giản hơn đối với sinh viên trong khuôn khổ một đề án tốt nghiệp.

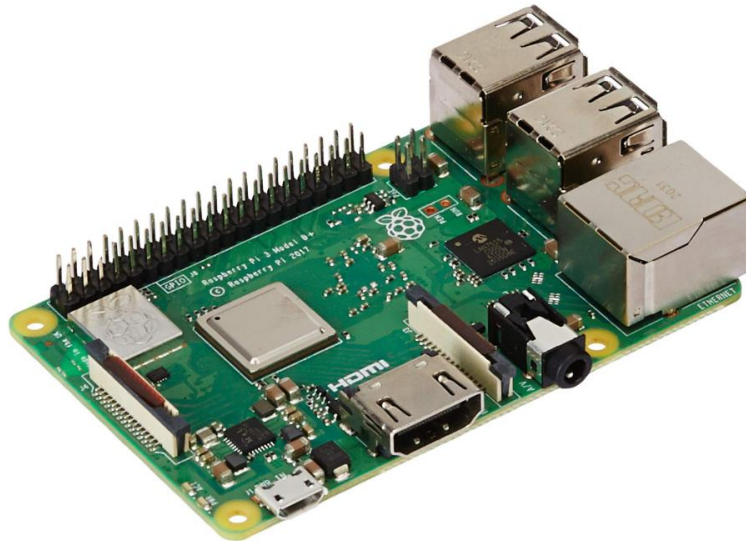
Lý do thứ ba, và cũng là yếu tố then chốt đối với một hệ thống giám sát an toàn, là **yêu cầu về tính nhất quán và độ tin cậy của dữ liệu**. Các sự kiện cảnh báo cháy hoặc rò rỉ gas cần được ghi nhận một cách chính xác tuyệt đối; việc mất mát hoặc sai lệch một bản ghi có thể dẫn đến những hậu quả nghiêm trọng trong công tác đánh giá sự cố. Các hệ quản trị cơ sở dữ liệu quan hệ như MariaDB được thiết kế để tuân thủ các thuộc tính **ACID** (Tính nguyên tử, Nhất quán, Cô lập, Bền vững), đảm bảo rằng mỗi giao dịch với cơ sở dữ liệu đều được thực thi một cách trọn vẹn và an toàn. Điều này mang lại sự yên tâm hơn so với mô hình “nhất quán cuối cùng” (eventual consistency) thường thấy ở nhiều giải pháp NoSQL.

Cuối cùng, xét trên **quy mô triển khai thực tế của đề tài**, hệ thống chỉ bao gồm một số lượng hạn chế các nút cảm biến (trong phạm vi thử nghiệm) với tần suất gửi dữ liệu không quá dày đặc (khoảng vài giây đến 10 giây một lần đo). Với lưu lượng dữ liệu như vậy, MariaDB hoàn toàn đủ khả năng xử lý khi chạy trực tiếp trên phần cứng Raspberry Pi 3B+. Giải pháp này không chỉ đáp ứng tốt các yêu cầu về lưu trữ và truy vấn mà còn đơn giản hóa quá trình cài đặt, cấu hình và bảo trì hệ thống, giúp nhóm tập trung hơn vào việc hoàn thiện các chức năng chính của đề tài.

Tóm lại, với những ưu điểm về khả năng quản lý dữ liệu có cấu trúc, hỗ trợ truy vấn linh hoạt, tính nhất quán cao và sự phù hợp với quy mô phần cứng sẵn có, MariaDB là lựa chọn hợp lý cho bài toán lưu trữ dữ liệu lịch sử cảm biến trong hệ thống này.

CHƯƠNG 3: THIẾT KẾ VÀ TRIỂN KHAI MÁY CHỦ TRUNG GIAN (BROKER) VÀ CƠ SỞ DỮ LIỆU

3.1. LỰA CHỌN PHẦN CỨNG MÁY CHỦ RASPBERRY PI.



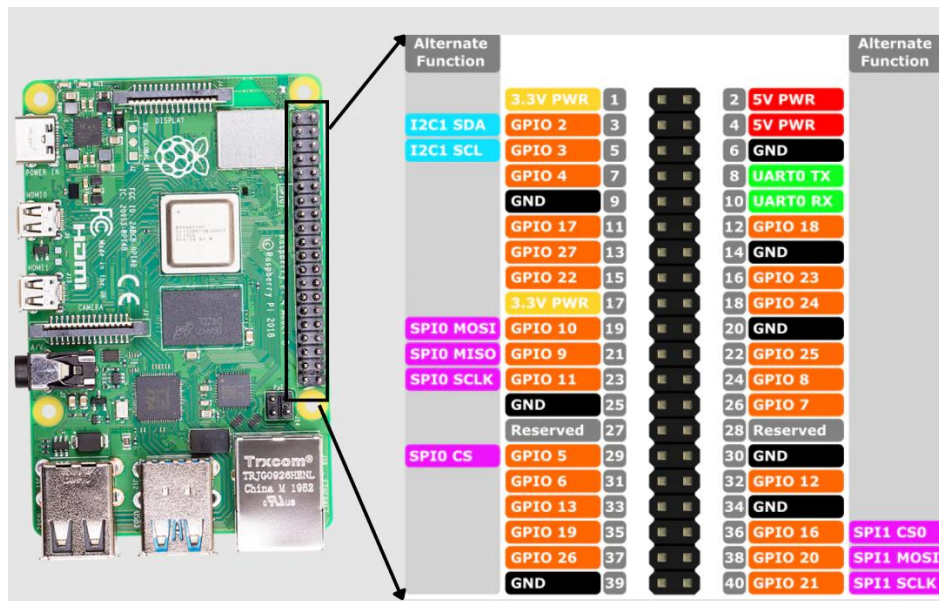
Hình 3.1: Module Raspberry Pi

3.1.1. Giới thiệu Raspberry Pi 3B+

Raspberry Pi 3 Model B+ là máy tính nhúng đơn bo mạch thuộc thế hệ thứ ba của dòng Raspberry Pi, được phát triển nhằm đáp ứng các nhu cầu xử lý dữ liệu, giao tiếp mạng và điều khiển trong các hệ thống nhúng và IoT. Thiết bị sử dụng bộ vi xử lý Broadcom BCM2837B0 kiến trúc 64 bit, gồm 4 nhân ARM Cortex-A53 hoạt động ở xung nhịp 1,4 GHz, đi kèm bộ nhớ RAM 1 GB.

Về khả năng kết nối và mở rộng phần cứng, Raspberry Pi 3B+ được trang bị 40 chân GPIO, 4 cổng USB 2.0, cổng HDMI, khe cắm thẻ microSD, cổng CSI cho camera, cổng DSI cho màn hình, jack âm thanh/video 3,5 mm và cổng Ethernet RJ45. Ngoài ra, thiết bị còn tích hợp Wi-Fi băng tần kép 2,4/5 GHz chuẩn 802.11ac, Bluetooth 4.2 và Bluetooth Low Energy, giúp tăng tính linh hoạt khi kết nối với các cảm biến, bộ điều khiển và mạng truyền thông trong hệ thống IoT.

So với Raspberry Pi 3 Model B, phiên bản 3B+ được cải thiện về hiệu năng xử lý, khả năng kết nối không dây, tốc độ Ethernet và hỗ trợ Power over Ethernet thông qua PoE HAT rời. Nhờ đó, Raspberry Pi 3B+ phù hợp để sử dụng như một máy chủ mini, gateway IoT hoặc broker MQTT trong các hệ thống giám sát và cảnh báo.



3.1.2. Lý do lựa chọn Raspberry Pi 3B+

Trong đề tài hệ thống cảnh báo cháy và rò rỉ gas dựa trên nền tảng IoT, máy chủ trung tâm cần đáp ứng các yêu cầu như tiếp nhận dữ liệu từ nhiều nút cảm biến, duy trì kết nối mạng ổn định, xử lý dữ liệu theo thời gian thực và hỗ trợ triển khai broker MQTT. Với bộ xử lý 4 nhân 1,4 GHz, RAM 1 GB cùng các giao diện mạng có dây và không dây tích hợp sẵn, Raspberry Pi 3B+ có khả năng đáp ứng tốt các yêu cầu này trong phạm vi mô hình khóa luận.

Một trong những lý do quan trọng để lựa chọn Raspberry Pi 3B+ là thiết bị có cấu hình tương đối tốt so với nhiều nền tảng nhúng phổ thông, trong khi vẫn đảm bảo kích thước nhỏ gọn và mức tiêu thụ điện năng thấp. Điều này giúp hệ thống dễ dàng triển khai trong mô hình thực nghiệm, đồng thời tiết kiệm chi phí đầu tư và vận hành. Phần nhận định về mức độ phù hợp với mô hình khóa luận là suy luận kỹ thuật từ cấu hình phần cứng chính thức của thiết bị.

Bên cạnh đó, Raspberry Pi 3B+ tích hợp đồng thời Ethernet, Wi-Fi và Bluetooth, tạo điều kiện thuận lợi cho việc kết nối với các thiết bị cảm biến, bộ điều khiển và hạ tầng mạng. Trong trường hợp cần độ ổn định cao, thiết bị có thể sử dụng kết nối Ethernet; ngược lại, khi cần lắp đặt linh hoạt, Wi-Fi tích hợp sẵn cũng đáp ứng tốt yêu cầu truyền nhận dữ liệu trong hệ thống IoT.

Ngoài ưu điểm về phần cứng, Raspberry Pi 3B+ còn có hệ sinh thái phần mềm phong phú, được hỗ trợ bởi cộng đồng người dùng lớn và tài liệu kỹ thuật đầy đủ. Thiết bị tương thích tốt với Raspberry Pi OS và có thể dễ dàng cài đặt các phần mềm như Mosquitto MQTT Broker, Node-RED hoặc các công cụ

quản lý dữ liệu khác. Đây là yếu tố rất thuận lợi trong quá trình nghiên cứu, phát triển và mở rộng hệ thống.

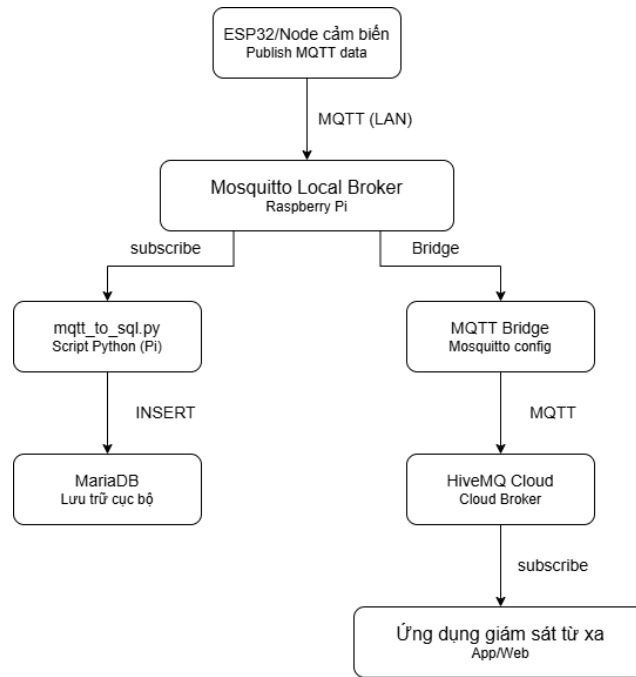
Nhờ những đặc điểm trên, Raspberry Pi 3B+ là lựa chọn phù hợp để đảm nhiệm vai trò máy chủ trung tâm trong hệ thống cảnh báo cháy và rò rỉ gas. Thiết bị không chỉ đáp ứng yêu cầu kỹ thuật của đề tài mà còn đảm bảo tính thực tiễn, khả năng mở rộng và sự thuận tiện trong quá trình triển khai thử nghiệm.

3.2. CÀI ĐẶT VÀ CẤU HÌNH MQTT BROKER

Trong hệ thống của đề tài, MQTT Broker được triển khai trên Raspberry Pi 3B+ bằng Mosquitto để làm trung tâm tiếp nhận và phân phối dữ liệu từ các node cảm biến ESP32. Dữ liệu từ cảm biến không được đưa thẳng vào ứng dụng hay cơ sở dữ liệu, mà trước hết được publish về broker cục bộ trên Raspberry Pi. Sau đó, broker phân phối dữ liệu cho chương trình *mqtt_to_sql.py* để lưu vào MariaDB, đồng thời bridge dữ liệu lên HiveMQ Cloud để phục vụ giám sát từ xa trên ứng dụng Android hoặc Web. Vì vậy, mục này tập trung trình bày vai trò của broker, quá trình cài đặt Mosquitto, cấu hình broker cục bộ, tổ chức topic, cấu hình bridge và kiểm tra hoạt động sau khi hoàn tất.

3.2.1. Vai trò của MQTT Broker trong kiến trúc hệ thống

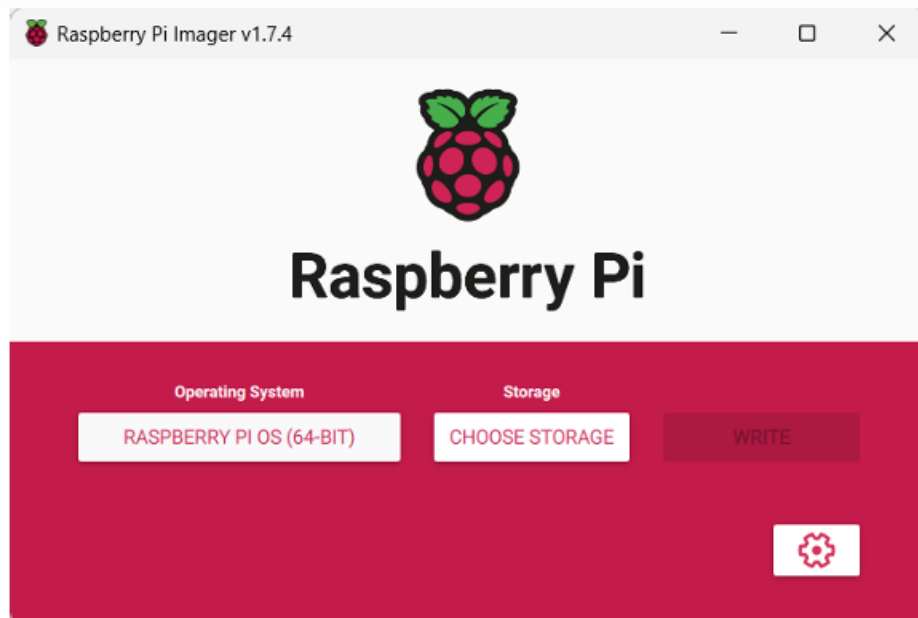
Trong kiến trúc truyền thông của hệ thống, MQTT Broker là điểm trung gian kết nối giữa các node cảm biến, phần lưu trữ dữ liệu và ứng dụng giám sát. Các ESP32 thực hiện publish dữ liệu theo từng topic về Mosquitto local trên Raspberry Pi. Tại đây, broker tiếp nhận bản tin, duy trì cơ chế publish/subscribe và phân phối dữ liệu đến các thành phần xử lý phía sau. Một nhánh dữ liệu được chuyển đến chương trình *mqtt_to_sql.py* để ghi vào MariaDB dưới dạng dữ liệu lịch sử cục bộ; nhánh còn lại được bridge lên HiveMQ Cloud để client từ xa có thể subscribe và hiển thị dữ liệu theo thời gian thực. Với cách tổ chức này, broker không chỉ là dịch vụ truyền bản tin mà còn là nút trung tâm của toàn bộ kiến trúc truyền thông trong hệ thống báo cháy.



Hình 3.2: Sơ đồ vị trí của MQTT Broker trong hệ thống cảnh báo cháy

3.2.2. Cài đặt Mosquitto Broker trên Raspberry Pi

Để triển khai broker, trước hết nhóm sử dụng Raspberry Pi Imager để ghi hệ điều hành Raspberry Pi OS lên thẻ microSD, đồng thời thiết lập sẵn một số thông tin ban đầu như tên thiết bị, tài khoản đăng nhập, Wi-Fi và quyền truy cập từ xa.



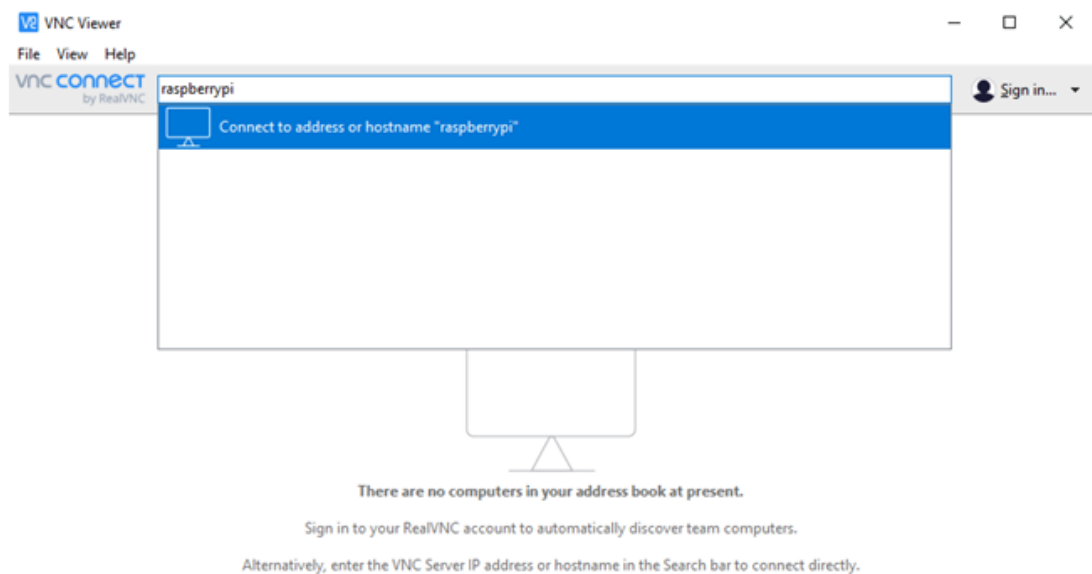
Hình 3.3: Phần mềm Raspberry Pi

Sau khi Raspberry Pi khởi động ổn định và kết nối mạng thành công, nhóm tiến hành cài đặt Mosquitto cùng các gói hỗ trợ cần thiết. Trong quá trình thao

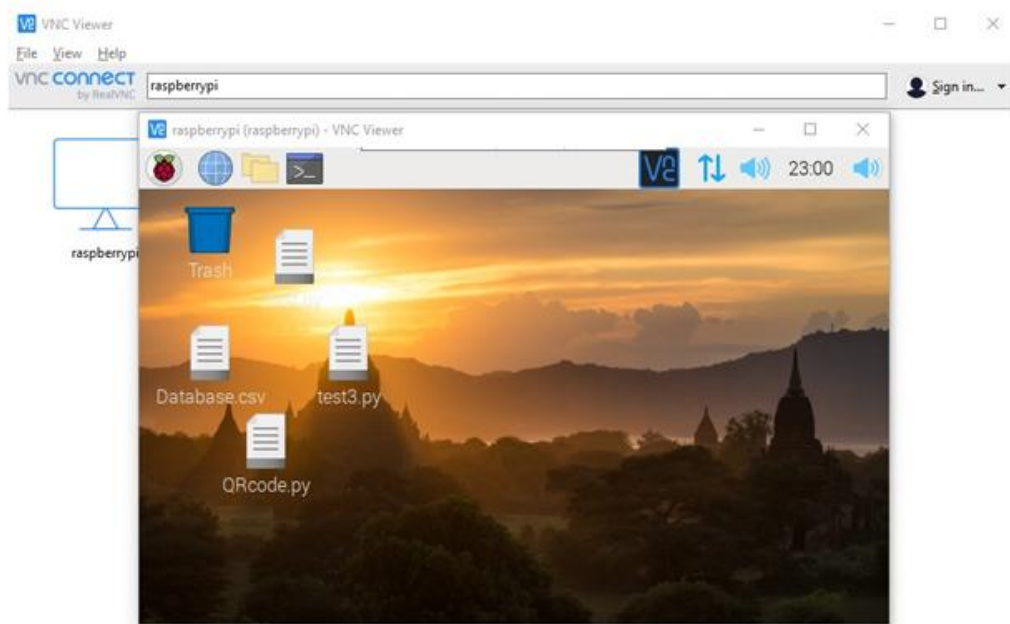
tác, nhóm sử dụng kết hợp VNC Viewer và Visual Studio Code. VNC phù hợp khi cần quan sát trực tiếp giao diện hệ điều hành hoặc thao tác bằng desktop của Raspberry Pi, còn VS Code thuận tiện hơn khi chỉnh sửa nhanh tệp cấu hình và mã nguồn. Cách dùng này khá thực tế: nếu VNC mở chậm hoặc không cần giao diện đồ họa thì vẫn có thể tiếp tục làm việc bằng VS Code; ngược lại, khi cần kiểm tra trực quan trên Raspberry Pi thì VNC hỗ trợ tốt hơn. Nói ngắn gọn, hai công cụ này không thay thế hoàn toàn cho nhau mà bổ trợ cho nhau trong quá trình cài đặt và cấu hình.



Hình 3.4: Sử dụng VNC



Hình 3.5: VNC điều khiển Raspberry



Hình 3.6: Kết thúc quá trình đăng nhập

Bên cạnh đó, HiveMQ Cloud được sử dụng làm cloud broker để nhận dữ liệu do Raspberry Pi chuyển tiếp lên, qua đó hỗ trợ giám sát dữ liệu theo thời gian thực từ bên ngoài mạng nội bộ.

Sau khi chuẩn bị xong môi trường phần mềm, nhóm tiến hành cài đặt MQTT Broker cục bộ trên Raspberry Pi. Để triển khai broker, nhóm sử dụng Mosquitto do đây là phần mềm gọn nhẹ, phù hợp với phần cứng Raspberry Pi và dễ tích hợp với các công cụ kiểm thử trong môi trường Linux. Trên Raspberry Pi, nhóm thực hiện cập nhật hệ thống và cài đặt các gói cần thiết bằng lệnh sau:

```
sudo apt update
sudo apt full-upgrade -y
sudo apt install -y mosquitto mosquitto-clients
sudo systemctl enable mosquitto
sudo systemctl start mosquitto
```

Trong đó, *mosquitto* là dịch vụ broker MQTT cục bộ, *mosquitto-clients* dùng để kiểm tra publish/subscribe bằng terminal, còn các gói còn lại phục vụ tầng cơ sở dữ liệu và tầng xử lý trung gian. Sau khi cài đặt xong, trạng thái của broker được kiểm tra bằng lệnh: `sudo systemctl status mosquitto --no-pager -l`

Khi dịch vụ hiển thị trạng thái active (running), có thể kết luận Mosquitto đã được cài đặt thành công và sẵn sàng cho bước cấu hình tiếp theo. Ở góc độ kỹ thuật, bước này là điều kiện tiên quyết, vì toàn bộ kiến trúc truyền thông của hệ thống đều phụ thuộc vào việc broker cục bộ hoạt động ổn định trên Raspberry Pi.

3.2.3. Cấu hình broker cục bộ và thiết lập bảo mật

Sau khi cài đặt thành công Mosquitto, nhóm cấu hình broker cục bộ để các node ESP32 trong cùng mạng nội bộ có thể gửi dữ liệu về Raspberry Pi qua cổng 1883. Với quy mô mô hình thử nghiệm, đây là cách triển khai phù hợp vì đơn giản, dễ kiểm tra và thuận tiện cho việc theo dõi dữ liệu trong mạng LAN. Một tệp cấu hình riêng được tạo trong thư mục `/etc/mosquitto/conf.d/` với nội dung như sau:

```
printf "listener 1883\nallow_anonymous true\n" | sudo tee
/etc/mosquitto/conf.d/fireiot.conf

sudo systemctl restart mosquitto
sudo systemctl status mosquitto --no-pager -l
```

Sau khi tạo tệp cấu hình sẽ hiển thị nội dung sau:

```
listener 1883
allow_anonymous true
```

Ở đây, `listener 1883` cho phép broker lắng nghe tại cổng 1883, còn `allow_anonymous true` giúp các node trong mạng nội bộ kết nối mà chưa cần tài khoản đăng nhập. Cách cấu hình này phù hợp cho giai đoạn triển khai và kiểm thử trong phạm vi mô hình, vì mục tiêu chính là bảo đảm các node publish dữ liệu ổn định về Raspberry Pi. Tuy nhiên, lớp bảo mật này chỉ nên áp dụng ở mạng nội bộ; khi dữ liệu được chuyển ra ngoài Internet thì cần có mức bảo vệ cao hơn.

```

pi@pi3b:~/fireiot $ sudo systemctl status mosquitto --no-pager -l
● mosquitto.service - Mosquitto MQTT Broker
   Loaded: loaded (/usr/lib/systemd/system/mosquitto.service; enabled; preset: enabled)
   Active: active (running) since Sun 2026-04-19 22:45:24 +07; 1h 11min ago
     Invocation: 469d1b1ea1a649c494fb8a2d637730e8
       Docs: man:mosquitto.conf(5)
            man:mosquitto(8)
    Process: 18926 ExecStartPre=/bin/mkdir -m 740 -p /var/log/mosquitto (code=exited, status=0/SUCCESS)
    Process: 18928 ExecStartPre=/bin/chown mosquitto:mosquitto /var/log/mosquitto (code=exited, status=0/SUCCESS)
    Process: 18930 ExecStartPre=/bin/mkdir -m 740 -p /run/mosquitto (code=exited, status=0/SUCCESS)
    Process: 18932 ExecStartPre=/bin/chown mosquitto:mosquitto /run/mosquitto (code=exited, status=0/SUCCESS)
   Main PID: 18934 (mosquitto)
      Tasks: 1 (limit: 1555)
         CPU: 2.891s
    CGroup: /system.slice/mosquitto.service
            └─18934 /usr/sbin/mosquitto -c /etc/mosquitto/mosquitto.conf

● pi@pi3b:~/fireiot $ cat /etc/mosquitto/conf.d/fireiot.conf
listener 1883
allow_anonymous true
pi@pi3b:~/fireiot $ mosquitto_sub -h localhost -p 1883 -t "test/topic" -v
test/topic hello mqtt

```

Hình 3.7: Tập cấu hình broker cục bộ của hệ thống.

3.2.4. Thiết kế cấu trúc topic và cấu hình bridge lên HiveMQ Cloud

Sau khi broker local hoạt động ổn định, nhóm tiến hành tổ chức topic theo dạng phân cấp để dễ quản lý dữ liệu theo từng phòng và từng loại thông số. Cấu trúc topic được lựa chọn là: *nha/<ten_phong>/<ten_thong_so>*

Ví dụ một số topic được sử dụng trong hệ thống: *nha/bep/nhietdo*, *nha/bep/gas*, *nha/bep/khoi*, *nha/phongkhach/nhietdo*...

Cách đặt topic này giúp dữ liệu được tách rõ theo từng khu vực và từng chỉ số. Khi cần theo dõi toàn bộ hệ thống, client có thể subscribe *nha/#*; khi chỉ cần theo dõi riêng một khu vực, có thể subscribe *nha/bep/#* hoặc *nha/phongkhach/#*. Đồng thời, chương trình *mqtt_to_sql.py* cũng có thể tách *room_name* và *metric_name* từ tên topic để ghi dữ liệu vào cơ sở dữ liệu một cách trực tiếp hơn.

Sau phần topic, nhóm tiếp tục cấu hình bridge để Mosquitto trên Raspberry Pi chuyển tiếp dữ liệu lên HiveMQ Cloud. Việc này giúp Raspberry Pi vẫn giữ vai trò là broker trung tâm trong mạng nội bộ, nhưng dữ liệu vẫn có thể được gửi lên cloud để ứng dụng từ xa theo dõi theo thời gian thực. Tập bridge được tạo trong thư mục */etc/mosquitto/conf.d/* với nội dung rút gọn như sau:

```

connection pi3b-bridge-01

address <host_hivemq_cloud>:8883

bridge_protocol_version mqttv311

bridge_cafile /etc/ssl/certs/ca-certificates.crt

try_private false

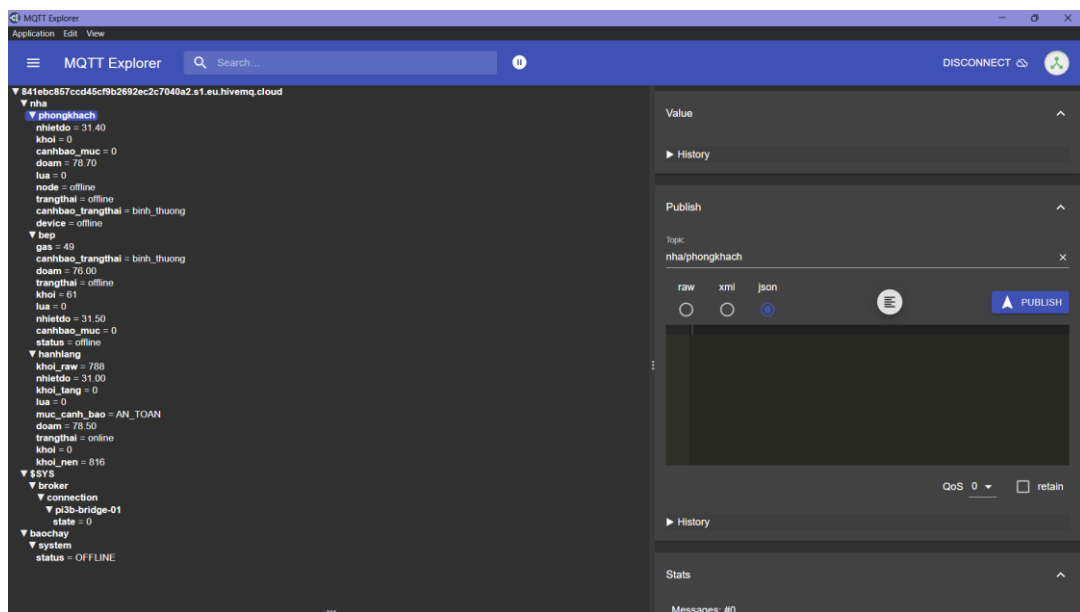
```

```
notifications true
remote_username <username_hivemq>
remote_password <password_hivemq>
remote_clientid pi3b-bridge-01
topic nha/# out 1
topic baochay/# out 1
```

Trong cấu hình trên, address là địa chỉ broker cloud, bridge_cafile dùng cho xác thực TLS, còn hai dòng topic nha/# out 1 và topic baochay/# out 1 cho biết chỉ những topic thuộc hệ thống báo cháy mới được đẩy từ Raspberry Pi lên cloud. Cách cấu hình này gọn hơn và tránh việc bridge dư thừa những topic không cần thiết. Sau khi tạo xong tệp bridge, broker được khởi động lại để nạp cấu hình mới:

```
sudo systemctl restart mosquitto
```

```
sudo systemctl status mosquitto --no-pager -l
```



Hình 3.8: Cấu trúc topic MQTT của hệ thống báo cháy theo từng khu vực và từng thông số

3.2.5. Kiểm tra hoạt động của broker sau cấu hình

Sau khi hoàn tất cài đặt và cấu hình, nhóm kiểm tra hoạt động của broker theo ba mức. Mức thứ nhất là kiểm tra dịch vụ Mosquitto trên Raspberry Pi có hoạt động ổn định hay không bằng lệnh `systemctl status`. Mức thứ hai là kiểm tra cơ chế publish/subscribe cục bộ bằng cấp lệnh `mosquitto sub` và

mosquitto_pub. Mức thứ ba là kiểm tra bridge cloud thông qua trạng thái kết nối và việc xuất hiện của các topic trên MQTT Explorer hoặc giao diện cloud broker. Quy trình kiểm tra theo từng lớp như vậy giúp xác định lỗi theo đúng vị trí xảy ra lỗi, tránh nhầm lẫn giữa lỗi broker local, lỗi mạng nội bộ, lỗi bridge hoặc lỗi client giám sát.

Trong quá trình vận hành, một số thao tác kiểm tra được sử dụng thường xuyên. Ví dụ, lệnh *hostname -I* được dùng để kiểm tra địa chỉ IP hiện tại của Raspberry Pi; lệnh *systemctl status mosquitto* dùng để kiểm tra dịch vụ broker local; lệnh *systemctl status fireiot-mqttsql.service* và *journalctl -u fireiot-mqttsql.service -f* dùng để theo dõi chương trình trung gian lưu dữ liệu vào MariaDB. Việc chuẩn hóa các thao tác kiểm tra và theo dõi log giúp hệ thống dễ bảo trì hơn trong giai đoạn thử nghiệm.

Bảng 3.1: Một số lệnh kiểm tra hoạt động của hệ thống MQTT Broker

Mục đích kiểm tra	Lệnh sử dụng
Kiểm tra broker local	<i>sudo systemctl status mosquitto --no-pager -l</i>
Kiểm tra IP Raspberry Pi	<i>hostname -I</i>
Kiểm tra bridge cloud	<i>mosquitto_sub -h localhost -p 1883 -t '\$SYS/broker/connection/#' -v</i>
Kiểm tra log chương trình trung gian	<i>journalctl -u fireiot-mqttsql.service -f</i>

3.3. THIẾT KẾ CƠ SỞ DỮ LIỆU:

Trong một hệ thống cảnh báo cháy dựa trên IoT, dữ liệu từ các nút cảm biến đóng vai trò cốt lõi để nhận diện nguy cơ và kích hoạt cảnh báo. Nếu MQTT broker đảm bảo dòng dữ liệu được truyền tải kịp thời, thì cơ sở dữ liệu là thành phần lưu giữ mọi thông tin đó một cách bền vững, phục vụ cho cả giám sát trực tuyến lẫn phân tích sau vận hành. Với định hướng triển khai mô hình ngay trên Raspberry Pi 3B+ – thiết bị vừa chạy Mosquitto broker, vừa vận hành chương trình trung gian *mqtt_to_sql.py* – việc thiết kế một lược đồ cơ sở dữ liệu hợp lý là bắt buộc để đảm bảo hiệu năng và khả năng mở rộng.

Toàn bộ dữ liệu được quản lý bởi hệ quản trị MariaDB, chạy cùng máy chủ Raspberry Pi. Khi các nút ESP32 publish dữ liệu cảm biến lên Mosquitto local, chương trình *mqtt_to_sql.py* sẽ subscribe các topic tương ứng, phân tích cú pháp tên topic, chuẩn hóa giá trị và ghi vào các bảng đã được định nghĩa trước. Do mỗi bản tin MQTT thường chỉ mang một thông số đơn lẻ (nhiệt độ,

khí gas, khói, lửa, trạng thái...), việc chỉ sử dụng một bảng duy nhất sẽ gây khó khăn cho việc truy vấn nhanh và tổng hợp dữ liệu.

Vì vậy, cơ sở dữ liệu được tổ chức thành năm bảng với những vai trò riêng biệt: quản lý thiết bị (*devices*), lưu trữ lịch sử toàn bộ bản tin MQTT (*mqtt_topic_history*), tra cứu nhanh giá trị mới nhất theo từng topic (*mqtt_topic_latest*), tổng hợp trạng thái giám sát theo phòng (*room_state_current*), lưu trữ các bản ghi cảm biến tổng hợp phục vụ phân tích (*sensor_readings*). Trong số này, ba bảng *devices*, *mqtt_topic_history* và *room_state_current* giữ vai trò trung tâm; hai bảng còn lại đóng vai trò hỗ trợ truy vấn và thống kê.

3.3.1. Mục tiêu và nguyên tắc thiết kế

Việc thiết kế cơ sở dữ liệu của hệ thống được thực hiện dựa trên yêu cầu thu thập dữ liệu liên tục từ nhiều node cảm biến và nhiều loại thông số khác nhau. Các node cảm biến được đặt tại các khu vực giám sát như bếp, phòng khách và gara. Mỗi node có thể gửi nhiều loại dữ liệu lên MQTT Broker, bao gồm nhiệt độ, độ ẩm, khí gas, khói, lửa, trạng thái hoạt động và mức cảnh báo. Để đáp ứng tốt cả nhu cầu giám sát hiện tại lẫn khả năng kiểm chứng, phân tích về sau, cơ sở dữ liệu cần thỏa mãn các mục tiêu:

Thứ nhất, hệ thống phải quản lý được thông tin của từng node cảm biến. Mỗi node cần có mã định danh riêng, tên khu vực lắp đặt và trạng thái hoạt động. Điều này giúp hệ thống xác định dữ liệu nhận được thuộc về phòng nào, node nào và node đó hiện đang online hay offline.

Thứ hai, hệ thống cần lưu được toàn bộ lịch sử bản tin MQTT nhận được từ broker local. Đây là yêu cầu quan trọng vì dữ liệu lịch sử giúp kiểm tra quá trình truyền nhận, đánh giá số lượng bản tin, phân tích khả năng mất gói và phục vụ các nội dung đo đạc trong chương 4.

Thứ ba, hệ thống cần có khả năng truy vấn nhanh trạng thái mới nhất. Trong quá trình giám sát, người dùng thường quan tâm đến trạng thái hiện tại của từng phòng, ví dụ phòng bếp đang an toàn hay cảnh báo, nhiệt độ hiện tại là bao nhiêu, cảm biến khí gas có vượt ngưỡng hay không. Nếu mỗi lần hiển thị đều truy vấn lại toàn bộ bảng lịch sử thì sẽ gây tốn tài nguyên và làm giảm hiệu năng hệ thống. Vì vậy, cần có bảng riêng để lưu trạng thái mới nhất.

Thứ tư, dữ liệu cần được tổ chức thuận lợi cho việc thống kê, vẽ biểu đồ và đánh giá kết quả thử nghiệm. Các thông số như nhiệt độ, khí gas, khói và

lửa cần có khả năng truy vấn theo thời gian, theo phòng hoặc theo node để phục vụ phân tích sau thử nghiệm.

Xuất phát từ các mục tiêu trên, cơ sở dữ liệu được phân thành ba nhóm chức năng chính:

- Nhóm quản lý thiết bị: Bảng *devices*
- Nhóm lưu dữ liệu MQTT theo topic: Bảng *mqtt_topic_history* (lưu toàn bộ lịch sử) và bảng *mqtt_topic_latest* (lưu giá trị mới nhất của mỗi topic).
- Nhóm tổng hợp trạng thái và dữ liệu đo: Bảng *room_state_current* (trạng thái tổng hợp theo phòng) và bảng *sensor_readings* (bản ghi cảm biến tổng hợp theo sự kiện).

Cách tổ chức này giúp hệ thống vừa lưu được dữ liệu gốc từ MQTT, vừa có dữ liệu đã được tổng hợp để phục vụ giám sát nhanh. Trong đó, bảng *mqtt_topic_history* được xem là bảng trung tâm của phần lưu trữ, vì mọi bản tin MQTT sau khi được chương trình trung gian tiếp nhận đều được ghi lại tại bảng này.

3.3.2. Cấu trúc các bảng dữ liệu

Trước khi tạo các bảng dữ liệu, nhóm tiến hành tạo cơ sở dữ liệu chính có tên *fire_iot*. Cơ sở dữ liệu này được đặt trên Raspberry Pi nhằm bảo đảm dữ liệu có thể được lưu cục bộ ngay sau khi broker nhận bản tin MQTT từ các node ESP32.

```
CREATE DATABASE IF NOT EXISTS fire_iot
CHARACTER SET utf8mb4
COLLATE utf8mb4_unicode_ci;

CREATE USER IF NOT EXISTS 'fireuser'@'localhost' IDENTIFIED BY
'YOUR_SQL_PASSWORD';

GRANT ALL PRIVILEGES ON fire_iot.* TO 'fireuser'@'localhost';

FLUSH PRIVILEGES;

USE fire_iot;
```

Trong đó, *fire_iot* là cơ sở dữ liệu chính của hệ thống. Tài khoản *fireuser* được dùng cho chương trình *mqtt_to_sql.py* khi kết nối đến MariaDB. Việc tạo tài khoản riêng cho chương trình trung gian giúp quá trình vận hành rõ ràng hơn, hạn chế việc sử dụng trực tiếp tài khoản quản trị trong mã nguồn.

a) Bảng lưu trữ thông tin thiết bị

Bảng lưu trữ thông tin thiết bị được sử dụng để quản lý danh tính và vị trí của từng node cảm biến trong hệ thống. Đây là bảng nền tảng, đóng vai trò liên kết giữa thiết bị vật lý ngoài thực tế và dữ liệu được lưu trong cơ sở dữ liệu.

Trong một hệ thống có nhiều khu vực giám sát, nếu chỉ lưu dữ liệu cảm biến mà không lưu thông tin thiết bị thì sẽ rất khó xác định: dữ liệu được gửi từ node nào, node đó được lắp đặt ở đâu, node nào đang hoạt động bình thường, node nào đã ngừng gửi dữ liệu. Do đó, bảng thiết bị là thành phần bắt buộc để đảm bảo tính quản lý và truy vết của hệ thống.

Bảng này được thiết kế với tên *devices*. Cấu trúc đề xuất như sau:

```
CREATE TABLE devices (
    id INT AUTO_INCREMENT PRIMARY KEY,
    device_id VARCHAR(50) NOT NULL UNIQUE,
    room_name VARCHAR(50) NOT NULL,
    location_detail VARCHAR(100) DEFAULT NULL,
    installed_at DATETIME DEFAULT CURRENT_TIMESTAMP,
    last_seen DATETIME DEFAULT NULL,
    node_status ENUM('online', 'offline') DEFAULT 'offline'
);
```

➤ Giải thích các trường dữ liệu:

Bảng 3.2: Mô tả các trường trong bảng lưu trữ thông tin thiết bị

Tên trường	Kiểu dữ liệu	Ý nghĩa
<i>device_id</i>	<i>VARCHAR(50)</i>	Mã định danh duy nhất của thiết bị
<i>room_name</i>	<i>VARCHAR(50)</i>	Tên khu vực lắp đặt thiết bị
<i>location_detail</i>	<i>VARCHAR(100)</i>	Mô tả chi tiết thiết bị lắp đặt
<i>installed_at</i>	<i>DATETIME</i>	Thời điểm thiết bị được thêm vào hệ thống
<i>last_seen</i>	<i>DATETIME</i>	Thời điểm gần nhất thiết bị gửi dữ liệu về
<i>node_status</i>	<i>ENUM</i>	Trạng thái hoạt động hiện tại của thiết bị: <i>online</i> hoặc <i>offline</i>

Trong hệ thống, bảng *devices* không trực tiếp lưu dữ liệu cảm biến, nhưng lại đóng vai trò nền tảng để quản lý các node. Khi chương trình trung gian nhận được dữ liệu từ một phòng, hệ thống có thể cập nhật lại trường *last_seen* và

node_status. Nhờ đó, người vận hành có thể biết node nào đang hoạt động bình thường, node nào mất kết nối hoặc chưa gửi dữ liệu trong một khoảng thời gian nhất định.

Ngay sau khi tạo bảng, dữ liệu của ba nút được nạp sẵn để hệ thống có thể đối chiếu ngay khi nhận được bản tin đầu tiên:

```
INSERT INTO devices (device_id, room_name, location_detail, node_status)
VALUES
    ('ROOM_BEP', 'bep', 'Node phong bep', 'offline'),
    ('ROOM_PHONGKHACH', 'phongkhach', 'Node phong khach', 'offline'),
    ('ROOM_GARA', 'gara', 'Node gara', 'offline')
ON DUPLICATE KEY UPDATE
    room_name = VALUES(room_name),
    location_detail = VALUES(location_detail),
    node_status = VALUES(node_status);
```

Việc thêm trước danh sách node giúp hệ thống có sẵn dữ liệu định danh để đối chiếu khi dữ liệu MQTT được ghi vào các bảng khác. Đây cũng là cơ sở để liên kết dữ liệu đo đạc với từng thiết bị trong bảng *sensor_readings*.

b) Bảng lưu trữ dữ liệu lịch sử

Bảng lưu trữ dữ liệu lịch sử là bảng quan trọng nhất trong mô hình cơ sở dữ liệu. Bảng này dùng để lưu toàn bộ lịch sử bản tin MQTT mà chương trình *mqtt_to_sql.py* nhận được từ Mosquitto local.

Mỗi bản tin MQTT sau khi được tiếp nhận sẽ được lưu lại với đầy đủ các thông tin: topic gốc, tên phòng, tên thông số, payload dạng chuỗi, payload dạng số nếu có thể chuyển đổi, QoS, trạng thái retained và thời điểm nhận bản tin.

Bảng này được định nghĩa như sau:

```
CREATE TABLE mqtt_topic_history (
    id BIGINT AUTO_INCREMENT PRIMARY KEY,
    topic VARCHAR(255) NOT NULL,
    room_name VARCHAR(50) DEFAULT NULL,
    metric_name VARCHAR(50) DEFAULT NULL,
    payload_text TEXT NOT NULL,
```

```

    payload_num DOUBLE DEFAULT NULL,
    qos TINYINT DEFAULT 0,
    retained TINYINT(1) DEFAULT 0,
    received_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP
);

CREATE INDEX idx_mqtt_topic_time
ON mqtt_topic_history(topic, received_at);

CREATE INDEX idx_mqtt_room_metric_time
ON mqtt_topic_history(room_name, metric_name, received_at);

```

➤ Giải thích các trường dữ liệu:

Bảng 3.3: Mô tả các trường trong bảng mqtt_topic_history

Tên trường	Kiểu dữ liệu	Ý nghĩa
<i>id</i>	<i>BIGINT</i>	Khóa chính tự tăng, phù hợp với số lượng bản tin lớn
<i>topic</i>	<i>VARCHAR(255)</i>	Topic MQTT gốc mà hệ thống nhận được
<i>room_name</i>	<i>VARCHAR(50)</i>	Tên phòng được tách từ topic
<i>metric_name</i>	<i>VARCHAR(50)</i>	Tên thông số (nhiệt độ, gas, khói...) được trích xuất từ topic
<i>payload_text</i>	<i>TEXT</i>	Nội dung payload dạng chuỗi
<i>payload_num</i>	<i>DOUBLE</i>	Giá trị số nếu payload có thể chuyển đổi (NULL nếu không)
<i>qos</i>	<i>TINYINT</i>	Mức QoS của bản tin MQTT
<i>retained</i>	<i>TINYINT(1)</i>	Cho biết bản tin có phải retained message hay không
<i>received_at</i>	<i>DATETIME</i>	Thời điểm bản tin được ghi vào cơ sở dữ liệu

Bảng này có vai trò rất quan trọng trong việc đánh giá hoạt động của hệ thống. Khi cần kiểm tra ESP32 có gửi dữ liệu đều hay không, broker local có nhận được bản tin hay không, hoặc số lượng bản tin lưu thực tế có khớp với số lượng bản tin kỳ vọng hay không, nhóm có thể truy vấn trực tiếp bảng *mqtt_topic_history*.

Ví dụ, nếu node gara gửi dữ liệu nhiệt độ qua topic: *nha/gara/nhietdo* với payload: *31.5* thì bảng *mqtt_topic_history* sẽ lưu lại một bản ghi có dạng:

topic	room_name	metric_name	payload_text	payload_num
nha/gara/nhietdo	gara	nhietdo	31.5	31.5

Việc lưu đồng thời *payload_text* và *payload_num* giúp hệ thống xử lý linh hoạt nhiều loại dữ liệu. Với các payload dạng số như nhiệt độ, khí gas, khói hoặc lửa, hệ thống có thể sử dụng *payload_num* để thống kê và vẽ biểu đồ. Với các payload dạng chuỗi như *online*, *offline*, *binh_thuong*, *canh_bao*, hệ thống vẫn lưu được nguyên bản trong *payload_text*.

Hai chỉ mục *idx_mqtt_topic_time* và *idx_mqtt_room_metric_time* giúp tối ưu các câu lệnh tìm kiếm thường gặp như lấy các bản tin mới nhất của một topic, thống kê dữ liệu của một phòng hay một loại cảm biến trong khoảng thời gian.

c) Bảng lưu trữ trạng thái theo từng phòng

Bảng *room_state_current* dùng để lưu trạng thái hiện tại của từng khu vực giám sát. Đây là bảng quan trọng vì nó cho phép truy vấn nhanh dữ liệu tổng hợp của từng phòng mà không cần quét toàn bộ bảng lịch sử.

Khác với *mqtt_topic_history*, bảng này không lưu từng bản tin MQTT riêng lẻ mà gom các thông số mới nhất của cùng một phòng vào một bản ghi. Mỗi phòng chỉ có một dòng dữ liệu duy nhất. Khi có thông số mới được gửi lên, chương trình trung gian sẽ cập nhật vào cột tương ứng của phòng đó.

```
CREATE TABLE room_state_current (
    room_name VARCHAR(50) PRIMARY KEY,
    nhietdo DECIMAL(6,2) DEFAULT NULL,
    doam DECIMAL(6,2) DEFAULT NULL,
    gas INT DEFAULT NULL,
    khoi INT DEFAULT NULL,
    khoi_raw INT DEFAULT NULL,
    khoi_tang INT DEFAULT NULL,
    khoi_nen INT DEFAULT NULL,
    lua INT DEFAULT NULL,
```

```

    canhbao_muc INT DEFAULT NULL,
    canhbao_trangthai VARCHAR(50) DEFAULT NULL,
    muc_canh_bao VARCHAR(50) DEFAULT NULL,
    trangthai VARCHAR(50) DEFAULT NULL,
    status VARCHAR(50) DEFAULT NULL,
    node VARCHAR(50) DEFAULT NULL,
    device VARCHAR(50) DEFAULT NULL,
    updated_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP
    ON UPDATE CURRENT_TIMESTAMP
);

```

Bảng 3.4: Cấu trúc bảng lưu trạng thái hiện tại theo phòng

Tên trường	Kiểu dữ liệu	Ý nghĩa
<i>room_name</i>	<i>VARCHAR(50)</i>	Tên phòng, đồng thời là khóa chính
<i>nhietdo</i>	<i>DECIMAL(6,2)</i>	Giá trị nhiệt độ mới nhất
<i>doam</i>	<i>DECIMAL(6,2)</i>	Giá trị độ ẩm mới nhất
<i>gas</i>	<i>INT</i>	Giá trị cảm biến khí gas mới nhất
<i>khoi</i>	<i>INT</i>	Giá trị cảm biến khói mới nhất
<i>Lua</i>	<i>INT</i>	Giá trị cảm biến lửa mới nhất
<i>canhbao_muc</i>	<i>INT</i>	Mức cảnh báo dạng số
<i>canhbao_trangthai</i>	<i>VARCHAR(50)</i>	Trạng thái cảnh báo dạng chuỗi
<i>trangthai</i>	<i>VARCHAR(50)</i>	Trạng thái hoạt động của node
<i>updated_at</i>	<i>DATETIME</i>	Thời điểm cập nhật gần nhất

Bảng *room_state_current* đặc biệt phù hợp với chức năng giám sát trạng thái hiện tại. Ví dụ, khi cần biết tình trạng của phòng bếp, hệ thống chỉ cần truy vấn một dòng có *room_name* = 'bep'. Dữ liệu trả về có thể bao gồm nhiệt độ, khí gas, khói, lửa và trạng thái cảnh báo mới nhất của phòng đó.

Ví dụ dữ liệu tổng hợp theo phòng có thể có dạng:

room_name	nhietdo	gas	khoi	lua	canhbao_trangthai	trangthai
bep	34.2	420	180	0	binh_thuong	online
phongkhach	31.0	210	95	0	binh_thuong	online
gara	38.5	1250	890	1	canh_bao	online

Nhờ bảng này, hệ thống không cần ghép nhiều bản tin riêng lẻ từ bảng lịch sử để xác định trạng thái hiện tại của từng phòng. Điều này giúp giảm tải truy vấn và phù hợp với các chức năng hiển thị nhanh trên giao diện giám sát.

d) Các bảng hỗ trợ truy vấn và thống kê

Bên cạnh ba bảng chính, cơ sở dữ liệu còn sử dụng thêm hai bảng hỗ trợ là *mqtt_topic_latest* và *sensor_readings*.

Bảng *mqtt_topic_latest* dùng để lưu giá trị mới nhất của từng topic MQTT. Nếu *mqtt_topic_history* lưu toàn bộ lịch sử bản tin, thì *mqtt_topic_latest* chỉ giữ lại bản ghi mới nhất tương ứng với mỗi topic. Bảng này giúp truy vấn nhanh giá trị hiện tại của một topic cụ thể, ví dụ *nha/bep/nhietdo* hoặc *nha/gara/trangthai*, mà không cần sắp xếp và tìm kiếm trong bảng lịch sử.

```
CREATE TABLE mqtt_topic_latest (
    topic VARCHAR(255) PRIMARY KEY,
    room_name VARCHAR(50) DEFAULT NULL,
    metric_name VARCHAR(50) DEFAULT NULL,
    payload_text TEXT NOT NULL,
    payload_num DOUBLE DEFAULT NULL,
    qos TINYINT DEFAULT 0,
    retained TINYINT(1) DEFAULT 0,
    updated_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP
    ON UPDATE CURRENT_TIMESTAMP
);
```

Trong khi đó, bảng *sensor_readings* dùng để lưu các bản ghi cảm biến tổng hợp theo thời gian. Mỗi bản ghi tương ứng với một lần ghi nhận dữ liệu của một node, bao gồm nhiệt độ, khí gas, khói, lửa và trạng thái hệ thống. Bảng này phù hợp khi cần thống kê dữ liệu theo thiết bị, theo thời gian hoặc phục vụ vẽ biểu đồ.

```
CREATE TABLE sensor_readings (
```

```

    id BIGINT AUTO_INCREMENT PRIMARY KEY,
    device_id VARCHAR(50) NOT NULL,
    recorded_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
    temperature DECIMAL(6,2) DEFAULT NULL,
    gas_value INT DEFAULT NULL,
    smoke_value INT DEFAULT NULL,
    flame_value INT DEFAULT NULL,
    system_status ENUM('SAFE', 'WARNING', 'DANGER') NOT NULL DEFAULT
'SAFE',
    CONSTRAINT fk_sensor_device
        FOREIGN KEY (device_id) REFERENCES devices(device_id)
        ON UPDATE CASCADE
        ON DELETE RESTRICT
);

CREATE INDEX idx_sensor_device_time
ON sensor_readings(device_id, recorded_at);

```

3.3.3. Quan hệ dữ liệu và vai trò của các bảng trong hệ thống

Các bảng trong cơ sở dữ liệu được thiết kế theo hướng tách biệt giữa dữ liệu quản lý thiết bị, dữ liệu lịch sử bản tin MQTT và dữ liệu trạng thái hiện tại theo khu vực giám sát. Cách tổ chức này giúp hệ thống vừa lưu trữ đầy đủ dữ liệu gốc, vừa hỗ trợ truy vấn nhanh các thông tin cần thiết trong quá trình giám sát và đánh giá.

Bảng *devices* là bảng quản lý thông tin các node cảm biến. Mỗi node được định danh bằng một *device_id* duy nhất và gắn với một khu vực lắp đặt cụ thể như bếp, phòng khách hoặc gara. Bảng này giúp hệ thống xác định nguồn phát sinh dữ liệu và theo dõi trạng thái hoạt động của từng node thông qua các trường như *last_seen* và *node_status*.

Bảng *mqtt_topic_history* là bảng lưu trữ lịch sử bản tin MQTT. Mỗi dòng trong bảng tương ứng với một bản tin được hệ thống tiếp nhận, bao gồm topic gốc, tên phòng, tên thông số, payload và thời điểm ghi nhận. Đây là bảng có vai trò trung tâm trong quá trình kiểm chứng dữ liệu vì nó cho phép đối chiếu

số lượng bản tin thực tế, kiểm tra dữ liệu theo từng topic và phục vụ đánh giá độ ổn định của quá trình truyền nhận.

Bảng *room_state_current* lưu trạng thái mới nhất của từng phòng. Khác với bảng lịch sử, bảng này chỉ duy trì một bản ghi cho mỗi khu vực giám sát. Các trường như *nhietdo*, *gas*, *khoi*, *lua*, *canhbao_muc* và *canhbao_trangthai* phản ánh trạng thái hiện tại của phòng đó. Bảng này phù hợp cho các thao tác truy vấn nhanh, đặc biệt trong trường hợp cần hiển thị trạng thái tổng quan của hệ thống.

Hai bảng *mqtt_topic_latest* và *sensor_readings* có vai trò hỗ trợ. Bảng *mqtt_topic_latest* lưu giá trị mới nhất của từng topic, giúp truy vấn nhanh dữ liệu theo từng nhánh MQTT. Trong khi đó, bảng *sensor_readings* lưu các bản ghi cảm biến tổng hợp theo thời gian, phù hợp cho thống kê, vẽ biểu đồ hoặc đánh giá xu hướng dữ liệu.

Bảng 3.5: Tổng hợp chức năng các bảng trong cơ sở dữ liệu

Tên bảng	Mức độ vai trò	Chức năng chính
<i>devices</i>	Bảng chính	Quản lý thông tin node cảm biến, phòng lắp đặt và trạng thái online/offline
<i>mqtt_topic_history</i>	Bảng chính	Lưu toàn bộ lịch sử bản tin MQTT nhận được từ broker local
<i>room_state_current</i>	Bảng chính	Lưu trạng thái hiện tại theo từng phòng để phục vụ giám sát nhanh
<i>mqtt_topic_latest</i>	Bảng hỗ trợ	Lưu giá trị mới nhất của từng topic MQTT
<i>sensor_readings</i>	Bảng hỗ trợ	Lưu bản ghi cảm biến tổng hợp theo thời gian để phục vụ thống kê, biểu đồ

3.3.4. Kiểm tra và đánh giá thiết kế cơ sở dữ liệu

Sau khi tạo cơ sở dữ liệu và các bảng chính, nhóm tiến hành kiểm tra cấu trúc bằng các lệnh SQL cơ bản. Trước hết, kiểm tra danh sách bảng trong cơ sở

dữ liệu bằng lệnh “*SHOW TABLES;*” và kết quả mong đợi là cơ sở dữ liệu hiển thị đủ năm bảng đã thiết kế ở trên. Tiếp theo, để kiểm tra cấu trúc của từng bảng thì sử dụng lệnh “*DESCRIBE <tên_bảng>;*”, giúp đối chiếu tên trường, kiểu dữ liệu, khóa chính và khóa ngoại với định nghĩa ban đầu. Ví dụ: *DESCRIBE devices;*

Để kiểm tra khả năng lưu trữ và truy xuất dữ liệu sau khi thiết kế cơ sở dữ liệu, nhóm sử dụng một số câu lệnh truy vấn cơ bản trên MariaDB. Các truy vấn này tập trung vào việc kiểm tra dữ liệu lịch sử MQTT, trạng thái hiện tại theo từng phòng, dữ liệu cảm biến tổng hợp và số lượng bản tin đã được lưu trong hệ thống. Cụ thể, các lệnh kiểm tra được thực hiện như sau:

```
SELECT * FROM mqtt_topic_history ORDER BY received_at DESC LIMIT 20;
```

```
SELECT * FROM room_state_current;
```

```
SELECT * FROM sensor_readings ORDER BY recorded_at DESC LIMIT 20;
```

```
SELECT topic, COUNT(*) AS so_luong_ban_tin FROM mqtt_topic_history GROUP BY topic ORDER BY so_luong_ban_tin DESC;
```

```
SELECT room_name, COUNT(*) AS so_luong_ban_tin FROM mqtt_topic_history GROUP BY room_name;
```

Nhìn chung, mô hình cơ sở dữ liệu hiện tại đã đáp ứng đồng thời ba yêu cầu: (1) lưu trữ bền vững và đầy đủ lịch sử bản tin, (2) truy vấn nhanh trạng thái thời gian thực, (3) tổ chức dữ liệu thuận tiện cho phân tích và đánh giá hiệu năng. Nhờ kiến trúc phân tách rõ ràng giữa các bảng, hệ thống có thể mở rộng thêm nút cảm biến hoặc thêm loại dữ liệu mới mà không phải thay đổi cấu trúc cốt lõi. Mô hình này cũng phù hợp với tài nguyên hạn chế của Raspberry Pi 3B+, khi mà các bảng hỗ trợ được thiết kế để giảm tải truy vấn lặp lại trên bảng lịch sử khổng lồ.

Tổng kết lại, cơ sở dữ liệu không chỉ là nơi lưu trữ mà còn là một mắt xích quan trọng trong kiến trúc hệ thống, kết nối chặt chẽ với MQTT broker và chương trình trung gian để hình thành một pipeline dữ liệu hoàn chỉnh, sẵn sàng cho các phép kiểm thử và đánh giá ở các chương tiếp theo.

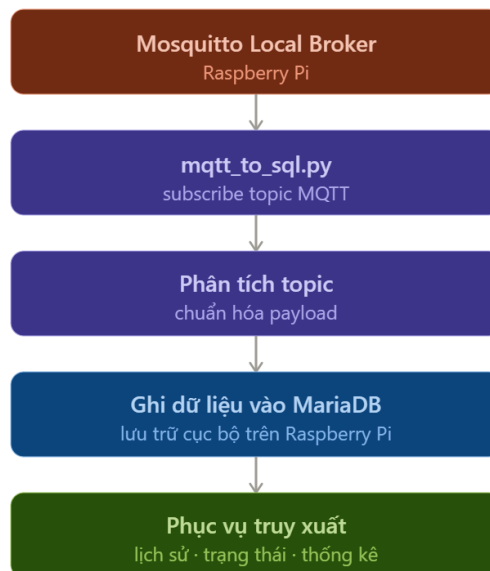
3.4. XÂY DỰNG PHẦN TRUNG GIAN ĐỂ TRUY XUẤT DỮ LIỆU.

Trong kiến trúc hệ thống, dữ liệu từ các node ESP32 được gửi về Mosquitto Broker cục bộ trên Raspberry Pi dưới dạng các bản tin MQTT. Tuy nhiên, MQTT Broker chỉ đảm nhiệm vai trò tiếp nhận và phân phối dữ liệu theo

cơ chế publish/subscribe, không trực tiếp lưu trữ dữ liệu lâu dài. Vì vậy, để dữ liệu cảm biến có thể được truy xuất, thống kê và đánh giá sau thử nghiệm, cần xây dựng một chương trình trung gian làm nhiệm vụ nhận dữ liệu từ broker và ghi vào cơ sở dữ liệu MariaDB.

Chương trình trung gian được xây dựng bằng ngôn ngữ Python với tên file *mqtt_to_sql.py*, chạy trực tiếp trên Raspberry Pi. Chương trình này subscribe các topic từ Mosquitto local, tiếp nhận payload từ các node cảm biến, phân tích cấu trúc topic, chuẩn hóa dữ liệu và ghi vào các bảng trong MariaDB. Nhờ đó, dữ liệu MQTT dạng rời rạc được chuyển thành dữ liệu có cấu trúc, phục vụ cho việc lưu trữ lịch sử, cập nhật trạng thái hiện tại và hỗ trợ phân tích ở các phần sau.

Luồng xử lý tổng quát của chương trình trung gian được mô tả như sau:



Hình 3.9: Sơ đồ khối hoạt động của chương trình trung gian *mqtt_to_sql.py*

3.4.1. Vai trò và môi trường triển khai của chương trình trung gian

Chương trình *mqtt_to_sql.py* đóng vai trò cầu nối giữa tầng truyền thông MQTT và tầng lưu trữ dữ liệu. Trong hệ thống, các node ESP32 chỉ cần gửi dữ liệu lên Mosquitto local theo đúng topic đã quy ước. Chương trình trung gian sẽ đảm nhiệm phần còn lại, bao gồm nhận bản tin, xử lý nội dung và ghi dữ liệu vào cơ sở dữ liệu.

Về mặt chức năng, chương trình được xây dựng nhằm thực hiện các nhiệm vụ chính trong quá trình thu thập và xử lý dữ liệu của hệ thống. Trước hết, chương trình thiết lập kết nối đến Mosquitto Broker cục bộ được triển khai trên

Raspberry Pi. Sau đó, chương trình tiến hành đăng ký (subscribe) các nhánh topic cần giám sát để tiếp nhận dữ liệu từ các node cảm biến theo thời gian thực.

Dữ liệu sau khi tiếp nhận sẽ được phân tích cấu trúc topic để xác định vị trí và loại thông số, đồng thời chuẩn hóa payload về các định dạng số thực, số nguyên hoặc chuỗi nhằm đảm bảo tính đồng bộ. Sau đó, thông tin được lưu trữ vào hệ quản trị MariaDB, kết hợp với việc cập nhật trạng thái hoạt động của các node và tạo lập các bản ghi tổng hợp để phục vụ nhu cầu truy vấn, thống kê.

Chương trình được triển khai trong thư mục dự án trên Raspberry Pi, cụ thể: `/home/pi/fireiot`

Về mặt công nghệ, chương trình sử dụng các thư viện Python chính bao gồm *paho-mqtt* cho việc giao tiếp với giao thức MQTT và *pymysql* để thực hiện các thao tác với cơ sở dữ liệu MariaDB. Phần khai báo thư viện và các tham số kết nối cơ bản được trình bày như sau:

```
import pymysql

import paho.mqtt.client as mqtt

MQTT_BROKER = "localhost"

MQTT_PORT = 1883

MQTT_SUB_TOPICS = [("nha/#", 1), ("baochay/#", 1)]

DB_HOST = "localhost"

DB_USER = "fireuser"

DB_PASS = "YOUR_SQL_PASSWORD"

DB_NAME = "fire_iot"
```

Trong đó, *MQTT_BROKER* = "localhost" cho biết chương trình kết nối đến broker MQTT cục bộ đang chạy trên chính Raspberry Pi. Cổng 1883 là cổng MQTT local đã được cấu hình ở mục 3.2. Danh sách *MQTT_SUB_TOPICS* gồm hai nhánh topic chính là *nha/#* và *baochay/#*, giúp chương trình tiếp nhận dữ liệu cảm biến và dữ liệu cảnh báo của hệ thống.

Phần cấu hình cơ sở dữ liệu gồm địa chỉ máy chủ MariaDB, tên tài khoản, mật khẩu và tên cơ sở dữ liệu. Trong quá trình triển khai thực tế, mật khẩu cần được thay bằng mật khẩu đã tạo cho tài khoản fireuser.

3.4.2. Thiết kế luồng xử lý bản tin MQTT

mqtt_to_sql.py hoạt động theo mô hình hướng sự kiện. Ngay sau khi kết nối thành công đến Mosquitto Broker, chương trình tự động đăng ký tất cả các topic đã khai báo thông qua hàm *on_connect()*. Mỗi khi có bản tin mới được phát từ các node ESP32, hàm callback *on_message()* sẽ được gọi để tiếp nhận và xử lý.

Quá trình kết nối và subscribe MQTT được thực hiện thông qua hàm *on_connect()*:

```
def on_connect(client, userdata, flags, reason_code, properties=None):
    print("MQTT LOCAL connected, rc =", reason_code)
    for topic, qos in MQTT_SUB_TOPICS:
        client.subscribe(topic, qos=qos)
        print("Subscribed to:", topic)
```

Với cấu hình này, chương trình sẽ nhận được toàn bộ bản tin thuộc hai không gian *nha/#* và *baochay/#*. Trong đó *nha/#* dành cho dữ liệu cảm biến và trạng thái theo từng phòng, *baochay/#* dành cho các bản tin cảnh báo hoặc thông tin hệ thống.

Khi một bản tin MQTT đến, hàm *on_message()* thực thi ba bước xử lý cơ bản. Thứ nhất, topic được chuẩn hóa qua hàm *normalize_topic()* để loại bỏ các khác biệt không cần thiết. Thứ hai, payload được giải mã từ dạng byte sang chuỗi với bảng mã UTF-8. Thứ ba, chương trình phân tích topic để trích xuất tên phòng (*room_name*) và loại thông số (*metric_name*) thông qua hàm *parse_topic()*.

```
def on_message(client, userdata, msg):
    topic = normalize_topic(msg.topic)
    payload_text = msg.payload.decode("utf-8", errors="replace").strip()
    room_name, metric_name = parse_topic(topic)
```

Cấu trúc topic chính của hệ thống tuân theo khuôn mẫu: *nha/<ten_phong>/<ten_thong_so>*. Ví dụ: *nha/bep/nhietdo*, *nha/bep/gas*, *nha/phongkhach/khoi*, *nha/gara/lua*, *nha/gara/trangthai*...

Từ một topic như *nha/gara/nhietdo*, hàm phân tích sẽ trả về *room_name* = "*gara*" và *metric_name* = "*nhietdo*". Nhằm đảm bảo tính nhất quán của dữ

liệu, chương trình tích hợp cơ chế chuẩn hóa tên phòng. Trong quá trình phát triển, có thể tồn tại những tên gọi khác nhau cho cùng một khu vực (ví dụ *hanhlang*, *hallway*). Bảng ánh xạ *ROOM_NAME_MAP* được định nghĩa để quy đổi tự động về tên chuẩn:

```
ROOM_NAME_MAP = {
    "hanhLang": "gara",
    "hallway": "gara",
}

def normalize_room_name(room_name: str):
    if room_name is None:
        return None

    room_name = room_name.strip().lower()
    return ROOM_NAME_MAP.get(room_name, room_name)
```

Nhờ đó, toàn bộ dữ liệu liên quan đến cùng một không gian vật lý sẽ được ghi nhận dưới một mã định danh duy nhất, tránh phân mảnh thông tin trong cơ sở dữ liệu.

Bên cạnh chuẩn hóa tên phòng, chương trình còn quản lý việc chuyển đổi payload sang kiểu số khi cần thiết. Các trường như *nhietdo*, *doam* được chuyển thành số thực; *gas*, *khoi*, *lua* thành số nguyên; trong khi các trạng thái như *online*, *offline*, *binh_thuong* được giữ nguyên dạng chuỗi. Việc phân loại này được thực hiện thông qua hai tập hợp:

```
NUMERIC_INT_FIELDS = {
    "gas",    "khoi",    "khoi_raw",    "khoi_tang",    "khoi_nen",    "lua",
    "canhbao_muc"
}

NUMERIC_FLOAT_FIELDS = {
    "nhietdo", "doam"
}
```

Nhờ bước chuẩn hóa này, dữ liệu sau khi lưu trữ trong MariaDB vừa có thể phục vụ hiển thị trạng thái dạng văn bản, vừa sẵn sàng cho các phép toán thống kê trên các giá trị đo dạng số.

3.4.3. Cơ chế ghi dữ liệu vào MariaDB

Sau khi bản tin MQTT được tiếp nhận và chuẩn hóa, chương trình thực hiện ghi dữ liệu vào MariaDB. Nội dung này là phần quan trọng nhất của chương trình trung gian, vì nó quyết định dữ liệu MQTT có được chuyển thành dữ liệu có cấu trúc hay không.

Kết nối đến MariaDB được quản lý thông qua hàm `get_db_connection()` với cơ chế tự động xác nhận (*autocommit*) để đảm bảo mỗi thao tác ghi được thực thi ngay lập tức:

```
def get_db_connection():
    return pymysql.connect(
        host=DB_HOST,
        user=DB_USER,
        password=DB_PASS,
        database=DB_NAME,
        charset="utf8mb4",
        autocommit=True
    )
```

Sau khi có kết nối cơ sở dữ liệu, bản tin được xử lý và ghi vào các bảng tương ứng. Trong hàm `on_message()`, các thao tác ghi dữ liệu được gọi theo thứ tự:

```
save_topic_history(conn, topic, room_name, metric_name, payload_text,
msg.qos, msg.retain)

upsert_topic_latest(conn, topic, room_name, metric_name, payload_text,
msg.qos, msg.retain)

upsert_room_state(conn, room_name, metric_name, payload_text)

upsert_device_status(conn, room_name, metric_name, payload_text)

insert_sensor_reading_snapshot(conn, room_name, metric_name)
```

Vai trò của từng hàm được tóm tắt trong bảng dưới đây:

Hàm xử lý	Vai trò
<code>save_topic_history()</code>	Ghi toàn bộ bản tin MQTT vào bảng <code>mqtt_topic_history</code>

<code>upsert_topic_latest()</code>	Cập nhật giá trị mới nhất của từng topic vào bảng <i>mqtt_topic_latest</i>
<code>upsert_room_state()</code>	Cập nhật trạng thái hiện tại theo phòng vào bảng <i>room_state_current</i>
<code>upsert_device_status()</code>	Cập nhật trạng thái <i>online/offline</i> của node trong bảng <i>devices</i>
<code>insert_sensor_reading_snapshot()</code>	Tạo bản ghi tổng hợp trong bảng <i>sensor_readings</i> khi có dữ liệu phù hợp

Trước hết, hàm `save_topic_history()` ghi toàn bộ bản tin MQTT vào bảng lịch sử. Đây là bước bắt buộc đối với mọi bản tin hợp lệ, vì bảng *mqtt_topic_history* được sử dụng để kiểm chứng quá trình truyền nhận dữ liệu và phục vụ đánh giá ở chương sau.

```
def save_topic_history(conn, topic, room_name, metric_name, payload_text,
qos, retained):
```

```
    topic = normalize_topic(topic)
    room_name = normalize_room_name(room_name)
    payload_num = extract_payload_num(metric_name, payload_text)
    with conn.cursor() as cur:
        cur.execute("""
            INSERT INTO mqtt_topic_history
                (topic, room_name, metric_name, payload_text, payload_num,
qos, retained, received_at)
            VALUES (%s, %s, %s, %s, %s, %s, %s, NOW())
            """, (topic, room_name, metric_name, payload_text, payload_num,
qos, int(retained)))
```

Trong đó, *payload_text* lưu payload gốc dưới dạng chuỗi, còn *payload_num* lưu giá trị số nếu payload có thể chuyển đổi được. Cách lưu này giúp hệ thống vừa giữ được dữ liệu nguyên bản, vừa thuận tiện cho thống kê.

Tiếp theo, chương trình cập nhật bảng *mqtt_topic_latest*. Bảng này chỉ lưu giá trị mới nhất của từng topic, vì vậy chương trình sử dụng cơ chế *ON DUPLICATE KEY UPDATE*. Nếu topic chưa tồn tại, hệ thống sẽ thêm mới; nếu topic đã tồn tại, hệ thống sẽ cập nhật payload và thời điểm mới nhất.

```

def upsert_topic_latest(conn, topic, room_name, metric_name,
payload_text, qos, retained):

    topic = normalize_topic(topic)

    room_name = normalize_room_name(room_name)

    payload_num = extract_payload_num(metric_name, payload_text)

    with conn.cursor() as cur:

        cur.execute("""

            INSERT INTO mqtt_topic_latest

            (topic, room_name, metric_name, payload_text, payload_num,
qos, retained, updated_at)

            VALUES (%s, %s, %s, %s, %s, %s, %s, NOW())

            ON DUPLICATE KEY UPDATE

                room_name = VALUES(room_name),

                metric_name = VALUES(metric_name),

                payload_text = VALUES(payload_text),

                payload_num = VALUES(payload_num),

                qos = VALUES(qos),

                retained = VALUES(retained),

                updated_at = NOW()

            """, (topic, room_name, metric_name, payload_text, payload_num,
qos, int(retained)))

```

Sau đó, trạng thái của từng phòng được cập nhật vào bảng *room_state_current*. Bảng này được thiết kế với các cột tương ứng với từng loại thông số (ví dụ: *nhietdo*, *doam*, *gas*, *khoi*, *lua*). Ánh xạ giữa *metric_name* và tên cột được định nghĩa trong từ điển *ROOM_COLUMNS*:

```

ROOM_COLUMNS = {

    "nhietdo": "nhietdo",

    "doam": "doam",

    "gas": "gas",

    "khoi": "khoi",

```

```

    "lua": "lua",
    "canhbao_muc": "canhbao_muc",
    "canhbao_trangthai": "canhbao_trangthai",
    "trangthai": "trangthai",
    "status": "status",
    "node": "node",
    "device": "device",
}

```

Cách ánh xạ này giúp chương trình xác định metric nào sẽ được ghi vào cột nào trong bảng *room_state_current*. Nếu metric không nằm trong danh sách được định nghĩa, chương trình sẽ bỏ qua để tránh ghi sai cột.

Đối với trạng thái thiết bị, hàm *upsert_device_status()* được sử dụng để cập nhật bảng *devices*. Khi payload là online, node được cập nhật trạng thái online; các trạng thái khác được xử lý là offline.

```

def upsert_device_status(conn, room_name, metric_name, payload_text):
    room_name = normalize_room_name(room_name)

    if room_name is None or metric_name not in {"trangthai", "status",
        "node", "device"}:
        return

    state = payload_text.strip().lower()
    node_status = "online" if state == "online" else "offline"
    device_id = f"ROOM_{room_name.upper()}"

    # Thực hiện INSERT hoặc UPDATE vào bảng devices

```

Cuối cùng, để hỗ trợ các truy vấn thống kê và hiển thị biểu đồ, chương trình tạo bản ghi tổng hợp trong bảng *sensor_readings*. Khi phát hiện một bản tin mang tính chất cảnh báo hoặc thay đổi trạng thái quan trọng, hàm *insert_sensor_reading_snapshot()* sẽ truy vấn các giá trị hiện tại của phòng từ *room_state_current* và tạo một bản ghi chứa đồng thời nhiệt độ, nồng độ gas, khói, lửa và trạng thái hệ thống. Cách làm này cung cấp một lát cắt dữ liệu có ý nghĩa tại từng thời điểm, rất hữu ích cho các phân tích xu hướng và phát hiện sự kiện bất thường.

Tổng kết lại, cơ chế ghi dữ liệu của *mqtt_to_sql.py* không chỉ dừng ở việc lưu từng bản tin riêng lẻ mà còn tổ chức dữ liệu theo nhiều tầng: nhật ký đầy đủ, giá trị mới nhất, trạng thái phòng, trạng thái thiết bị và bản ghi tổng hợp. Cách tiếp cận này đảm bảo dữ liệu sau khi được lưu trữ có thể phục vụ linh hoạt cho cả giám sát tức thời lẫn phân tích chuyên sâu.

3.4.4. Triển khai chạy nền và kiểm tra hoạt động

Sau khi hoàn thiện mã nguồn, nhóm tiến hành chạy thử nghiệm chương trình *mqtt_to_sql.py* trực tiếp trên Raspberry Pi để xác nhận khả năng kết nối đến Mosquitto và MariaDB. Lệnh thực thi được sử dụng: *python3 /home/pi/fireiot/mqtt_to_sql.py*

Khi chương trình khởi động thành công, terminal sẽ hiển thị các thông báo như:

```
MQTT LOCAL connected, rc = 0
Subscribed to: nha/#
Subscribed to: baochay/#
```

Ở giai đoạn này, có thể sử dụng *mosquitto_pub* hoặc dữ liệu từ chính các node ESP32 để kiểm tra quá trình nhận bản tin. Khi có bản tin mới, chương trình sẽ in log dạng: *Saved topic=nha/gara/nhietdo, payload=31.5*. Log này cho biết chương trình đã nhận được bản tin MQTT, xử lý payload và thực hiện ghi dữ liệu vào MariaDB.

Tuy nhiên, việc chạy chương trình một cách thủ công trên terminal chỉ phù hợp cho mục đích thử nghiệm, bởi tiến trình sẽ bị chấm dứt khi phiên terminal kết thúc hoặc Raspberry Pi khởi động lại. Để đảm bảo tính sẵn sàng và độ tin cậy trong vận hành thực tế, chương trình được đóng gói thành một dịch vụ systemd (systemd service). Tập được tạo tại đường dẫn: */etc/systemd/system/fireiot-mqttsql.service* với nội dung sau:

```
[Unit]
Description=Fire IoT MQTT to SQL sync from Local Mosquitto
Wants=network-online.target
After=network-online.target mariadb.service mosquitto.service

[Service]
ExecStart=/usr/bin/python3 -u /home/pi/fireiot/mqtt_to_sql.py
```

```
WorkingDirectory=/home/pi/fireiot
```

```
User=pi
```

```
Restart=always
```

```
RestartSec=10
```

```
[Install]
```

```
WantedBy=multi-user.target
```

Trong đó, dòng *After=network-online.target mariadb.service mosquito.service* bảo đảm chương trình trung gian chỉ chạy sau khi mạng, MariaDB và Mosquitto đã sẵn sàng. Dòng *ExecStart* chỉ định lệnh khởi chạy chương trình Python. Tham số *Restart=always* giúp service tự khởi động lại khi chương trình gặp lỗi hoặc bị dừng bất thường, nhờ đó tăng độ ổn định của hệ thống.

Sau khi tạo file service, nhóm nạp lại cấu hình systemd, kích hoạt service và kiểm tra trạng thái bằng các lệnh:

```
sudo systemctl daemon-reload
```

```
sudo systemctl enable fireiot-mqttsql.service
```

```
sudo systemctl start fireiot-mqttsql.service
```

```
sudo systemctl status fireiot-mqttsql.service --no-pager -l
```

Khi cần theo dõi log theo thời gian thực, nhóm sử dụng: *journalctl -u fireiot-mqttsql.service -f*

Khi service hoạt động bình thường, log sẽ hiển thị trạng thái kết nối MQTT và các bản tin được lưu vào cơ sở dữ liệu. Trường hợp xảy ra lỗi, log cũng giúp xác định nguyên nhân như sai mật khẩu MariaDB, broker chưa chạy, topic không đúng định dạng hoặc chương trình không có quyền truy cập thư mục làm việc.

Sau khi xác nhận service hoạt động ổn định, nhóm kiểm tra dữ liệu trong MariaDB bằng các truy vấn đã trình bày ở mục 3.3.4. Nếu bảng *mqtt_topic_history* xuất hiện các bản tin mới, bảng *room_state_current* cập nhật trạng thái từng phòng và bảng *devices* cập nhật trạng thái node, có thể kết luận chương trình trung gian đã hoạt động đúng theo thiết kế.

Nhìn chung, việc triển khai *mqtt_to_sql.py* dưới dạng *systemd service* giúp hệ thống có khả năng vận hành liên tục trên Raspberry Pi. Chương trình tự động

nhận dữ liệu từ Mosquitto local, ghi vào MariaDB và khởi động lại khi gặp lỗi, qua đó nâng cao độ tin cậy cho toàn bộ hạ tầng truyền thông và lưu trữ dữ liệu của hệ thống.

CHƯƠNG 4: THỬ NGHIỆM VÀ ĐÁNH GIÁ HIỆU NĂNG MẠNG

4.1. KỊCH BẢN KIỂM THỬ.

Sau khi hoàn thành thiết kế và triển khai hạ tầng truyền thông, cơ sở dữ liệu cùng chương trình trung gian ở chương 3, hệ thống cần được kiểm thử để xác nhận khả năng vận hành đúng theo kiến trúc đã đề xuất. Mục tiêu của phần kiểm thử không chỉ dừng ở việc kiểm tra từng thành phần riêng lẻ mà còn đánh giá sự phối hợp đồng bộ giữa các thành phần trong toàn bộ chuỗi truyền nhận dữ liệu.

Trong hệ thống triển khai thực tế, dữ liệu từ node ESP32 được gửi về Mosquitto local trên Raspberry Pi. Tại đây, dữ liệu được xử lý theo hai nhánh song song. Nhánh thứ nhất, chương trình *mqtt_to_sql.py* đăng ký nhận dữ liệu từ broker nội bộ và ghi vào MariaDB. Nhánh thứ hai, Mosquitto Bridge chuyển tiếp dữ liệu lên HiveMQ Cloud để các ứng dụng giám sát từ xa như Android hoặc MQTT Explorer có thể tiếp cận.

Vì vậy, kịch bản kiểm thử được xây dựng theo hướng kiểm tra từng thành phần và toàn bộ chuỗi xử lý, nhằm đảm bảo hệ thống hoạt động ổn định và dễ dàng xác định vị trí lỗi khi xảy ra sự cố.

4.1.1. Mục đích và phạm vi kiểm thử

Mục đích chính của hoạt động kiểm thử là xác nhận hệ thống có khả năng tiếp nhận, xử lý, lưu trữ và chuyển tiếp dữ liệu cảm biến đúng như kiến trúc đã thiết kế. Các kịch bản được xây dựng nhằm đánh giá lần lượt từng lớp chức năng: broker cục bộ, chương trình trung gian, cơ sở dữ liệu MariaDB, kết nối bridge lên HiveMQ Cloud và khả năng nhận dữ liệu từ các client giám sát.

Các nội dung kiểm thử chính tập trung vào việc đánh giá hoạt động của từng thành phần trong hệ thống cũng như sự phối hợp giữa chúng. Cụ thể, hệ thống được kiểm tra khả năng tiếp nhận và phân phối bản tin MQTT của Mosquitto Broker cục bộ trên Raspberry Pi trong mạng nội bộ. Đồng thời, chương trình trung gian *mqtt_to_sql.py* được đánh giá về khả năng đăng ký nhận dữ liệu từ broker và ghi chính xác vào cơ sở dữ liệu MariaDB.

Bên cạnh đó, cơ chế chuyển tiếp dữ liệu từ Raspberry Pi lên HiveMQ Cloud thông qua Mosquitto Bridge cũng được kiểm thử nhằm đảm bảo dữ liệu được đồng bộ ổn định lên nền tảng đám mây. Hệ thống client giám sát, bao gồm ứng dụng Android và MQTT Explorer, được sử dụng để kiểm tra khả năng nhận dữ liệu theo thời gian thực từ cloud broker. Cuối cùng, hệ thống được đánh giá

khả năng xử lý đồng thời nhiều topic hoặc nhiều node cùng gửi dữ liệu, nhằm xác định mức độ đáp ứng trong điều kiện hoạt động thực tế.

Phạm vi kiểm thử của mục 4.1 tập trung vào việc xác nhận hệ thống hoạt động đúng chức năng. Các chỉ tiêu định lượng như độ trễ truyền dữ liệu, tỷ lệ mất gói, độ ổn định và khả năng chịu tải sẽ được phân tích ở các mục sau. Nói cách khác, mục 4.1 đóng vai trò là bước kiểm tra nền tảng trước khi tiến hành đo đạc và đánh giá hiệu năng mạng.

4.1.2. Môi trường, điều kiện và nguyên tắc kiểm thử

Quá trình kiểm thử được thực hiện trong môi trường mạng nội bộ, trong đó Raspberry Pi 3B+ đóng vai trò máy chủ trung gian. Trên Raspberry Pi, nhóm triển khai Mosquitto Broker, MariaDB và chương trình `mqtt_to_sql.py`. Các node ESP32 hoặc terminal kiểm thử gửi dữ liệu MQTT về địa chỉ IP của Raspberry Pi trong cùng mạng Wi-Fi.

Môi trường kiểm thử bao gồm các thành phần chính được liệt kê trong bảng dưới đây:

Thành phần	Vai trò trong kiểm thử
ESP32 / terminal kiểm thử	Publish dữ liệu MQTT mô phỏng node cảm biến
Raspberry Pi 3B+	Máy chủ trung gian cục bộ
Mosquitto local	Broker MQTT nhận dữ liệu đầu tiên
<code>mqtt_to_sql.py</code>	Chương trình trung gian ghi dữ liệu vào MariaDB
MariaDB	Lưu dữ liệu lịch sử và trạng thái hệ thống
Mosquitto Bridge	Chuyển tiếp dữ liệu từ broker local lên HiveMQ Cloud
HiveMQ Cloud	Broker cloud phục vụ giám sát từ xa
Android / MQTT Explorer	Client subscribe dữ liệu thời gian thực

Trước khi tiến hành kiểm thử, tất cả các dịch vụ nền trên Raspberry Pi cần được kiểm tra trạng thái để đảm bảo hệ thống ở chế độ sẵn sàng:

```
sudo systemctl status mosquitto --no-pager -l
```

```
sudo systemctl status mariadb --no-pager -l
```

```
sudo systemctl status fireiot-mqttsql.service --no-pager -l
```

Nếu các dịch vụ trên đều hiển thị trạng thái *active (running)*, có thể tiến hành các kịch bản kiểm thử. Trường hợp một trong các dịch vụ chưa chạy, cần

xử lý trước khi kiểm thử để tránh nhầm lẫn giữa lỗi cấu hình và lỗi luồng dữ liệu.

Về nguyên tắc, các kịch bản được sắp xếp theo thứ tự từ đơn giản đến tổng hợp. Trước hết, nhóm kiểm tra broker nội bộ bằng các bản tin thử nghiệm cơ bản. Sau đó, lần lượt xác nhận khả năng ghi dữ liệu vào MariaDB, kiểm tra bridge lên HiveMQ Cloud và khả năng nhận dữ liệu từ MQTT Explorer hoặc ứng dụng Android. Cuối cùng, toàn bộ chuỗi xử lý được kiểm tra, bao gồm cả kịch bản mô phỏng nhiều topic hoặc nhiều node gửi dữ liệu đồng thời.

Cách tiếp cận theo từng lớp cho phép cô lập lỗi một cách hiệu quả. Ví dụ, nếu dữ liệu không xuất hiện trong MariaDB nhưng broker nội bộ vẫn nhận được bản tin, nguyên nhân có khả năng nằm ở chương trình *mqtt_to_sql.py* hoặc kết nối cơ sở dữ liệu. Nếu dữ liệu đã lưu thành công trong MariaDB nhưng không xuất hiện trên HiveMQ Cloud, vấn đề có thể liên quan đến cấu hình bridge. Nhờ lập luận đó, quá trình đánh giá hệ thống trở nên minh bạch và có cơ sở kỹ thuật.

4.1.3. Các kịch bản kiểm thử

a) Kiểm thử broker cục bộ trên Raspberry Pi

Kịch bản đầu tiên hướng đến việc kiểm tra khả năng hoạt động của Mosquitto local, thành phần đóng vai trò tiếp nhận dữ liệu đầu tiên từ các node ESP32. Tuy đây là phép kiểm thử cơ bản, nhưng mang ý nghĩa nền tảng quan trọng.

Trong kịch bản này, nhóm sử dụng hai phiên terminal trên cùng Raspberry Pi: một phiên để đăng ký nhận một topic thử nghiệm, phiên còn lại thực hiện phát (publish) dữ liệu vào topic đó.

```
mosquitto_sub -h localhost -p 1883 -t "test/topic" -v
```

```
mosquitto_pub -h localhost -p 1883 -t "test/topic" -m "hello mqtt"
```

Sau khi chạy hai lệnh này và kết quả xuất hiện kiểu *test/topic hello mqtt*, có thể kết luận Mosquitto local đã hoạt động đúng ở mức publish/subscribe cơ bản. Kịch bản này giúp xác nhận Raspberry Pi đã sẵn sàng đóng vai trò broker cục bộ trước khi chuyển sang kiểm thử các tầng xử lý phía sau.

b) Kiểm thử ghi dữ liệu vào MariaDB

Sau khi broker local được xác nhận hoạt động, nhóm tiến hành kiểm thử khả năng ghi dữ liệu vào MariaDB. Trong kịch bản này, dữ liệu thử nghiệm

được publish vào các topic có cấu trúc giống dữ liệu thật của hệ thống. Ví dụ, nhóm mô phỏng dữ liệu từ khu vực gara bằng chuỗi lệnh sau:

```
mosquitto_pub -h localhost -p 1883 -t "nha/gara/nhietdo" -m "31.5" -r
mosquitto_pub -h localhost -p 1883 -t "nha/gara/doam" -m "74.2" -r
mosquitto_pub -h localhost -p 1883 -t "nha/gara/gas" -m "35" -r
mosquitto_pub -h localhost -p 1883 -t "nha/gara/khoi" -m "12" -r
mosquitto_pub -h localhost -p 1883 -t "nha/gara/Lua" -m "0" -r
mosquitto_pub -h localhost -p 1883 -t "nha/gara/canhbao_trangthai" -m
"binh_thuong" -r
mosquitto_pub -h localhost -p 1883 -t "nha/gara/canhbao_muc" -m "0" -r
mosquitto_pub -h localhost -p 1883 -t "nha/gara/trangthai" -m "online" -r
```

Sau khi gửi dữ liệu, kiểm tra log của chương trình trung gian: `sudo journalctl -u fireiot-mqttsql.service -f`. Kết quả mong đợi là log hiển thị các dòng tương tự:

```
Saved topic=nha/gara/nhietdo, payload=31.5
Saved topic=nha/gara/doam, payload=74.2
```

Tiếp theo, truy vấn MariaDB để kiểm tra dữ liệu đã được ghi vào các bảng hay chưa:

```
SELECT * FROM mqtt_topic_history ORDER BY received_at DESC LIMIT 20;
SELECT * FROM room_state_current;
SELECT * FROM sensor_readings ORDER BY recorded_at DESC LIMIT 20;
```

Kết quả mong đợi là bảng `mqtt_topic_history` xuất hiện các bản tin vừa gửi, bảng `room_state_current` có dòng tương ứng với phòng gara, và bảng `sensor_readings` có bản ghi tổng hợp nếu dữ liệu cảnh báo hoặc mức cảnh báo đủ điều kiện tạo snapshot.

Kịch bản này có ý nghĩa quan trọng vì nó xác nhận dữ liệu từ lớp MQTT đã được chuyển thành dữ liệu có cấu trúc trong cơ sở dữ liệu. Đây là cơ sở để phục vụ truy vấn lịch sử, thống kê và đánh giá ở các phần sau.

c) Kiểm thử bridge dữ liệu lên HiveMQ Cloud

Sau khi xác nhận dữ liệu được lưu trữ cục bộ, nhóm tập trung kiểm tra khả năng chuyển tiếp dữ liệu từ Raspberry Pi lên HiveMQ Cloud thông qua Mosquitto Bridge. Mục tiêu của kịch bản này là khẳng định rằng những gì được

phát vào broker nội bộ có thể xuất hiện trên cloud broker, sẵn sàng cho các ứng dụng giám sát từ xa. Trước hết, trạng thái kết nối bridge được kiểm tra bằng lệnh: `mosquitto_sub -h localhost -p 1883 -t '$SYS/broker/connection/#' -v`

Nếu kết quả trả về có chứa thông tin như: `$SYS/broker/connection/pi3b-bridge-01/state 1` thì kết nối bridge giữa Raspberry Pi và HiveMQ Cloud đang ở trạng thái hoạt động. Sau đó, nhóm mở MQTT Explorer hoặc Web client kết nối đến HiveMQ Cloud và đăng ký nhận các topic: `nha/#`, `baochay/#`.

Tiếp theo, một vài bản tin được phát vào broker nội bộ. Kết quả mong đợi là trên MQTT Explorer hoặc Web client, các topic thuộc các nhánh trên sẽ xuất hiện với payload tương ứng. Kịch bản này xác nhận hệ thống không chỉ dừng lại ở việc lưu trữ dữ liệu cục bộ mà còn mở rộng khả năng giám sát từ bên ngoài thông qua kết nối Internet.

d) Kiểm thử toàn bộ chuỗi xử lý dữ liệu

Sau khi từng lớp đã được kiểm tra riêng lẻ, nhóm tiến hành kiểm thử toàn bộ chuỗi xử lý dữ liệu. Đây là kịch bản quan trọng nhất trong mục 4.1 vì nó đánh giá sự phối hợp đồng thời giữa node cảm biến, broker local, chương trình trung gian, cơ sở dữ liệu, bridge cloud và client giám sát.

Trong kịch bản này, dữ liệu được gửi từ ESP32 hoặc terminal kiểm thử về Mosquitto local trên Raspberry Pi. Sau khi broker tiếp nhận bản tin, chương trình `mqtto_sql.py` tự động xử lý và ghi dữ liệu vào MariaDB. Đồng thời, Mosquitto Bridge chuyển tiếp dữ liệu lên HiveMQ Cloud để Android hoặc MQTT Explorer có thể nhận dữ liệu thời gian thực.

Kết quả mong đợi của kịch bản kiểm thử bao gồm: Mosquitto local tiếp nhận đúng topic và payload; chương trình `mqtto_sql.py` xử lý và ghi log tương ứng; dữ liệu được lưu thành công trong MariaDB; đồng thời bản tin được chuyển tiếp lên HiveMQ Cloud và hiển thị chính xác trên các client giám sát.

Nếu toàn bộ những điều kiện trên đều được thỏa mãn, có thể khẳng định chuỗi xử lý dữ liệu của hệ thống vận hành đúng theo kiến trúc đã thiết kế. Đây là tiền đề cần thiết trước khi chuyển sang các nội dung đo độ trễ truyền dữ liệu (mục 4.2) và đánh giá độ ổn định của hệ thống (mục 4.3).

e) Kiểm thử nhiều topic hoặc nhiều node đồng thời

Nhằm mô phỏng gần hơn với điều kiện vận hành thực tế, nhóm xây dựng kịch bản kiểm thử với nhiều topic, tương ứng với nhiều khu vực cùng phát sinh dữ liệu trong một khoảng thời gian ngắn. Trong bối cảnh hệ thống cảnh báo

cháy, điều này tương đương tình huống các phòng như bếp, phòng khách và gara đồng loạt gửi dữ liệu về broker trung tâm. Ví dụ, có thể sử dụng chuỗi lệnh mô phỏng như sau:

```
mosquitto_pub -h localhost -p 1883 -t "nha/bep/nhietdo" -m "34.2"
```

```
mosquitto_pub -h localhost -p 1883 -t "nha/phongkhach/nhietdo" -m "31.0"
```

```
mosquitto_pub -h localhost -p 1883 -t "nha/gara/nhietdo" -m "38.5"
```

```
mosquitto_pub -h localhost -p 1883 -t "nha/bep/gas" -m "420"
```

```
mosquitto_pub -h localhost -p 1883 -t "nha/phongkhach/khoi" -m "95"
```

```
mosquitto_pub -h localhost -p 1883 -t "nha/gara/khoi" -m "890"
```

Kết quả mong đợi là tất cả các bản tin vẫn được tiếp nhận đầy đủ. Trong MariaDB, dữ liệu được ghi nhận chính xác theo từng topic và từng phòng. Đồng thời, các bản tin tương ứng vẫn được chuyển tiếp thành công lên HiveMQ Cloud.

Kịch bản này cung cấp đánh giá sơ bộ về khả năng xử lý nhiều bản tin liên tiếp của broker nội bộ và chương trình trung gian. Tuy nhiên, trong phạm vi mục 4.1, nội dung chỉ dừng ở mức kiểm thử chức năng. Các phân tích chuyên sâu về khả năng chịu tải, số lượng bản tin kỳ vọng, số bản tin thực tế được lưu và tỷ lệ mất gói sẽ được trình bày trong mục 4.3.

4.1.4. Tổng hợp kịch bản và tiêu chí đánh giá

Để quá trình kiểm thử rõ ràng và thuận tiện theo dõi, nhóm tổng hợp các kịch bản kiểm thử chính trong Bảng 4.1.

Bảng 4.1: Tổng hợp các kịch bản kiểm thử hệ thống

STT	Kịch bản kiểm thử	Mục đích	Kết quả mong đợi
1	Kiểm thử broker local	Xác nhận Mosquitto local nhận và phân phối bản tin đúng	Subscriber nhận đúng bản tin được publish
2	Kiểm thử lưu dữ liệu vào MariaDB	Xác nhận mqtt_to_sql.py ghi dữ liệu đúng vào CSDL	Bản ghi xuất hiện trong các bảng lịch sử, latest, room state, devices

3	Kiểm thử bridge lên HiveMQ Cloud	Xác nhận dữ liệu từ broker local được chuyển tiếp ra cloud	Topic và payload xuất hiện trên cloud broker
4	Kiểm thử toàn chuỗi xử lý	Xác nhận toàn bộ luồng dữ liệu hoạt động đồng bộ	Dữ liệu được nhận, lưu cục bộ và hiển thị từ xa
5	Kiểm thử nhiều topic đồng thời	Mô phỏng điều kiện vận hành gần thực tế	Hệ thống vẫn xử lý đúng và không mất bản tin rõ rệt

Thông qua các kịch bản trên, hệ thống được kiểm tra từ mức thành phần đến mức toàn chuỗi. Kết quả của mục 4.1 là cơ sở để khẳng định hệ thống đã đủ điều kiện bước sang các phần đánh giá định lượng như đo độ trễ truyền dữ liệu, kiểm tra khả năng chịu tải và phân tích độ ổn định của broker ở các mục tiếp theo.

4.2. ĐO ĐẠC ĐỘ TRỄ TỪ LÚC CẢM BIẾN PHÁT HIỆN ĐẾN KHI SERVER NHẬN ĐƯỢC.

4.2.1. Mục đích và phương pháp đo đạc

Trong một hệ thống cảnh báo cháy dựa trên IoT, độ trễ truyền dữ liệu là yếu tố sống còn quyết định tính thời gian thực. Khi một node cảm biến phát hiện khói, gas, lửa hay nhiệt độ vượt ngưỡng, thông tin cần đến được máy chủ giám sát sau một khoảng thời gian đủ ngắn để hệ thống kịp kích hoạt cảnh báo, giúp người quản lý phản ứng ngay. Song song với việc cảnh báo tức thời, dữ liệu còn cần được lưu trữ đầy đủ vào cơ sở dữ liệu cục bộ để phục vụ truy vết sự cố, và đồng thời phải được chuyển tiếp lên nền tảng đám mây để người dùng ở xa có thể theo dõi. Bởi vậy, chúng em cần đo đạc thật chính xác xem dữ liệu mất bao lâu để đi qua từng chặng trong tuyến truyền, từ đó đánh giá xem hạ tầng đã xây dựng có thực sự đáp ứng được yêu cầu “gần thời gian thực” hay không.

Để có được số liệu tin cậy và chi tiết đến từng mili giây, chúng em sử dụng phương pháp gán nhãn thời gian (timestamp) có đồng bộ qua giao thức NTP (Network Time Protocol). Trên mỗi node ESP32, đồng hồ được đồng bộ NTP lúc khởi động và cập nhật định kỳ; ngay trước khi publish một bản tin MQTT, chương trình nhúng sẽ đọc thời gian hiện tại (độ phân giải mili giây) và chèn vào payload dưới dạng trường "ts": <giá trị ms>. Như vậy, mỗi gói tin đều

mang theo đúng thời điểm nó được tạo ra ở phía node cảm biến. Phía máy chủ Raspberry Pi, đồng hồ hệ thống cũng được đồng bộ NTP liên tục. Tại mỗi điểm quan sát trên tuyến truyền, chúng em ghi lại thời gian mà bản tin đến đích thông qua các script Python chạy kèm dịch vụ.

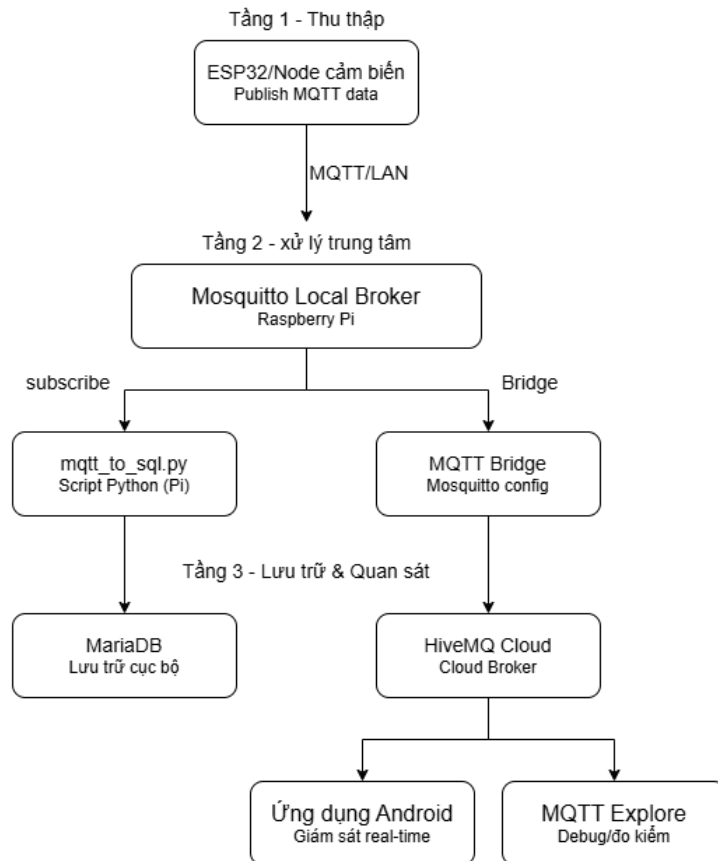
Bốn mốc thời gian chính được xác định để phục vụ việc tính toán: T1 – thời điểm ESP32 tạo bản tin (lấy từ payload); T2 – thời điểm Mosquitto broker cục bộ nhận được bản tin và phân phối cho subscriber (ghi nhận trong một script Python độc lập); T3 – thời điểm dòng log *Saved topic=...* xuất hiện, báo hiệu chương trình *mqtt_to_sql.py* đã ghi xong vào MariaDB; T4 – thời điểm dữ liệu hiển thị trên MQTT Explorer kết nối đến HiveMQ Cloud. Từ các mốc này, ba chặng trễ được định nghĩa: $\Delta_1 = T2 - T1$ (truyền nội mạng từ ESP32 đến broker cục bộ), $\Delta_2 = T3 - T2$ (xử lý và ghi cơ sở dữ liệu), $\Delta_3 = T4 - T2$ (chuyển tiếp lên cloud).

Để có đủ mẫu đánh giá xu hướng và tính ổn định, mỗi node (bếp, phòng khách, gara) được thực hiện 4 lần đo riêng biệt cho từng đại lượng khói, gas, lửa, nhiệt độ. Tổng cộng có 48 phép đo cho mỗi chặng. Các bảng số liệu thô của toàn bộ 48 phép đo này được chúng em tổng hợp và đặt trong Phụ lục của báo cáo, nhằm giúp mạch văn trong chương chính tập trung vào việc phân tích và đánh giá.

4.2.2. Mô hình tuyến truyền và các chặng đo độ trễ

Quá trình đo độ trễ được thực hiện dựa trên mô hình truyền dữ liệu thực tế của hệ thống. Các node ESP32 ở ba khu vực bếp, phòng khách và gara liên tục thu thập thông số môi trường rồi gửi về Mosquitto broker cục bộ trên Raspberry Pi qua mạng Wi-Fi. Khi đến được máy chủ, dữ liệu đi theo hai nhánh song song. Nhánh thứ nhất là chương trình *mqtt_to_sql.py* (chạy dưới dạng service nền) subscribe các topic, phân tích và ghi vào cơ sở dữ liệu MariaDB. Nhánh thứ hai là Mosquitto Bridge duy trì một kết nối TLS lên HiveMQ Cloud và chuyển tiếp toàn bộ bản tin thuộc hai không gian *nha/#* và *baochay/#* lên cloud broker để phục vụ giám sát từ xa.

Mô hình tuyến truyền dữ liệu được sử dụng trong quá trình đo như sau:



Hình 4.1: Mô hình truyền dữ liệu từ node cảm biến đến server cục bộ và nền tảng cloud

Điều đáng chú ý là hai nhánh MariaDB và HiveMQ Cloud hoàn toàn độc lập, không nối tiếp nhau. Vì thế, khi tính tổng độ trễ từ ESP32 đến cloud, chúng ta chỉ cộng thời gian qua chặng 1 (ESP32 → Mosquitto local) và chặng 3 (Mosquitto local → Cloud), mà không cộng thêm chặng 2. Chặng 2 được đánh giá riêng để xem tác vụ ghi dữ liệu cục bộ có đủ nhanh, có làm nghẽn hệ thống hay không.

Nhằm định vị rõ từng phần trên đường truyền, nhóm chia toàn tuyến thành ba chặng đo chính.

a) Chặng 1: Từ ESP32 đến Mosquitto local trên Raspberry Pi

Đây là chặng đầu tiên, phản ánh thời gian cần thiết để một bản tin MQTT được truyền từ node cảm biến (ESP32) qua mạng Wi-Fi nội bộ đến broker Mosquitto đang chạy ngay trên Raspberry Pi. Nếu chặng này có độ trễ lớn hoặc dao động mạnh, rất có thể nguyên nhân nằm ở chất lượng kết nối không dây, vị trí đặt node hoặc nhiễu tín hiệu. Đây cũng là chặng mà nhóm có thể chủ động cải thiện bằng cách bố trí lại điểm truy cập hoặc lựa chọn kênh Wi-Fi ít nhiễu hơn.

Cách đo: Như đã mô tả, tại ESP32, ngay trước khi gọi hàm *publish*, chương trình đọc *millis()* (đã đồng bộ NTP) và chèn vào payload. Phía Raspberry Pi, script Python subscribe tất cả topic *nha/#* và ghi lại thời điểm callback *on_message* được thực thi (T2). Độ trễ chặng 1 được tính bằng công thức $\Delta_1 = T2 - T1$, trong đó T1 được trích xuất từ trường "ts" trong payload. Mỗi node và mỗi loại thông số được thực hiện đo 4 lần để đảm bảo độ tin cậy.

b) Chặng 2: Từ Mosquitto local đến mqtt_to_sql.py và MariaDB

Chặng này cho biết chương trình trung gian *mqtt_to_sql.py* mất bao lâu để bắt một bản tin từ broker, phân tích topic, chuẩn hóa payload, và ghi thành công vào cơ sở dữ liệu MariaDB. Tuy không ảnh hưởng đến thời gian xuất hiện dữ liệu trên cloud, nhưng nếu chặng này quá chậm sẽ làm chậm trễ việc cập nhật trạng thái nội bộ, gây khó khăn cho các thao tác truy vấn hay hiển thị dashboard tại chỗ. Ngược lại, nếu chặng này luôn ổn định và thấp, ta có thể yên tâm rằng kiến trúc lưu trữ cục bộ trên Raspberry Pi đáp ứng tốt.

Cách đo: Chúng em dựa vào hai mốc T2 (broker local phân phối bản tin) và T3 (dòng log *Saved topic=...* xuất hiện trong *journalctl -u fireiot-mqttsql.service*). Cả hai đều ghi nhận bằng đồng hồ hệ thống của Raspberry Pi (đã được đồng bộ NTP). Độ trễ chặng 2 được tính là $\Delta_2 = T3 - T2$. Để có số liệu chính xác, script subscribe dùng để lấy T2 được thiết lập sao cho không làm ảnh hưởng đến hiệu năng của *mqtt_to_sql.py*. Các timestamp T3 được trích xuất tự động từ log, giúp loại bỏ thao tác bấm giờ thủ công.

c) Chặng 3: Từ Mosquitto local đến HiveMQ Cloud

Đây là chặng quyết định dữ liệu có đến được tay người giám sát từ xa nhanh hay không. Nó bao gồm thời gian xử lý của Mosquitto Bridge, thời gian truyền qua Internet (có thể qua nhiều hop mạng), và thời gian mà HiveMQ Cloud nhận được rồi phân phối lại cho các client. Chặng này phụ thuộc nhiều vào chất lượng đường truyền Internet, vị trí địa lý của cloud broker và cơ chế bảo mật TLS. Bằng cách tách riêng chặng 3, chúng em có thể đánh giá xem hạ tầng cloud đang dùng có phù hợp hay không, và nếu cần thì có thể cân nhắc chuyển sang một cloud broker gần hơn về mặt địa lý để giảm trễ.

Cách đo: Chúng em sử dụng một máy trạm (laptop) cũng được đồng bộ NTP chung với Raspberry Pi. Trên máy trạm, MQTT Explorer được cấu hình kết nối đến HiveMQ Cloud và subscribe các topic tương ứng. Một script Python nhỏ sẽ lắng nghe và ghi lại thời điểm mỗi bản tin xuất hiện (T4). Do cả T2 và

T4 đều nằm trên cùng một miền thời gian đồng bộ, độ trễ chặng 3 được tính đơn giản là $\Delta_3 = T4 - T2$. Việc đo được thực hiện đồng thời với hai chặng trên, nên mỗi lần phát một bản tin từ ESP32, chúng em có ngay một bộ ba giá trị (Δ_1 , Δ_2 , Δ_3).

4.2.3. Kết quả đo độ trễ theo từng node

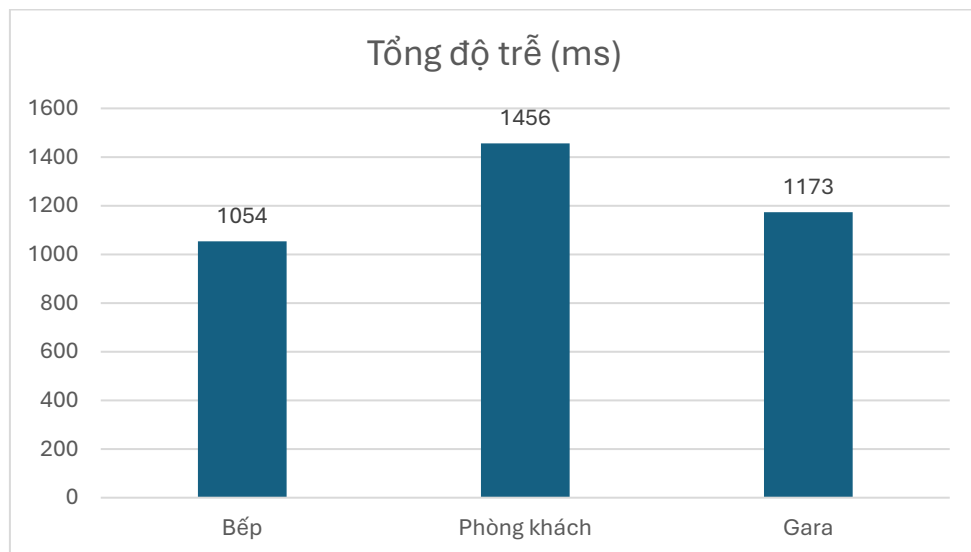
Sau khi xử lý số liệu đo (chi tiết trong Phụ lục), chúng em tính được các giá trị độ trễ trung bình của từng node trên từng chặng. Tổng độ trễ từ ESP32 đến HiveMQ Cloud được tính bằng trung bình Δ_1 + trung bình Δ_3 , vì hai chặng này nối tiếp nhau, còn chặng MariaDB là nhánh song song không cộng vào. Kết quả tóm tắt được trình bày trong **Bảng 4.6**.

Bảng 4.3: Tổng hợp độ trễ trung bình theo từng chặng

Node	ESP32 → Mosquitto	Local → MariaDB	Local → HiveMQ Cloud	Tổng ESP32 → Cloud
Bếp	605	120	449	1054
Phòng khách	782	136	674	1456
Gara	665	121	508	1173
Trung bình	684	126	544	1228

Để thấy rõ hơn bức tranh về hiệu năng hệ thống, nhóm sử dụng các biểu đồ trực quan.

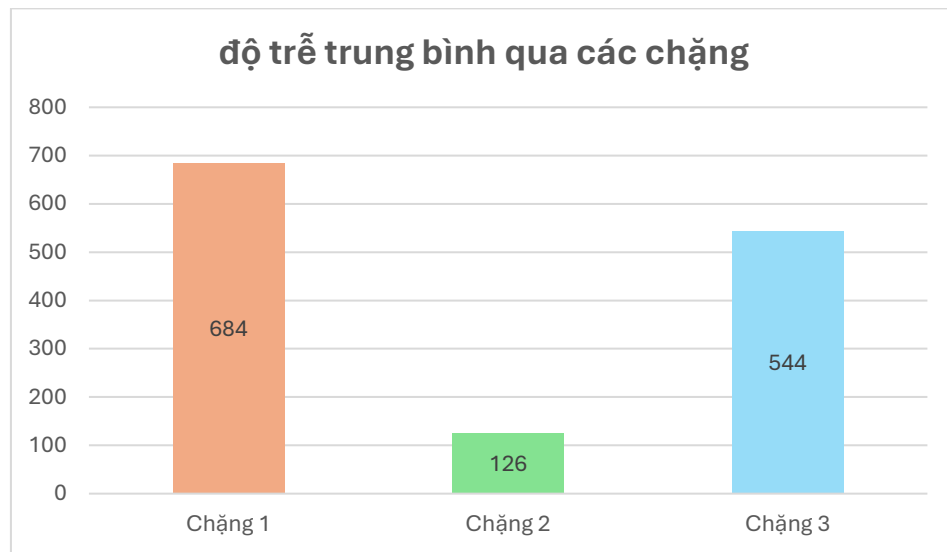
❖ So sánh tổng độ trễ đến Cloud giữa các node



Hình 4.2: Tổng độ trễ trung bình từ Node đến HiveMQ Cloud

Có thể thấy, node bếp đạt thời gian ngắn nhất (khoảng 1,05 giây), node gara ở mức trung bình (1,17 giây) và node phòng khách có độ trễ lớn nhất (1,46 giây). Sự chênh lệch này chủ yếu bắt nguồn từ chặng truyền Wi-Fi nội bộ, nơi node phòng khách đặt ở vị trí khuất sóng hơn. Dù vậy, với giá trị tuyệt đối đều dưới 1,5 giây, hệ thống hoàn toàn đáp ứng được yêu cầu phản hồi nhanh của một ứng dụng báo cháy. Người giám sát từ xa gần như có thể nắm bắt diễn biến môi trường trong khoảng hơn một giây, đủ sớm để triển khai các biện pháp ứng phó kịp thời.

❖ **Phân bố độ trễ qua các chặng xử lý**



Hình 4.3: Phân bố độ trễ trung bình qua các chặng xử lý

Nhìn vào biểu đồ, có thể thấy chặng ghi cơ sở dữ liệu (Δ_2) chỉ chiếm một phần rất nhỏ, trung bình 126 ms, gần như không đáng kể. Điều này khẳng định giải pháp dùng *mqtt_to_sql.py* và MariaDB ngay trên Raspberry Pi là hoàn toàn khả thi, không hề gây nghẽn cổ chai. Chặng truyền lên HiveMQ Cloud (Δ_3) đóng góp nhiều nhất với trung bình 544 ms, phản ánh đúng thực tế dữ liệu phải vượt qua Internet và cơ chế bảo mật TLS. Chặng Wi-Fi nội bộ (Δ_1) chiếm vị trí trung gian với 684 ms, và đây cũng là chặng có tiềm năng cải thiện nhiều nhất nếu tối ưu được mạng không dây.

❖ **Đánh giá tính ổn định của độ trễ**

Bên cạnh giá trị trung bình, tính ổn định của độ trễ qua các lần đo là yếu tố then chốt với một hệ thống báo cháy: dữ liệu không thể lúc nhanh lúc chậm thất thường. Để kiểm tra điều này, chúng em trích toàn bộ giá trị Δ_1 (chặng chịu ảnh hưởng trực tiếp từ môi trường không dây) từ các bảng số liệu trong Phụ lục và biểu diễn dưới dạng biểu đồ đường (Hình 4.9). Mỗi đường tương ứng với

một node, đi qua 4 lần đo (lấy trung bình của 4 thông số trong mỗi lần để quan sát xu thế chung).

Quan sát đồ thị, đường biểu diễn của node bếp và node gara gần như đi ngang, biên độ dao động giữa lần đo thấp nhất và cao nhất không quá 35 ms. Điều này cho thấy kết nối Wi-Fi ở hai khu vực này rất ổn định, tốc độ truyền hầu như không đổi. Đường biểu diễn của node phòng khách có biên độ dao động rộng hơn (khoảng 50 ms), phản ánh tín hiệu kém ổn định hơn, nhưng vẫn không xuất hiện đột biến lớn hay tình trạng tăng vọt bất thường. Tất cả các mẫu đều duy trì trong một vùng giá trị hẹp, không xảy ra mất gói hay tắc nghẽn tạm thời. Như vậy, hệ thống không chỉ có độ trễ thấp mà còn rất ổn định, một đặc tính quan trọng để đảm bảo độ tin cậy của giải pháp cảnh báo.

4.2.4. Nhận xét chung

Từ những kết quả đo đạc và phân tích ở trên, chúng em rút ra một số nhận định quan trọng. Thứ nhất, tổng độ trễ từ cảm biến đến cloud trung bình đạt 1,23 giây, đáp ứng tốt mục tiêu phản hồi nhanh của một hệ thống báo cháy. Thứ hai, chẳng ghi cơ sở dữ liệu cục bộ hầu như không làm ảnh hưởng đến tốc độ toàn tuyến, cho phép hệ thống vừa cảnh báo kịp thời vừa lưu trữ đầy đủ thông tin để truy vết. Thứ ba, dù chẳng truyền lên cloud chiếm thời gian lớn nhất, nhưng bản chất là phụ thuộc vào hạ tầng Internet và có thể cải thiện bằng cách chọn cloud broker gần hơn về mặt địa lý. Cuối cùng, tính ổn định của độ trễ qua nhiều phép thử cho thấy hạ tầng Wi-Fi và các dịch vụ nội bộ trên Raspberry Pi có thể vận hành liên tục mà không bị suy giảm chất lượng. Những kết quả này khẳng định rằng hạ tầng truyền thông mà nhóm đã xây dựng là hoàn toàn phù hợp với yêu cầu giám sát và cảnh báo cháy IoT trong mô hình quy mô nhỏ.

4.3. ĐÁNH GIÁ ĐỘ ỔN ĐỊNH VÀ KHẢ NĂNG CHỊU TẢI CỦA BROKER.

4.3.1. Mục đích và kịch bản kiểm thử tải

Sau khi đánh giá độ trễ trên từng tuyến truyền đơn lẻ (mục 4.2), một câu hỏi quan trọng được đặt ra là liệu hạ tầng truyền thông – đặc biệt là Mosquitto broker cục bộ và chương trình trung gian *mqtt_to_sql.py* – có duy trì được sự ổn định khi phải tiếp nhận đồng thời một lượng lớn bản tin từ nhiều nút cảm biến hay không. Trong thực tế triển khai, một hệ thống cảnh báo cháy IoT thường phải vận hành với nhiều khu vực giám sát cùng lúc, mỗi khu vực liên tục gửi dữ liệu về broker. Vì vậy, kiểm thử khả năng chịu tải là bước cần thiết để xác minh tính sẵn sàng của mô hình trước các kịch bản hoạt động cao điểm.

Mục tiêu của phép kiểm thử này là đánh giá mức độ ổn định của Mosquitto local trên Raspberry Pi, khả năng xử lý của script *mqtt_to_sql.py* và độ tin cậy ghi dữ liệu vào MariaDB khi đồng thời có nhiều node gửi dữ liệu liên tục. Các chỉ tiêu chính được quan tâm bao gồm: tỷ lệ thất thoát bản tin, tính liên tục của dòng dữ liệu ghi vào cơ sở dữ liệu và trạng thái hoạt động của các dịch vụ thành phần trong suốt quá trình chịu tải.

Kịch bản kiểm thử được thiết lập dựa trên mô hình triển khai thực tế: ba nút cảm biến đại diện cho ba khu vực bếp, phòng khách và gara đồng loạt publish dữ liệu về Mosquitto broker cục bộ. Mỗi nút gửi năm tham số giám sát chính gồm nhiệt độ (*nhietdo*), nồng độ khí gas (*gas*), nồng độ khói (*khoi*), trạng thái ngọn lửa (*lua*) và mức cảnh báo tổng hợp (*canhbao_muc*). Các topic tương ứng được tổ chức theo cấu trúc phân cấp *nha/<khuvuc>/<loai_cam_bien>* như bảng dưới đây:

Bảng 4.4: Cấu trúc topic trong kịch bản kiểm thử tải

Khu vực	Các topic cảm biến
Bếp	<i>nha/bep/nhietdo</i> , <i>nha/bep/gas</i> , <i>nha/bep/khoi</i> , <i>nha/bep/lua</i> , <i>nha/bep/canhbao_muc</i>
Phòng khách	<i>nha/phongkhach/nhietdo</i> , <i>nha/phongkhach/gas</i> , <i>nha/phongkhach/khoi</i> , <i>nha/phongkhach/lua</i> , <i>nha/phongkhach/canhbao_muc</i>
Gara	<i>nha/gara/nhietdo</i> , <i>nha/gara/gas</i> , <i>nha/gara/khoi</i> , <i>nha/gara/lua</i> , <i>nha/gara/canhbao_muc</i>

Như vậy, trong mỗi chu kỳ gửi dữ liệu, hệ thống tiếp nhận tổng cộng 15 bản tin MQTT, tương ứng với 3 node và 5 topic trên mỗi node. Thời gian kiểm thử được thực hiện trong 60 giây. Kịch bản này tạo ra tải lớn hơn so với điều kiện vận hành thông thường, từ đó giúp đánh giá rõ hơn khả năng chịu tải của broker và phần mềm trung gian.

Bảng 4.5: Thông số kịch bản kiểm thử tải broker

Thông số kiểm thử	Giá trị
Số node kiểm thử	3
Khu vực kiểm thử	Bếp, phòng khách, gara
Số topic mỗi node	5
Tổng số topic mỗi chu kỳ	15
Các thông số gửi lên	Nhiệt độ, gas, khói, lửa, mức cảnh báo

Thời gian kiểm thử	60s
Số chu kỳ ước tính	203

4.3.2. Phương pháp đánh giá mất mát gói tin và tài nguyên hệ thống

Để đánh giá hiện tượng mất mát bản tin, nhóm tiến hành đối chiếu giữa số lượng bản tin lý thuyết được gửi đi (suy ra từ kịch bản) và số lượng bản tin thực tế mà hệ thống đã ghi nhận thành công trong cơ sở dữ liệu. Hai bảng dữ liệu chính được sử dụng cho mục đích này là *mqtt_topic_history* và *sensor_readings* trong MariaDB.

Bảng *mqtt_topic_history* có chức năng lưu vết toàn bộ các bản tin MQTT mà chương trình *mqtt_to_sql.py* đã subscribe và xử lý. Mỗi bản tin khi được lưu thành công sẽ tạo ra một dòng dữ liệu, vì thế đây là căn cứ trực tiếp nhất để kiểm tra số lượng bản tin đã đến được đích.

Bảng *sensor_readings* lưu trữ các bản ghi tổng hợp hoặc các snapshot dữ liệu cảm biến theo từng thời điểm, không nhất thiết phản ánh từng bản tin riêng lẻ mà phụ thuộc vào logic xử lý bên trong *mqtt_to_sql.py*. Do đó, số dòng trong *sensor_readings* không kỳ vọng bằng tổng số bản tin MQTT, nhưng cần tương ứng với số lần dữ liệu cảnh báo thực sự được tổng hợp.

Tỷ lệ mất gói được tính theo công thức:

$$PLR(\%) = \frac{N_e - N_r}{N_e} \times 100\%$$

Trong đó:

- N_e : số bản tin kỳ vọng (*expected number of messages*), tức tổng số bản tin dự kiến được gửi trong quá trình kiểm thử.
- N_r : số bản tin thực lưu (*received/recorded number of messages*), tức số bản tin thực tế được ghi nhận trong bảng *mqtt_topic_history* của MariaDB.

Bên cạnh phép đối chiếu định lượng, nhóm còn giám sát trạng thái hoạt động của dịch vụ *fireiot-mqttsql.service* thông qua nhật ký hệ thống. Việc liên tục xuất hiện các dòng “Saved topic=...” với đầy đủ topic từ ba khu vực là chỉ báo quan trọng cho thấy chương trình trung gian không bị gián đoạn hay gặp lỗi trong quá trình tải.

Về tài nguyên hệ thống, Raspberry Pi 3B+ (4 nhân 1,4 GHz, RAM 1 GB) đảm nhiệm đồng thời vai trò Mosquitto broker, môi trường chạy *mqtt_to_sql.py* và máy chủ MariaDB. Trong điều kiện lý tưởng, các thông

số như mức sử dụng CPU và RAM cần được ghi nhận liên tục trong suốt quá trình kiểm thử để có đánh giá định lượng về mức độ chiếm dụng tài nguyên. Tuy nhiên, do giới hạn về công cụ giám sát tại thời điểm thực nghiệm, nghiên cứu hiện tại giới hạn ở việc quan sát định tính: hệ thống được xem là ổn định nếu broker không ghi nhận tình trạng ngắt kết nối, dịch vụ *fireiot-mqttsql.service* không dừng đột ngột, và MariaDB vẫn tiếp tục ghi dữ liệu bình thường trong suốt 60 giây kiểm thử.

4.3.3. Kết quả kiểm thử khả năng chịu tải của broker

Trong thời gian kiểm thử 60 giây, hệ thống thực hiện khoảng 203 chu kỳ gửi dữ liệu. Với 15 bản tin trong mỗi chu kỳ, tổng số bản tin kỳ vọng được xác định như sau: $N_e = 203 \times 15 = 3045$ bản tin

Kết quả truy vấn trực tiếp trên bảng *mqtt_topic_history* cho thấy số bản tin thực tế được lưu là 3044 bản tin. Như vậy, lượng bản tin chênh lệch so với lý thuyết chỉ là 1 bản tin. Áp dụng công thức đã nêu, tỷ lệ mất gói thu được là:

$$PLR = \frac{3045 - 3044}{3045} \times 100 \approx 0,033\%$$

Kết quả này cho thấy tỷ lệ mất gói rất thấp trong điều kiện kiểm thử. Với hơn 3000 bản tin được gửi liên tục trong 60 giây, việc chỉ chênh lệch 1 bản tin cho thấy Mosquitto Broker và chương trình trung gian *mqtt_to_sql.py* vẫn duy trì được khả năng tiếp nhận và lưu dữ liệu tương đối ổn định.

Bảng 4.6: Kết quả đối chiếu số lượng bản tin trong MariaDB

Nội dung đánh giá	Giá trị
Thời gian kiểm thử	60s
Số node kiểm thử	3
Số topic mỗi node	5
Số chu kỳ gửi ước tính	203
Tổng số bản ghi kỳ vọng	3045
Số bản tin lưu trong <i>mqtt_topic_history</i>	3044
Số bản ghi chênh lệch	1
Tỷ lệ mất gói	$\approx 0,033\%$
Số bản ghi trong <i>sensor_readings</i>	608

Khi xét theo từng node, số lượng bản tin được lưu trong *mqtt_topic_history* được tổng hợp như sau:

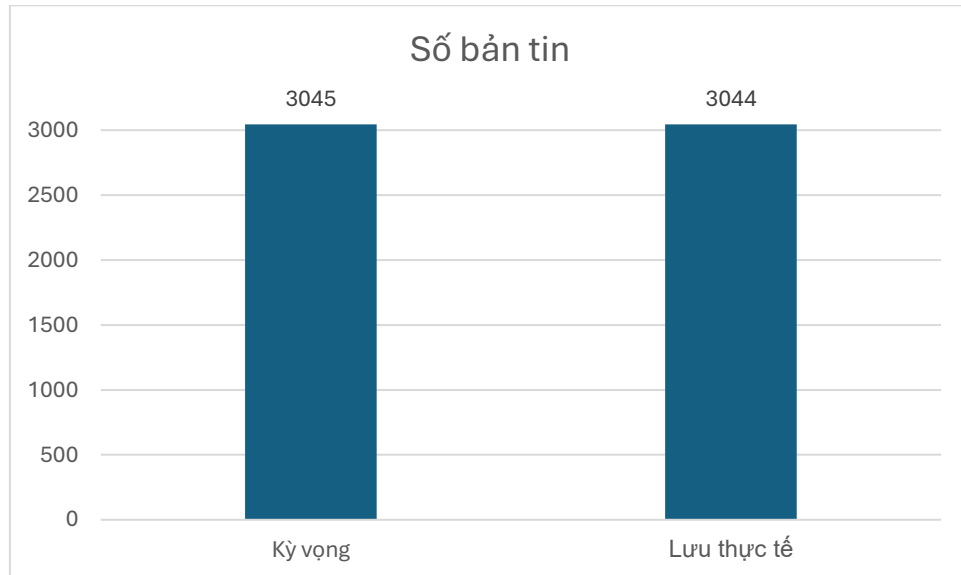
Bảng 4.7: Số lượng bản tin lưu theo từng node

Node	Số bản tin lưu trong <i>mqtt_topic_history</i>
Bếp	1015
Phòng khách	1015
Gara	1014
Tổng	3044

Có thể thấy nút bếp và phòng khách cùng ghi nhận 1015 bản tin, trong khi nút gara chỉ ít hơn đúng 1 bản tin. Sự chênh lệch này hoàn toàn nằm trong giới hạn của sai số thống kê và không thể hiện một xu hướng mất gói tập trung vào một nút cụ thể. Như vậy, không có bằng chứng cho thấy broker hoặc *mqtt_to_sql.py* bị quá tải cục bộ tại bất kỳ khu vực giám sát nào.

Đối với bảng *sensor_readings*, số bản ghi được tạo ra là 608. Con số này trùng khớp với tổng số topic cảnh báo (*canhbao_muc*) mà hệ thống đã xử lý trong thời gian kiểm thử, cho thấy quy trình tổng hợp và lưu trữ dữ liệu cảm biến vẫn vận hành đúng logic ngay cả khi lượng bản tin đầu vào lớn. Pipeline dữ liệu từ MQTT đến MariaDB thông qua *mqtt_to_sql.py* do đó được xác nhận là không bị gián đoạn hay nghẽn cục bộ trong kịch bản này.

Để trực quan hóa kết quả kiểm thử, nhóm sử dụng biểu đồ cột so sánh số bản tin kỳ vọng và số bản tin thực tế được lưu trong *mqtt_topic_history*.



Hình 4.4: Biểu đồ so sánh số bản tin kỳ vọng và số bản tin lưu thực tế trong *mqtt_topic_history*

Biểu đồ cho thấy số bản tin lưu thực tế gần như trùng với số bản tin kỳ vọng. Chênh lệch chỉ 1 bản tin trên tổng 3045 bản tin cho thấy khả năng tiếp nhận và ghi dữ liệu của hệ thống tương đối ổn định trong kịch bản kiểm thử tải.

4.3.4. Nhận xét độ ổn định của hệ thống khi chịu tải

Từ những kết quả thu được, có thể rút ra một số nhận định quan trọng về độ ổn định và khả năng chịu tải của hạ tầng truyền thông.

Thứ nhất, Mosquitto broker trên Raspberry Pi thể hiện năng lực tiếp nhận bản tin đáng tin cậy. Với hơn 3000 bản tin được gửi đến chỉ trong một phút, tỷ lệ thất thoát 0,033% là không đáng kể và hoàn toàn chấp nhận được đối với một hệ thống cảnh báo cháy quy mô nhỏ. Điều này khẳng định mô hình broker cục bộ có thể đáp ứng lưu lượng dữ liệu từ nhiều nút cảm biến đồng thời mà không xảy ra hiện tượng mất gói hàng loạt.

Thứ hai, chương trình trung gian *mqtt_to_sql.py* duy trì được khả năng xử lý liên tục trong suốt quá trình kiểm thử. Nhật ký dịch vụ ghi nhận đầy đủ các topic từ ba khu vực bếp, phòng khách và gara với tất cả năm tham số cảm biến. Điều này chứng tỏ script không chỉ bám sát được luồng dữ liệu từ broker mà còn thực hiện chính xác thao tác ghi vào MariaDB, không gây ra nghẽn hay gián đoạn trong pipeline xử lý.

Thứ ba, cơ sở dữ liệu MariaDB ghi nhận gần như toàn bộ bản tin vào bảng *mqtt_topic_history* (3044/3045), đồng thời vẫn tạo ra đúng 608 bản ghi tổng hợp trong *sensor_readings* – tương ứng với số lần cập nhật cảnh báo từ các nút. Sự nhất quán này cho thấy cả hai tuyến lưu trữ dữ liệu thô lẫn dữ liệu đã qua xử lý đều hoạt động ổn định, không bị ảnh hưởng bởi áp lực từ số lượng bản tin lớn trong thời gian ngắn.

Về mặt tài nguyên, Raspberry Pi 3B+ tuy có cấu hình khiêm tốn (RAM 1 GB, CPU 4 nhân) nhưng không ghi nhận bất kỳ dấu hiệu quá tải nào trong quá trình kiểm thử: Mosquitto không mất kết nối, dịch vụ *fireiot-mqttsql.service* không dừng bất thường, và MariaDB vẫn phản hồi truy vấn một cách bình thường. Tất nhiên, đây mới chỉ là những quan sát định tính; để có bức tranh đầy đủ hơn, các thông số như phần trăm sử dụng CPU, dung lượng RAM tiêu thụ và tốc độ ghi đĩa cần được giám sát định lượng trong các lần phát triển tiếp theo.

Dù vậy, kết quả kiểm thử mới phản ánh điều kiện tải trong phạm vi mô hình thử nghiệm. Khi mở rộng hệ thống với số lượng node lớn hơn, tần suất gửi

dữ liệu cao hơn hoặc thời gian vận hành liên tục dài hơn, Raspberry Pi có thể chịu áp lực lớn hơn về CPU, RAM và tốc độ ghi cơ sở dữ liệu. Vì vậy, trong các phiên bản phát triển tiếp theo, nhóm có thể bổ sung kiểm thử tải dài hạn, đo trực tiếp mức sử dụng CPU/RAM và đánh giá hiệu năng broker trong các kịch bản có quy mô lớn hơn.

Nhìn chung, kịch bản hiện tại cho thấy hệ thống có độ ổn định tốt. Tỷ lệ mất gói rất thấp, dữ liệu gần như được lưu đầy đủ vào *mqtt_topic_history*, bảng *sensor_readings* vẫn tạo bản ghi chính xác theo logic xử lý và không ghi nhận lỗi gián đoạn dịch vụ trong quá trình thử nghiệm. Những kết quả này cho thấy Mosquitto cục bộ, chương trình *mqtt_to_sql.py* và MariaDB đáp ứng được yêu cầu vận hành của một hệ thống cảnh báo cháy IoT quy mô nhỏ.

KẾT LUẬN VÀ ĐỀ NGHỊ

➤ Kết luận

Sau quá trình nghiên cứu và triển khai, đề tài “Xây dựng hạ tầng truyền thông và quản lý dữ liệu cho mạng lưới cảm biến của hệ thống cảnh báo cháy” đã hoàn thành các mục tiêu trọng tâm đề ra. Về mặt hạ tầng, nhóm đã thiết lập thành công Mosquitto MQTT Broker trên Raspberry Pi, tạo ra một trung tâm điều phối dữ liệu ổn định theo mô hình publish/subscribe. Lớp quản lý dữ liệu được vận hành dựa trên MariaDB, hỗ trợ lưu trữ toàn diện lịch sử bản tin, cập nhật trạng thái các khu vực giám sát và trích xuất dữ liệu phục vụ thống kê. Sự kết nối giữa broker và cơ sở dữ liệu được duy trì xuyên suốt bởi chương trình ``mqtt_to_sql.py`` hoạt động như một dịch vụ hệ thống ổn định. Đặc biệt, việc thiết lập cầu nối (bridge) tới HiveMQ Cloud đã hiện thực hóa khả năng giám sát từ xa qua Internet. Với ưu điểm chi phí thấp và cấu hình tinh gọn, hạ tầng này hoàn toàn khả thi để ứng dụng làm máy chủ cục bộ cho các hệ thống IoT quy mô vừa và nhỏ như nhà thông minh, giám sát môi trường hay cảnh báo cháy sớm.

➤ Đề xuất

Để nâng cao tính hoàn thiện và độ tin cậy của hệ thống trong thực tế, các hướng phát triển sau được đề xuất:

Tăng cường bảo mật: Thay thế cơ chế kết nối ẩn danh hiện tại bằng phương thức xác thực người dùng (username/password) và phân quyền truy cập theo từng topic nhằm ngăn chặn các truy cập trái phép.

Phát triển giao diện người dùng: Xây dựng Dashboard trên nền tảng Web hoặc ứng dụng di động để trực quan hóa dữ liệu nhiệt độ, khói, gas qua biểu đồ thời gian thực và cung cấp hệ thống thông báo đẩy cho người dùng.

Tối ưu hóa lưu trữ: Đối với kịch bản vận hành dài hạn, cần áp dụng cơ chế phân vùng bảng dữ liệu, tự động sao lưu và xóa dữ liệu cũ định kỳ hoặc chuyển bớt dữ liệu lên đám mây để giảm tải cho bộ nhớ của Raspberry Pi.

Nâng cao độ tin cậy: Bổ sung tính năng giám sát trạng thái hoạt động của các dịch vụ nội bộ (như script trung gian và kết nối bridge) để kịp thời cảnh báo khi có sự cố mạng hoặc phần mềm, đảm bảo hệ thống luôn sẵn sàng trong mọi tình huống.

TÀI LIỆU THAM KHẢO

Tiếng Việt:

1. Trần Thị Thu Hường (chủ biên), Phạm Anh Tuấn, *Tài liệu học tập Phát triển ứng dụng IoT*, Trường Đại học Kinh tế - Kỹ thuật Công nghiệp, Hà Nội.
2. Intech Group (2025), *Kiến trúc hệ thống IIoT*, <https://intech-group.vn/kien-truc-he-thong-iiot-bv879.htm>, truy cập ngày 2 tháng 3 năm 2026.
3. ITG Technology (2024), *Kiến trúc hệ thống IIoT*, <https://itgtechnology.vn/kien-truc-he-thong-iiot/>, truy cập ngày 2 tháng 3 năm 2026.
4. ITG Technology (2026), *Hệ thống nhúng và IoT là gì? Khái niệm, đặc điểm và ứng dụng*, <https://itgtechnology.vn/he-thong-nhung-va-iot/>, truy cập ngày 2 tháng 3 năm 2026.
5. ITviec (2024), “SQL vs NoSQL: Cách chọn hệ quản trị cơ sở dữ liệu phù hợp”, <https://itviec.com/blog/sql-vs-nosql/>, truy cập ngày 15 tháng 3 năm 2026.
6. PACE (n.d.), *IoT là gì? Ứng dụng của Internet vạn vật*, <https://www.pace.edu.vn/tin-kho-tri-thuc/iot-la-gi>, truy cập ngày 2 tháng 3 năm 2026.
7. QSystems (2026), *Hệ thống IoT là gì? Kiến trúc, mô hình xử lý dữ liệu và ứng dụng*, <https://qsystemsco.com/he-thong-iot.html>, truy cập ngày 2 tháng 3 năm 2026.

Tiếng Anh:

7. Alibaba Cloud (2026), “Subscription options”, *Alibaba Cloud Documentation*, <https://www.alibabacloud.com/help/en/apsaramq-for-mqtt/mqtt-upgraded/developer-reference/subscription-options>, truy cập ngày 6 tháng 3 năm 2026.
8. HiveMQ (2024), *MQTT Essentials – What is an MQTT Broker?*, <https://www.hivemq.com/mqtt-essentials/>, truy cập ngày 2 tháng 3 năm 2026.
9. NodeFlair (2022), “SQL vs NoSQL Databases: System Design Interview”, <https://nodeflair.com/blog/sql-vs-nosql-databases-system-design-interview>, truy cập ngày 15 tháng 3 năm 2026.

10. PUSR IOT (2025), “A Guide to Selecting Communication Protocols for Industrial LTE Routers in Cloud Connectivity”, *PUSR IOT Blog*, <https://www.pusr.com/blog/A-Guide-to-Selecting-Communication-Protocols-for-Industrial-LTE-Routers-in-Cloud-Connectivity>, truy cập ngày 6 tháng 3 năm 2026.
11. TagoIO (2025), “MQTT in Cellular IoT: Deep Dive into Protocol Variants, Advantages, and Real-World Use”, *TagoIO Blog*, <https://tago.io/fr/blog/mqtt-in-cellular-iot-variants-advantages-and-usecases/>, truy cập ngày 6 tháng 3 năm 2026

PHỤ LỤC

a) Độ trễ chặng 1: ESP32 $\rightarrow$ Mosquitto local (Δ_1)*Bảng 4.8: Kết quả tổng hợp độ trễ truyền dữ liệu của chặng 1*

Node	Số lần đo	Khói (ms)	Gas (ms)	Lửa (ms)	Nhiệt độ (ms)
Bếp	1	612	598	624	587
	2	594	605	618	592
	3	607	611	629	584
	4	620	590	613	596
Phòng khách	1	785	763	798	771
	2	792	775	804	765
	3	778	780	791	782
	4	801	759	812	774
Gara	1	671	655	688	658
	2	663	667	676	648
	3	680	660	680	665
	4	674	670	690	652

b) Độ trễ chặng 2: Mosquitto local $\rightarrow$ MariaDB (Δ_2)*Bảng 4.9: Kết quả tổng hợp độ trễ truyền dữ liệu của chặng 2*

Node	Số lần đo	Khói (ms)	Gas (ms)	Lửa (ms)	Nhiệt độ (ms)
Bếp	1	127	118	132	110
	2	115	124	121	108
	3	122	119	129	114
	4	130	116	126	112
Phòng khách	1	134	142	138	129
	2	141	136	145	133
	3	128	139	131	135
	4	137	144	140	130
Gara	1	119	125	121	116
	2	123	118	127	113
	3	117	122	124	120
	4	126	119	118	117

c) Độ trễ chặng 3: Mosquitto local $\rightarrow$ HiveMQ Cloud (Δ_3)*Bảng 4.10: Kết quả tổng hợp độ trễ truyền dữ liệu của chặng 3*

Node	Số lần đo	Khói (ms)	Gas (ms)	Lửa (ms)	Nhiệt độ (ms)
Bếp	1	443	458	432	465
	2	450	449	440	471

	3	436	462	445	459
	4	455	451	438	468
Phòng khách	1	667	680	654	672
	2	674	673	661	685
	3	659	688	670	679
	4	683	676	665	691
Gara	1	508	495	516	502
	2	514	502	509	498
	3	497	510	521	505
	4	521	499	513	511